

Leren en Redeneren

From Statistical Relational to Neural Symbolic AI

Luc De Raedt, Giuseppe Marra
firstname.lastname@kuleuven.be

reusing some slides from previous tutorials with
Angelika Kimmig, Kristian Kersting, David Poole, Robin Manhaeve,
Sebastijan Dumancic, and Sriraam Natarajan

You will find a variation of this tutorial and additional content at

<https://dtai.cs.kuleuven.be/tutorials/nesytutorial>
(after the tutorial)

See also

De Raedt, Dumancic, Marra, Manhaeve
From Statistical Relational to Neuro-Symbolic Artificial Intelligence
IJCAI 20, and long version on AIJ 24

Real-life problems involve two important aspects.



Who can go first ?

- A. The red car
- B. The blue van
- C. The white car

Reasoning

Sub-symbolic perception

**GenAI is great but
their reasoning abilities are
still flawed
(so use GenAI with care)**

Reasoning with LLMs

Some well known examples as to where purely data-driven approaches fail:

“A man and a goat are on one side of the river. They have a boat. How can they both go across?” :

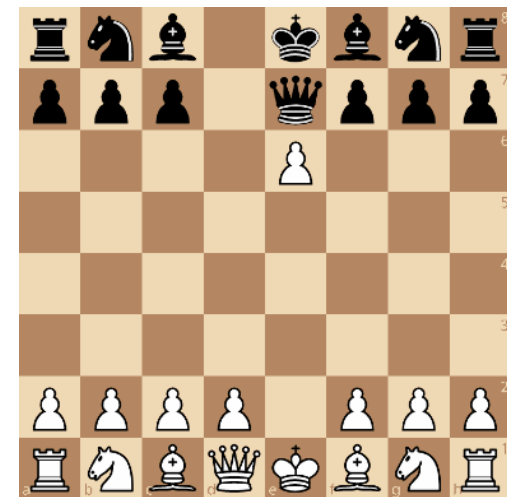
- **ChatGPT**: “The man takes the goat across the river first, leaving the boat on the original side. Then, the man returns alone with the boat. He leaves the goat on the other side and takes the boat back to the original side. Finally, he takes the cabbage across the river.”

“Playing chess as a professional player”

- **GPT 5**: can be tricked into illegal moves (for unusual sequences of move)

“The illusion of thinking”

- **results from Apple**



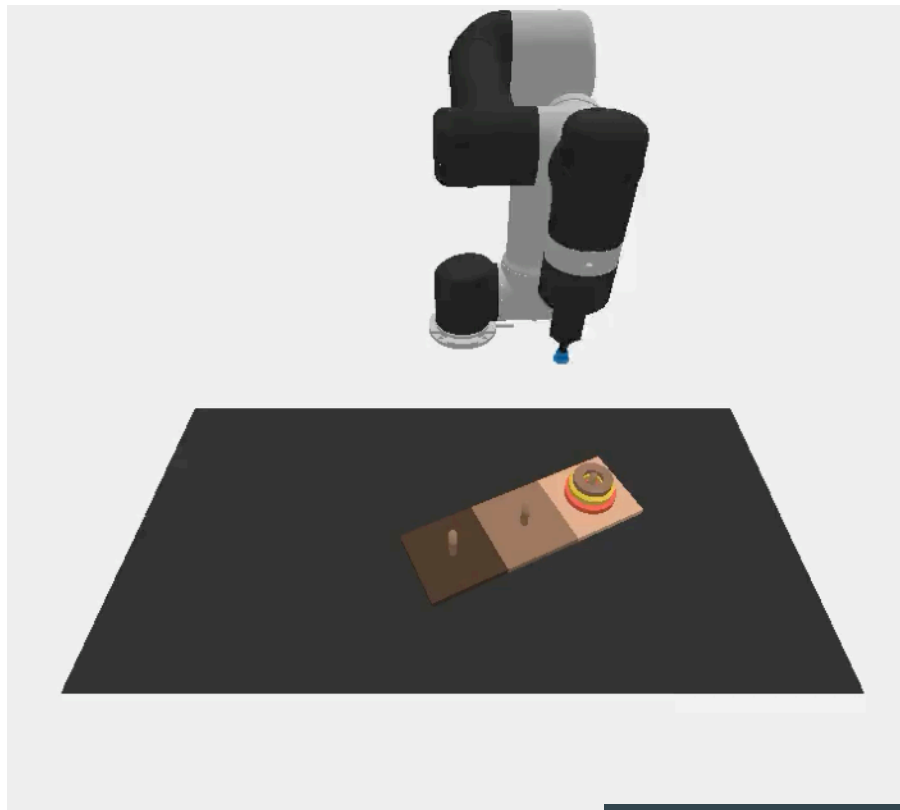
Planning = determining a sequence of actions that will lead to a goal state

See also <https://www.bbc.com/future/article/20240912-what-riddles-teach-us-about-the-human-mind>
<https://blog.mathieuacher.com/GPT5-IllegalChessBench/>
<https://machinelearning.apple.com/research/illusion-of-thinking>
and <https://ai.kuleuven.be/stories/post/2024-09-18-a-learning-and-reasoning-manifesto/> for some details and refs



Reasoning: planning

Planning = determining a sequence of actions that will lead to a goal state



Goal: move the yellow disk in rod 2 [middle rod].

Initial State: brown disk on top of yellow disk. yellow disk on top of red disk. red disk in rod 1. The disks can be moved in rod 1 [light brown], rod 2 [middle rod], rod 3 [dark brown].

Say

Step 1: put brown disk in rod 1

Step 2: put yellow disk in rod 2

Step 3: put red disk in rod 2

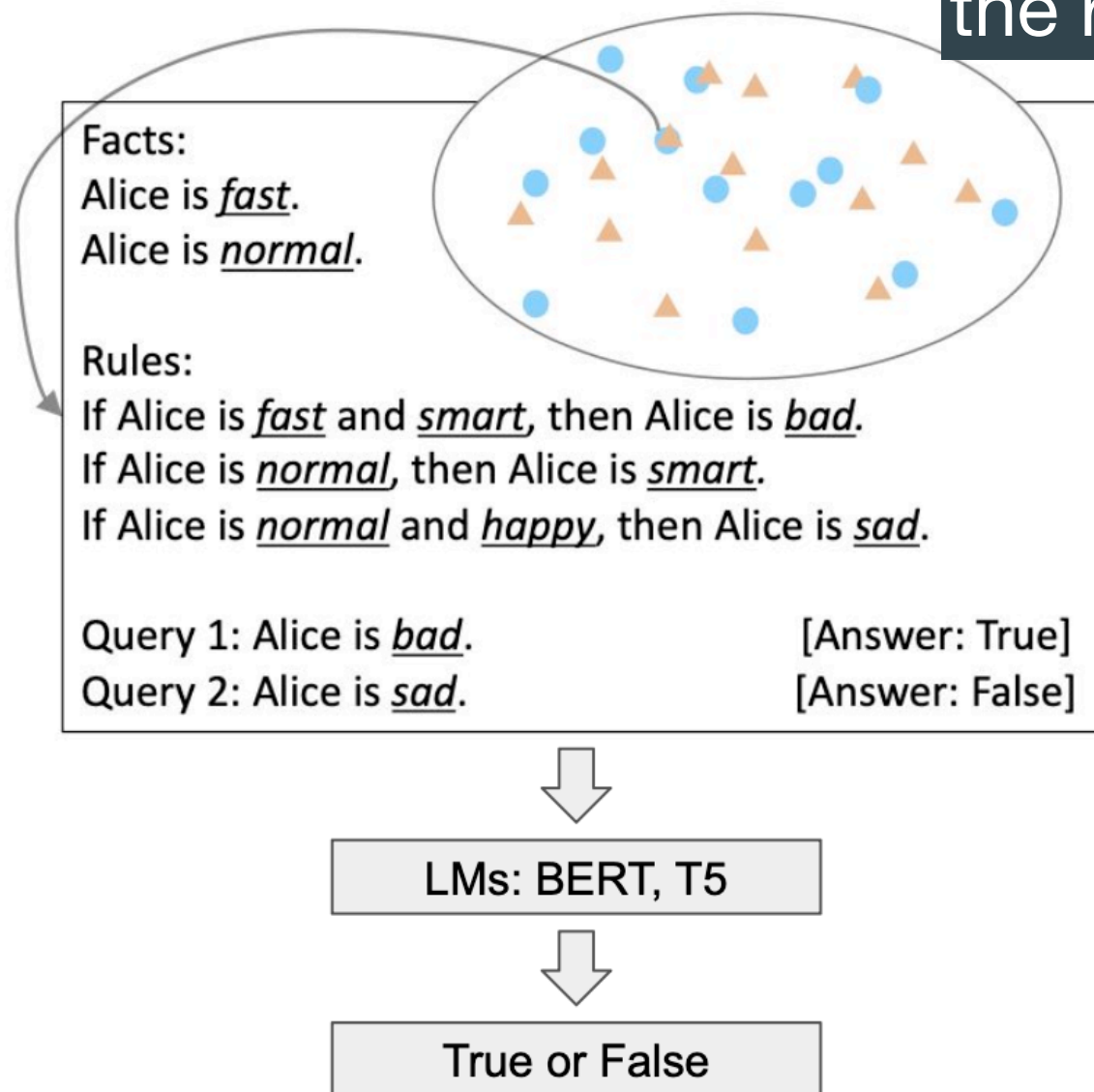
Step 4: done putting disks in rods

the reasoning is flawed

SayCanPay, Hazra et al, AAAI 24

LLMs and SimpleLogic

the reasoning is flawed



Simple tractable setting for logical reasoning (forward reasoning)

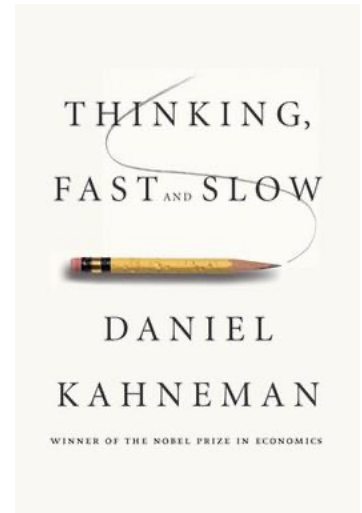
Two datasets (different generation process). Performance excellent if tested on same distribution, performance bad on other dataset

LLM does not learn correctly

Explanation: picks up “statistical features”

Honghua Zhang, Liunian Harold Li, Tao Meng, Kai-Wei Chang and Guy Van den Broeck. [On the Paradox of Learning to Reason from Data](#), ICML 2022

Learning and Reasoning both needed

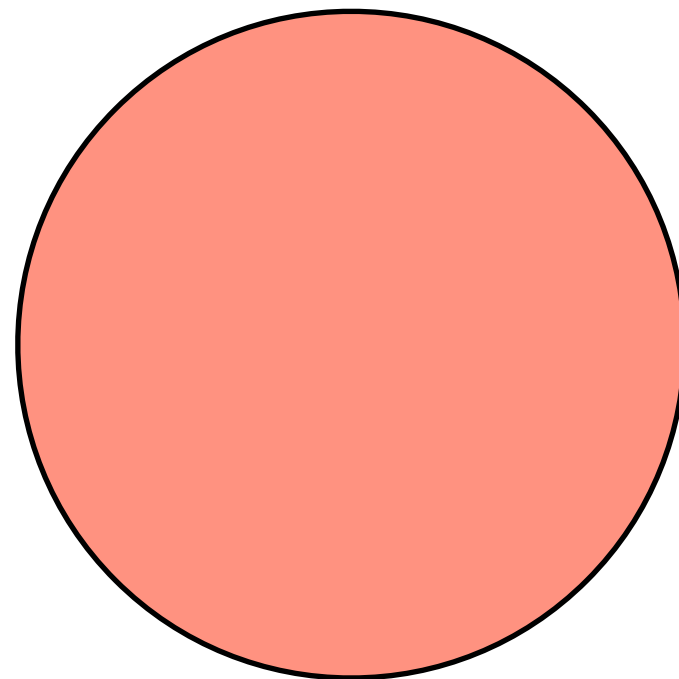


- System 1 - thinking fast - can do things like $2+2 = ?$ and recognise objects in image
- System 2 - thinking slow - can reason about solving complex problems - planning a complex task
- alternative terms — data-driven vs knowledge-driven, symbolic vs subsymbolic, solvers and learners, neuro-symbolic...
- **A lot of work on integrating learning and reasoning, neural symbolic computation to integrate logic / symbols reasoning with neural networks**

- see also arguments
by Marcus, Darwiche, Levesque, Tenenbaum, Geffner,
Bengio, Le Cun, Kautz, ...
see also AI Debates

Thinking fast

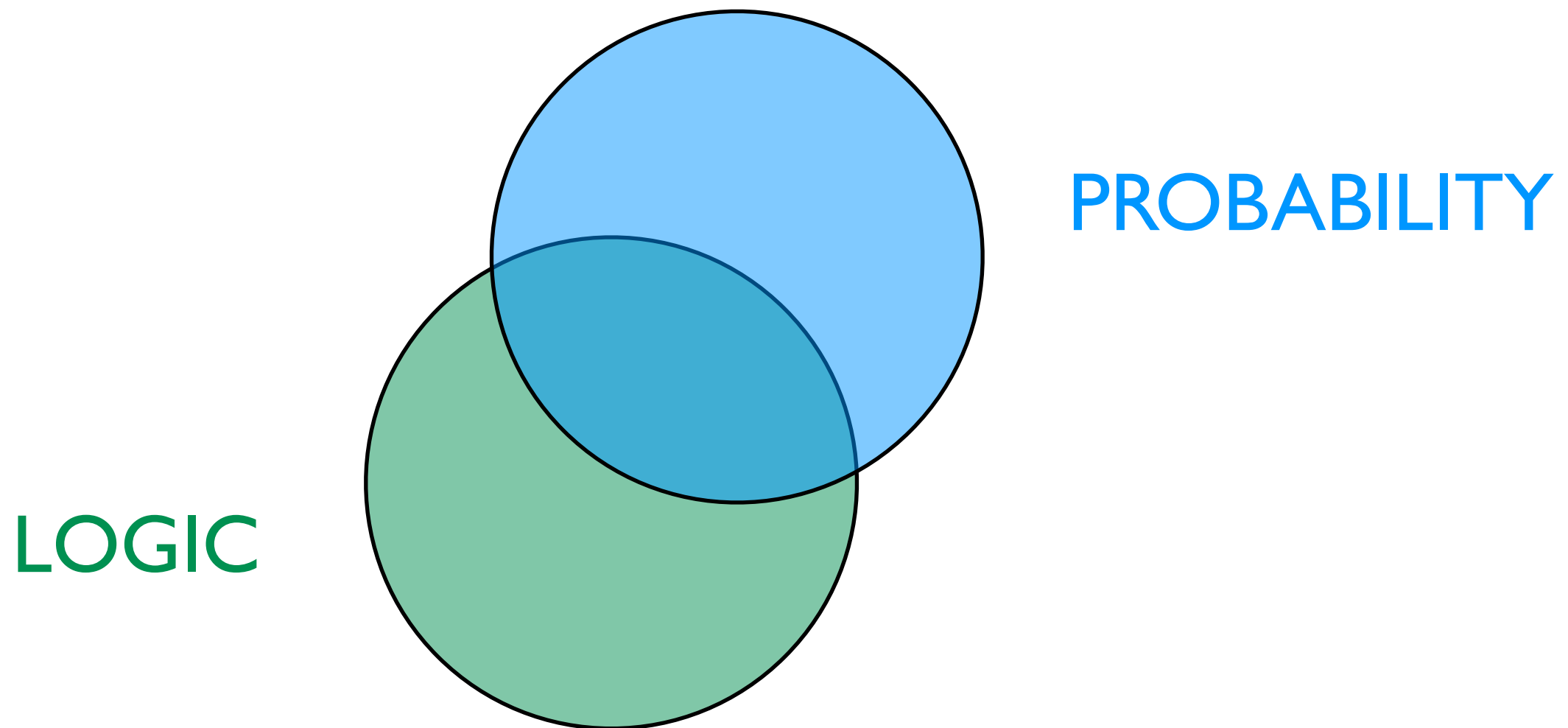
MAIN PARADIGM in AI
Focus on Learning



NEURAL

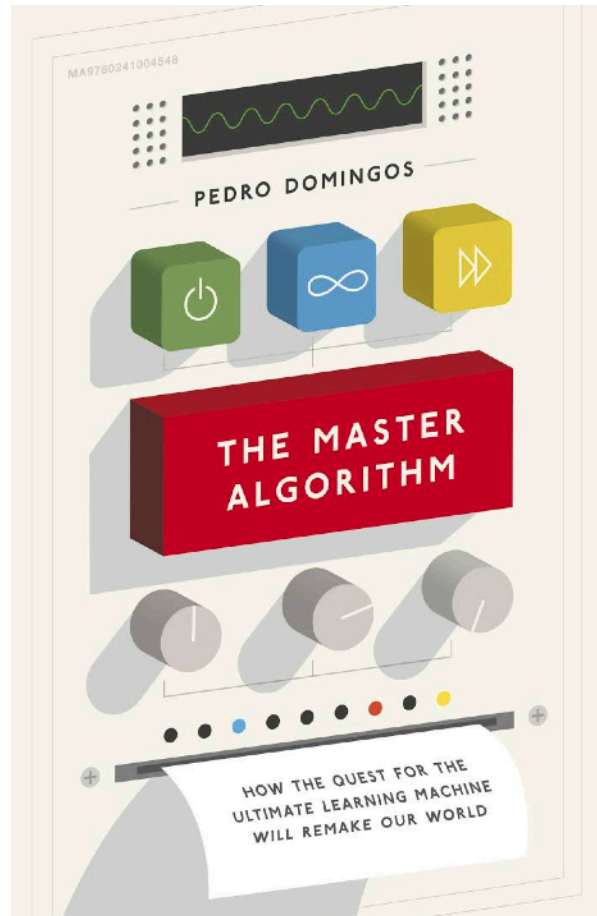
Thinking slow = reasoning

TWO MAIN PARADIGMS in AI

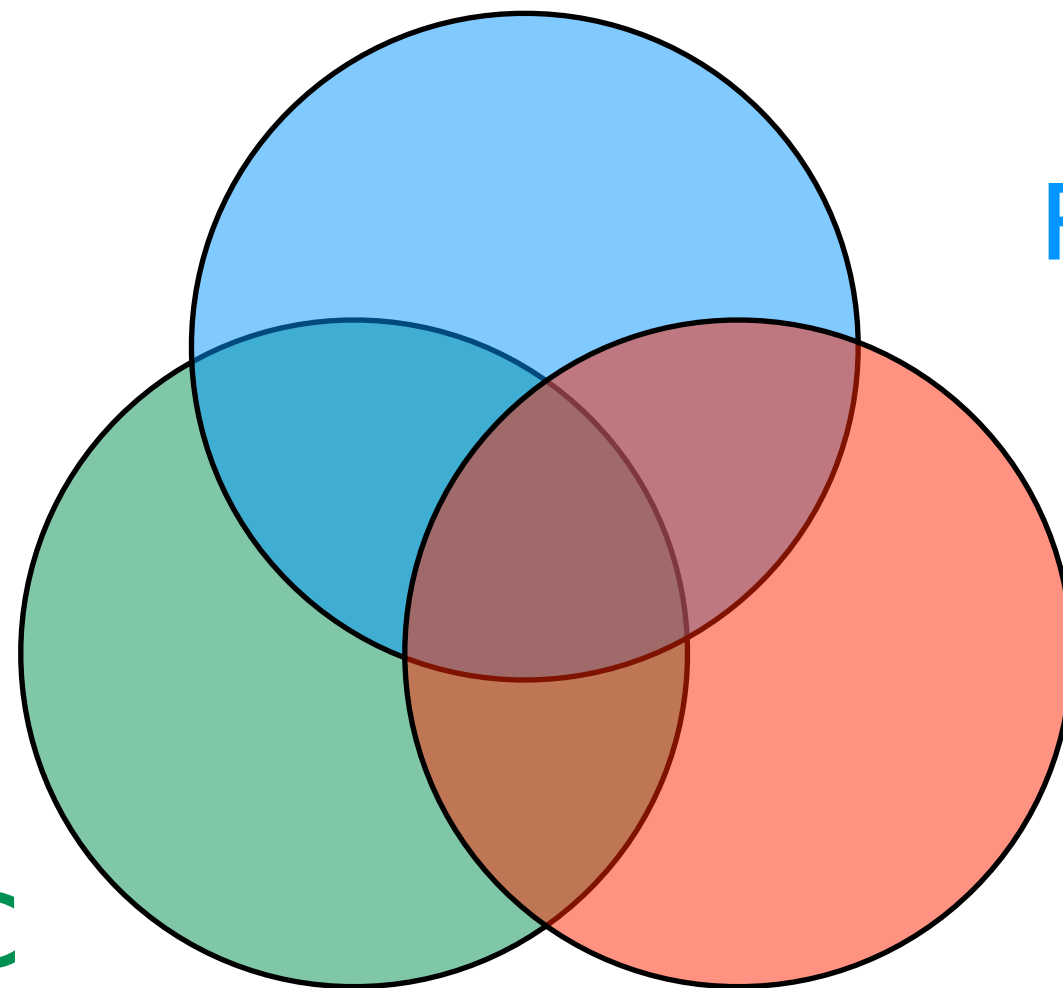


**Their integration has been well studied in
Probabilistic (Logic) Programming and Statistical Relational AI (StarAI)**

Learning



LOGIC



PROBABILITY

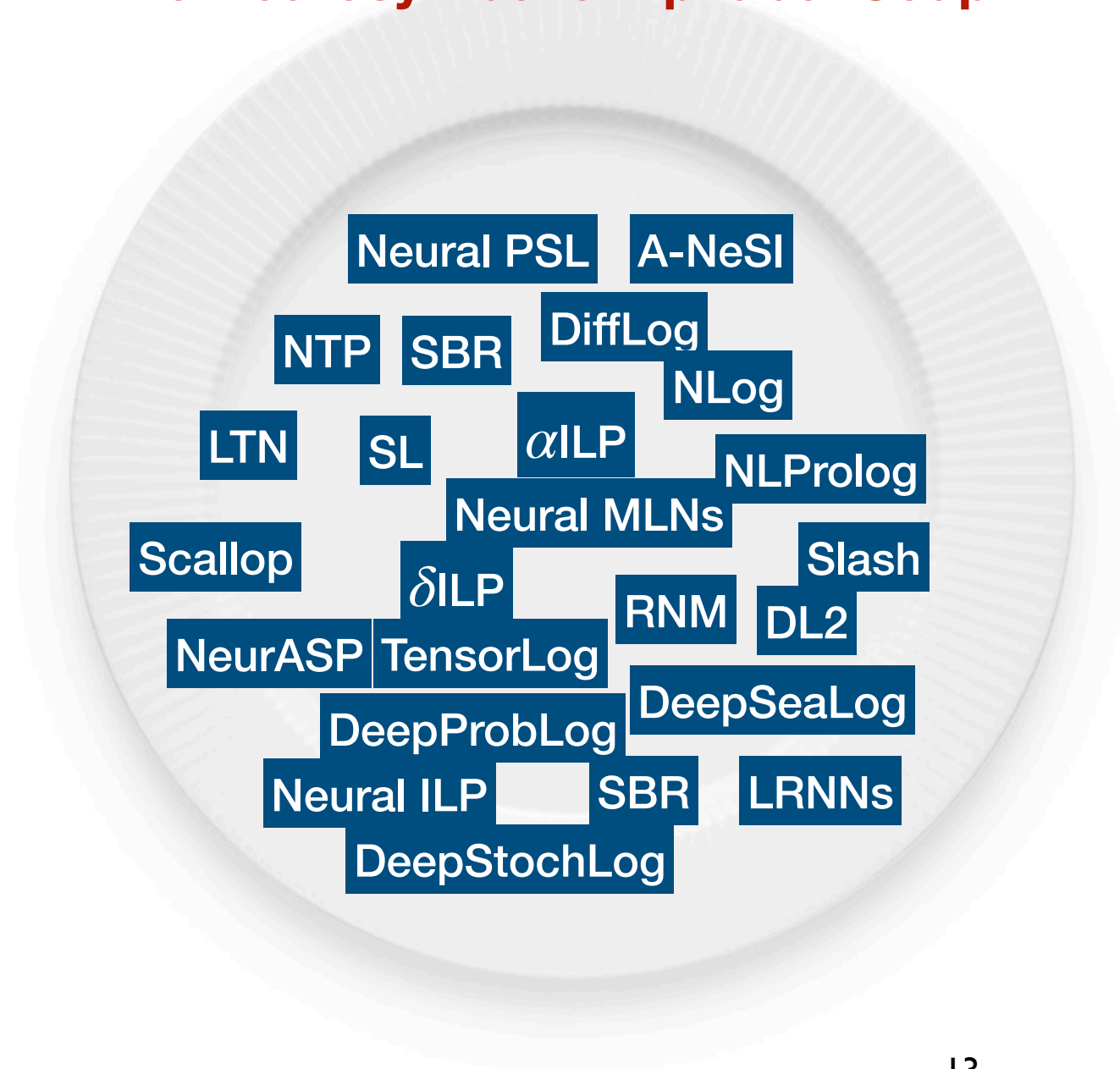
NEURAL

How to integrate these three paradigms in AI ?

A tutorial on Neurosymbolic AI

Towards a theory of neurosymbolic AI

The Neurosymbolic Alphabet Soup



Neurosymbolic Alchemy

Strong focus on **developing specific architectures** that score well on **particular benchmarks**

we want to **develop the scientific foundations for neurosymbolic AI**

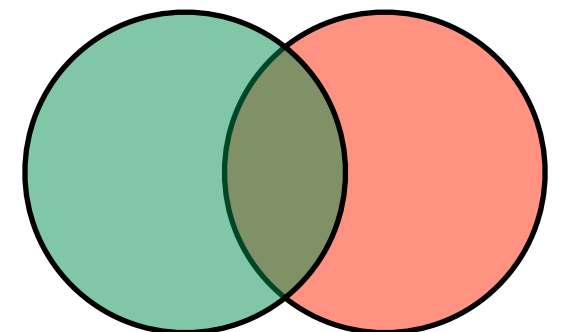
Neurosymbolic AI =



Neural + Logic

Traditional View

	Neural	Logic
perception	✓	
scalable	✓	
learning	✓	
reasoning		✓
explanations		✓
knowledge		✓



LOGIC NEURAL

Neurosymbolic AI =

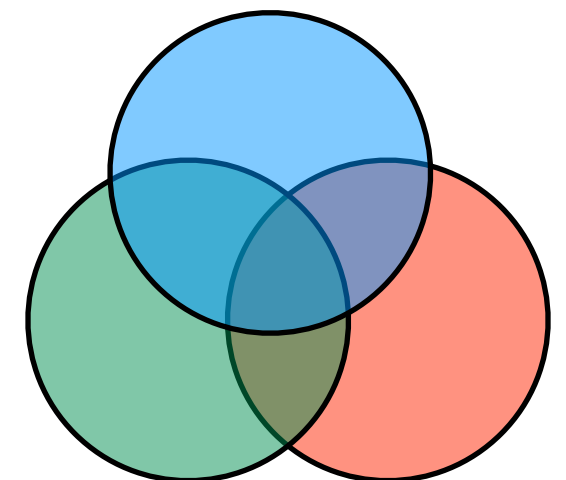


Neural + Logic + Probability

	Neural	Logic	Probability
perception	✓		
scalable	✓		
learning	✓		✓
reasoning		✓	✓
explanations		✓	✓
knowledge		✓	

Our Slogan

The necessary quantitative component

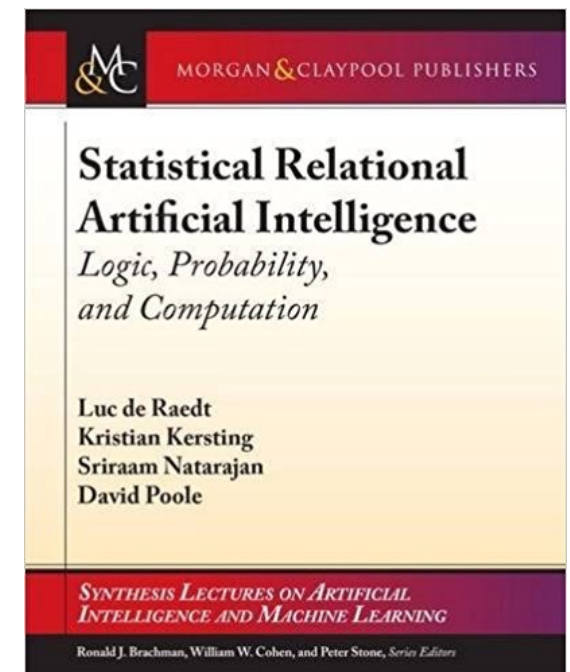


Probability

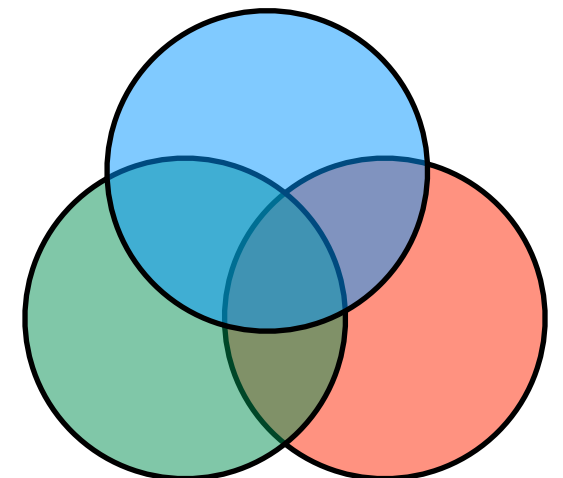
Neurosymbolic AI =

Neural + Logic + Probability

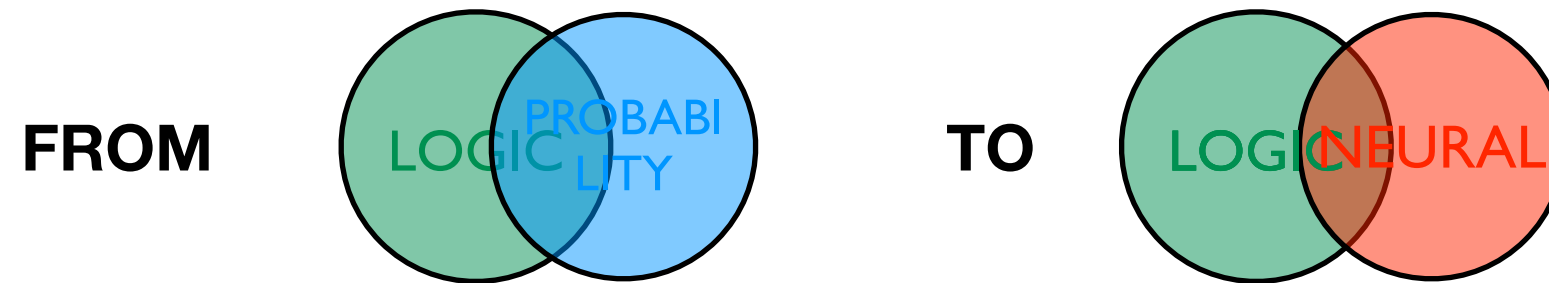
	Neural	Logic	Probability
perception	✓		
scalable	✓		
learning	✓		✓
reasoning		✓	✓
explanations		✓	✓
knowledge		✓	



Statistical Relational AI (StarAI)
→ methodology for Neurosymbolic AI



Key Message



**StarAI and NeSy share similar problems
and thus similar solutions apply**

See also

De Raedt, Dumancic, Marra, Manhaeve

From Statistical Relational to Neuro-Symbolic Artificial Intelligence

IJCAI 20, and long version on AIJ 24

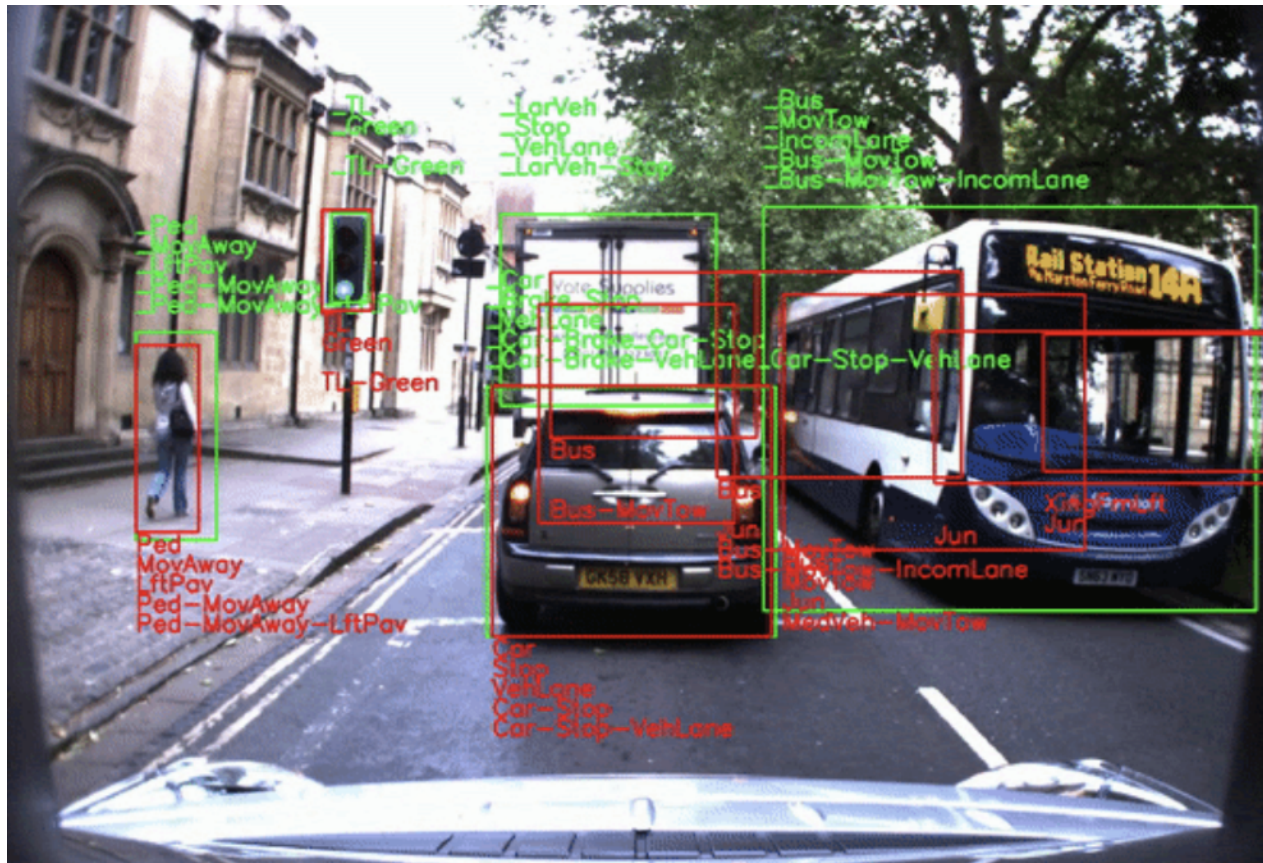
What is missing today ?

Knowledge compensates for data

- Today's standard paradigm is
 - do not use knowledge, only data
- At best, if one has knowledge
 - use it to generate data or simulate
 - relearn the knowledge from data with a neural net ?
- **Why not use knowledge or solvers directly ?**
 - seems much more natural, efficient and sustainable
 - provide guarantees



ROAD-R: The autonomous driving dataset with logical requirements



Natural Language Explanations

If an agent pushes an object then it is a pedestrian

A pedestrian can only push objects, move away, etc.

Only pedestrians, cars, cyclists, etc. can cross from left

Only pedestrians and cyclists can wait to cross

Only pedestrians, cars, cyclists, etc can stop

Only pedestrians, cars, cyclists, etc can move

Only pedestrians, cars, cyclists, etc can move towards

Only pedestrians, cars, cyclists, etc can move away

An emergency vehicle can only overtake, move away etc.

Only emergency vehicles, cars etc. can have hazards lights on

A bus can only overtake, move away move towards etc.

A medium vehicle can only overtake, move away, move towards etc.

Giunchiglia, Eleonora, Mihaela Cătălina Stoian, Salman Khan, Fabio Cuzzolin, and Thomas Lukasiewicz. "ROAD-R: The autonomous driving dataset with logical requirements." *Machine Learning* (2023): 1-31.

Both StarAI and NeSy

- Structured environments
 - objects, and
 - **relationships** amongst them
- and possibly
 - using **background knowledge**
- cope with **uncertainty and/or perception**
- **learn from data and reason with knowledge**

Power of Logic
Of Programs

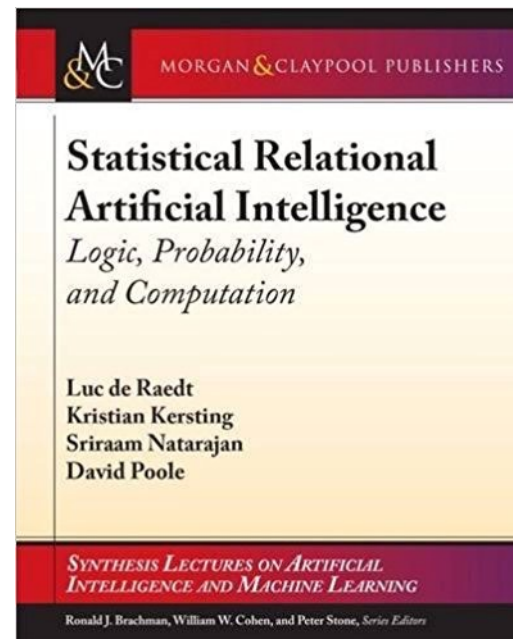
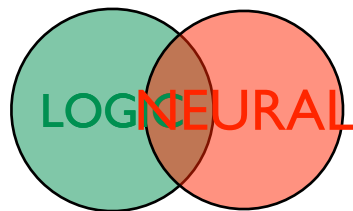
Two Types of Logical, Probabilistic, and Neurosymbolic Approaches

Two types of Neurosymbolic Systems

Logic in the architecture

Logic as a kind
of *neural program*

directed probabilistic
graphical models
(PGMs)



Just like in
StarAI

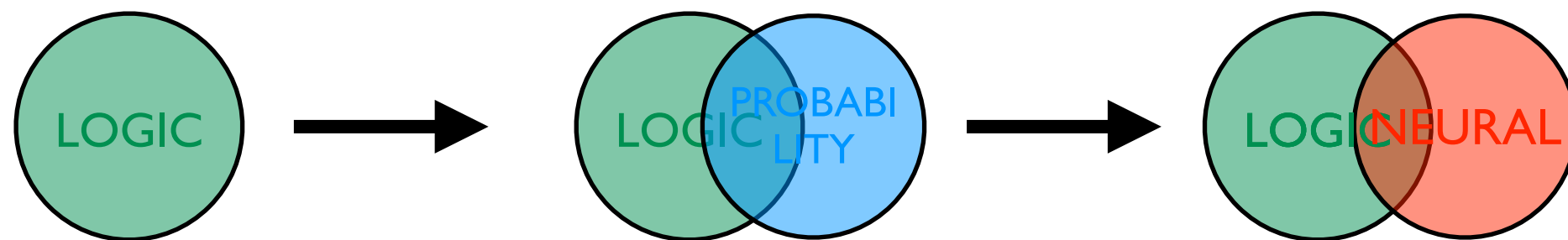
Logic in the loss

Logic as the
regularizer
(reminiscent of Markov
Logic Networks)

undirected
probabilistic
graphical models
(PGMs)



Dimension 1: Proof vs Model based



1. Proof vs Model based the logic dimension

- Model- vs proof-based
- First order / relational vs propositional
- Grounding
- Differences important for both StarAI and NeSY

Logic Programs

as in the programming language Prolog

Propositional logic program

```
burglary.  
hears_alarm_mary.
```

```
earthquake.  
hears_alarm_john.
```

facts :
burglary = true

```
alarm :- earthquake.
```

```
alarm :- burglary.
```

```
calls_mary :- alarm, hears_alarm_mary.
```

```
calls_john :- alarm, hears_alarm_john.
```

Logic Programs

as in the programming language Prolog

Propositional logic program

burglary.
hears_alarm_mary.

earthquake.
hears_alarm_john.

alarm :- earthquake.

alarm :- burglary. **rule:**
calls_mary = true IF alarm = true AND hears_alarm_mary = true

calls_mary :- alarm, hears_alarm_mary.

calls_john :- alarm, hears_alarm_john.

Logic Programs

as in the programming language Prolog

Propositional logic program

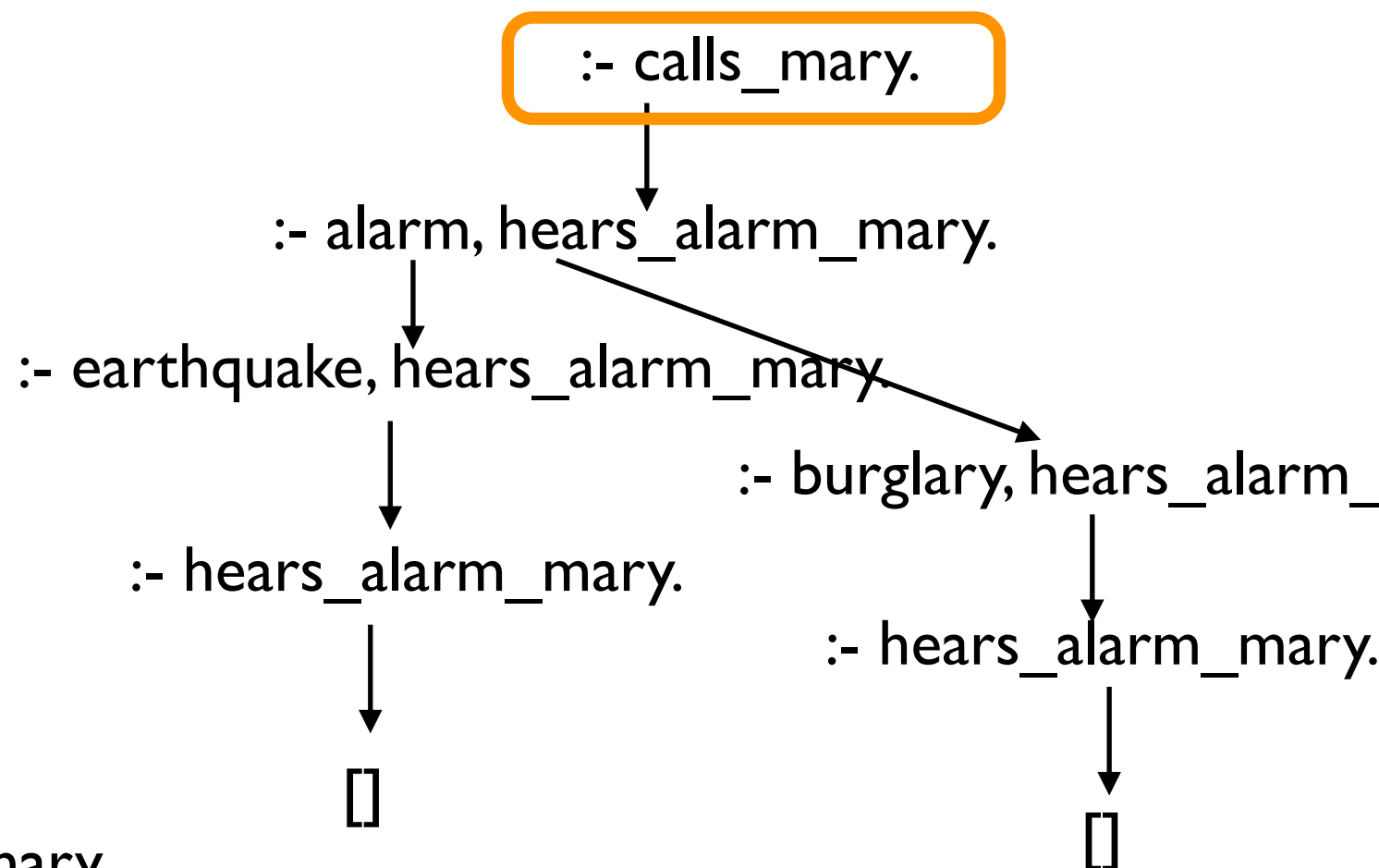
Two proofs (by refutation)

burglary.
hears_alarm_mary.

earthquake.
hears_alarm_john.

alarm :- earthquake.
alarm :- burglary.

calls_mary :- alarm, hears_alarm_mary.
calls_john :- alarm, hears_alarm_john.



A proof-theoretic view
backward chaining

Logic as constraints

as in SAT solvers

Propositional logic

Model / Possible World

IF

AND

$\text{calls}(\text{mary}) \leftarrow \text{hears_alarm}(\text{mary}) \wedge \text{alarm}$

$\text{calls}(\text{john}) \leftarrow \text{hears_alarm}(\text{john}) \wedge \text{alarm}$

OR

$\text{alarm} \leftarrow \text{earthquake} \vee \text{burglary}$

{ burglary,
hears_alarm(john),
alarm,
calls(john)}

the facts that are true
in this model / possible world

SAT: Find a model / possible world that satisfies all the constraints

SAT SOLVERS

A model-theoretic view

LOGIC

Relational/First Order Logic

Introduce Variables and Domains

The meaning of this is always the GROUNDED theory

allows to exploit symmetries / templates ...

burglary.
hears_alarm(**mary**).

earthquake.
hears_alarm(**john**).

alarm :- earthquake.

alarm :- burglary.
calls(**X**) :- alarm, hears_alarm(**X**).

Variable X

Domain = {mary, john}

burglary.
hears_alarm(mary).

earthquake.
hears_alarm(john).

alarm :- earthquake.

alarm :- burglary.
calls(mary) :- alarm, hears_alarm(mary).
calls(john) :- alarm, hears_alarm(john).

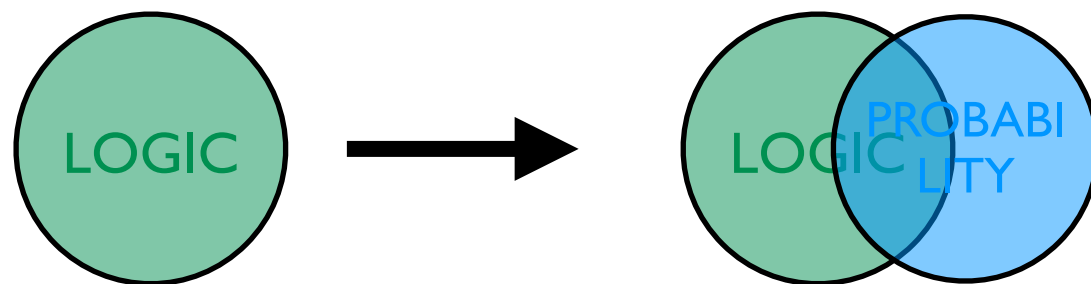
Grounded Theory

BOTH for model and proof-based approach

1. Proof vs Model based the logic dimension

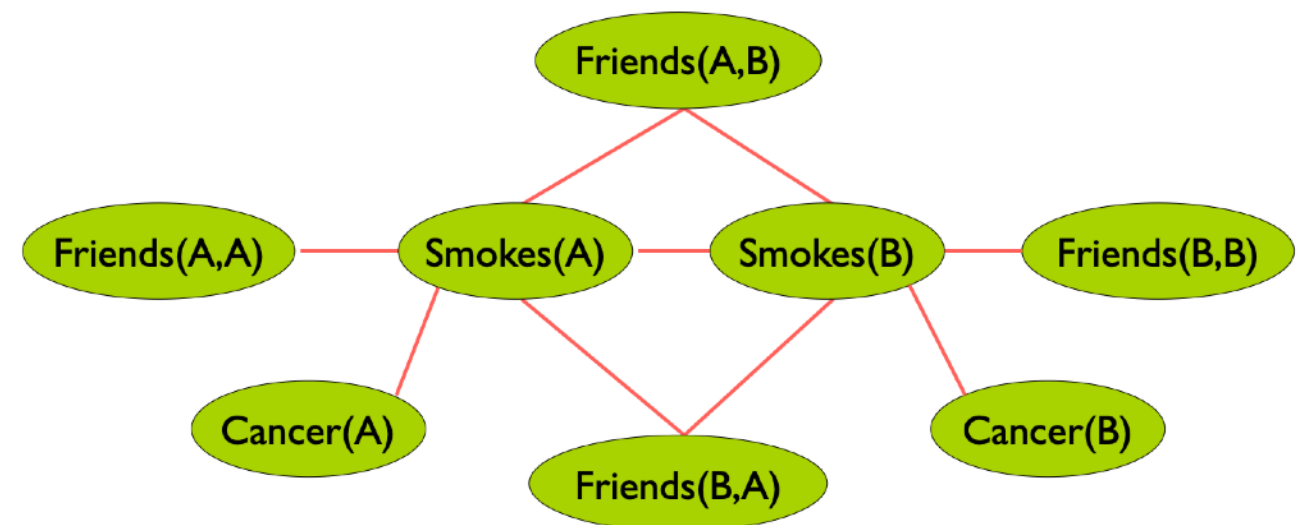
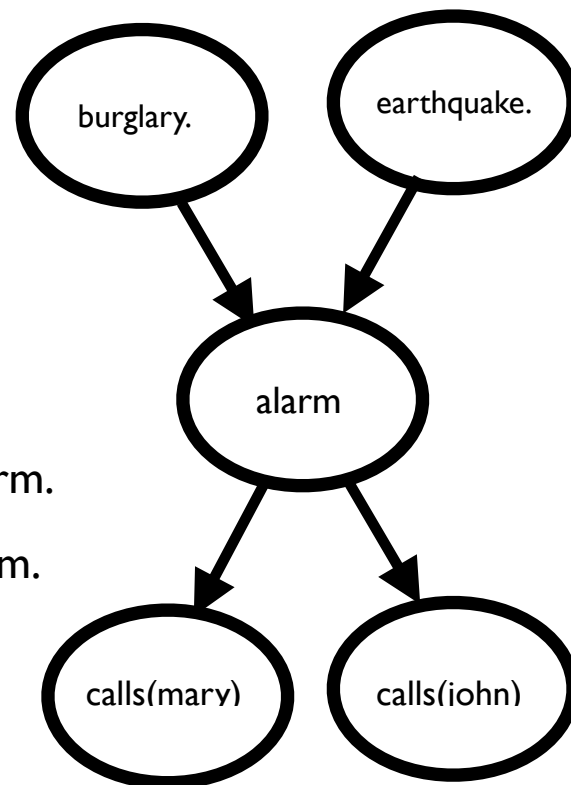
- Model- vs proof-based
- First order / relational vs propositional
- Grounding
- Differences important for both StarAI and NeSY

Dimension 2: Directed vs Undirected Graphical Model



2. Directed vs Undirected the PGM / StarAI dimension

0.1 :: burglary.
 0.05 :: earthquake.
 alarm :- earthquake.
 alarm :- burglary.
 0.7::calls(mary) :- alarm.
 0.6::calls(john) :- alarm.



$$1.5 \quad \forall x \text{ Smokes}(x) \Rightarrow \text{Cancer}(x)$$

$$1.1 \quad \forall x, y \text{ Friends}(x, y) \Rightarrow (\text{Smokes}(x) \Leftrightarrow \text{Smokes}(y))$$

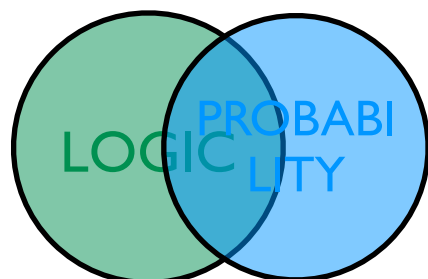
Probabilistic Logic Programs
ProbLog

directed
 Bayesian Net

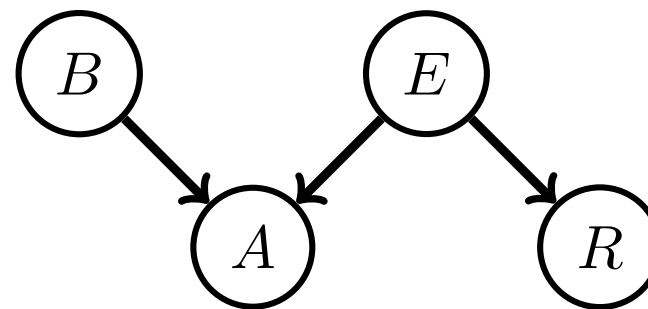
Markov Logic

undirected
 Markov Net
 model theoretic

key representatives



Bayesian Net



$$\mathbf{P}(A|B, E)$$

alarm (= true)	Burglar	Earthquake
0.9999	true	true
0.99	true	false
0.99	false	true
0.0001	false	false

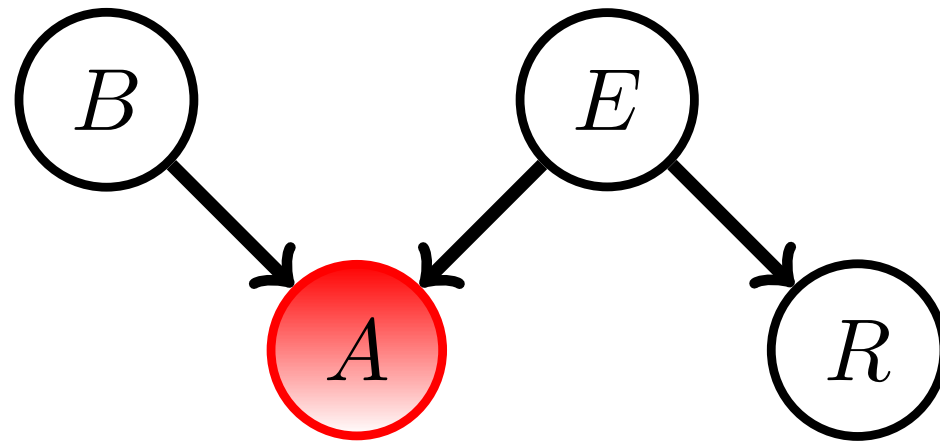
$$\mathbf{P}(R|E)$$

radio	Earthquake
1	true
0	false

The remaining tables are $P(b) = 0.01$ and $P(e) = 0.000001$. The tables and graphical structure fully specify the joint distribution $\mathbf{P}(A, R, E, B)$.

Queries

Initial evidence: The alarm is sounding



$$\begin{aligned} P(b|a) &= \frac{P(b, a)}{P(a)} = \frac{\sum_{e,r} P(b, e, a, e)}{\sum_{b,e,r} P(b, e, a, r)} \\ &= \frac{\sum_{e,r} P(r|b, e)P(b)P(e)P(r|e)}{\sum_{b,e,r} P(a|b, e)P(b)P(e)P(r|e)} \approx 0.99 \end{aligned}$$

Logic Programs

as in the programming language Prolog

Propositional logic program

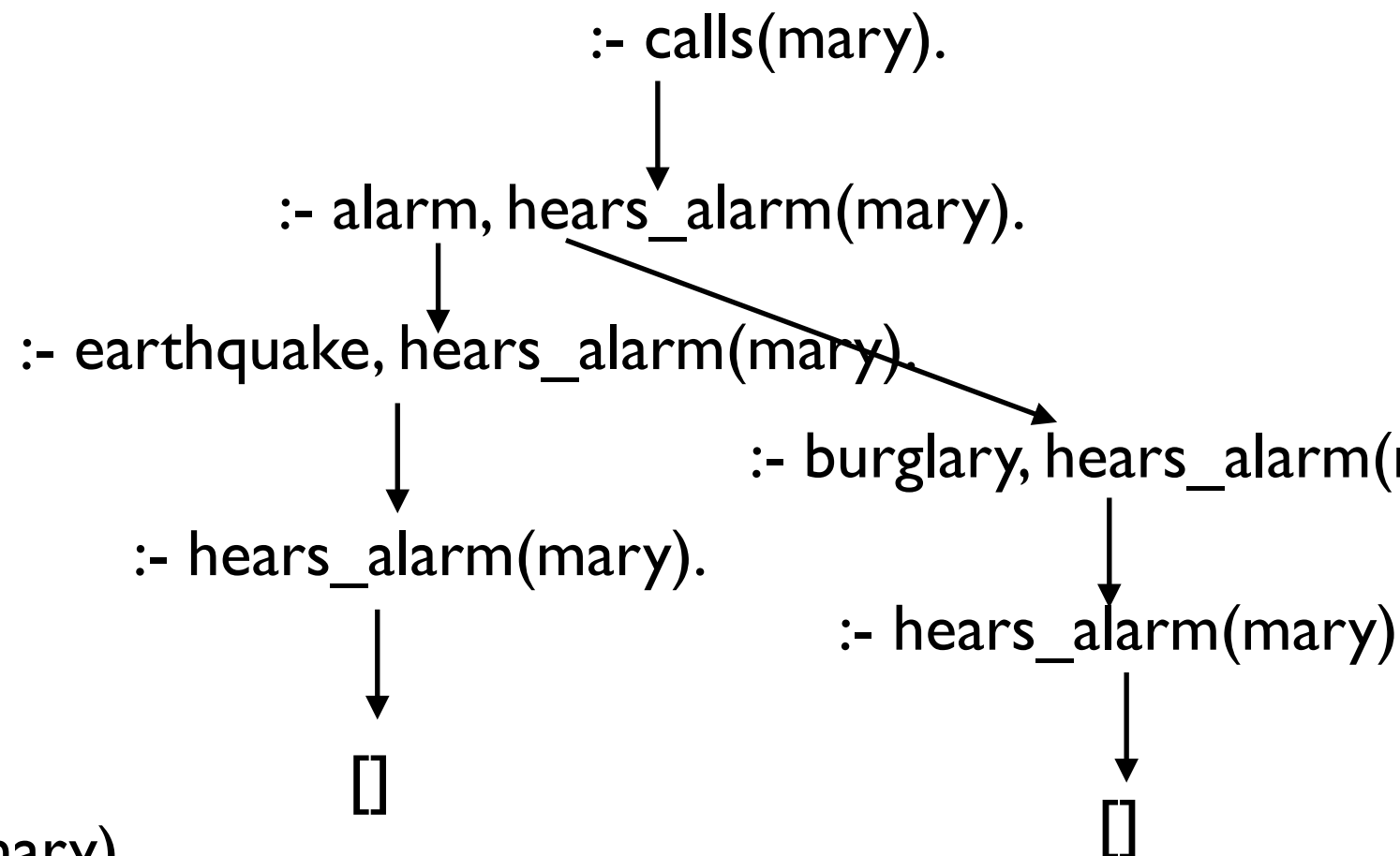
burglary.
hears_alarm(mary).

earthquake.
hears_alarm(john).

alarm :- earthquake.
alarm :- burglary.

calls(mary) :- alarm, hears_alarm(mary).
calls(john) :- alarm, hears_alarm(john).

Two proofs (by refutation)



A proof-theoretic view

Probabilistic Logic Programs

as in the probabilistic programming language ProbLog

Propositional logic program

0.1 :: burglary.
0.3 :: hears_alarm(mary).

Probabilistic facts

0.05 :: earthquake.
0.6 :: hears_alarm(john).

alarm :- earthquake.

alarm :- burglary.

calls(mary) :- alarm, hears_alarm(mary).

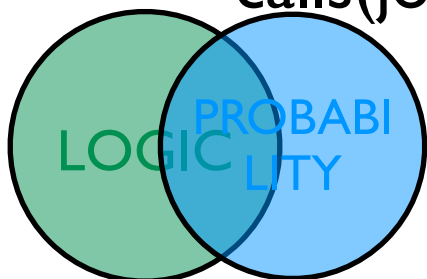
calls(john) :- alarm, hears_alarm(john).

Key Idea (Sato & Poole)
the distribution semantics:

**unify the basic concepts in logic
and probability:**

**random variable ~ propositional
variable**

**an interface between logic and
probability**



Probabilistic Logic Programs

as in the probabilistic programming language ProbLog

Propositional logic program

0.1 :: burglary.
0.3 :: hears_alarm(mary).

0.05 :: earthquake.
0.6 :: hears_alarm(john).

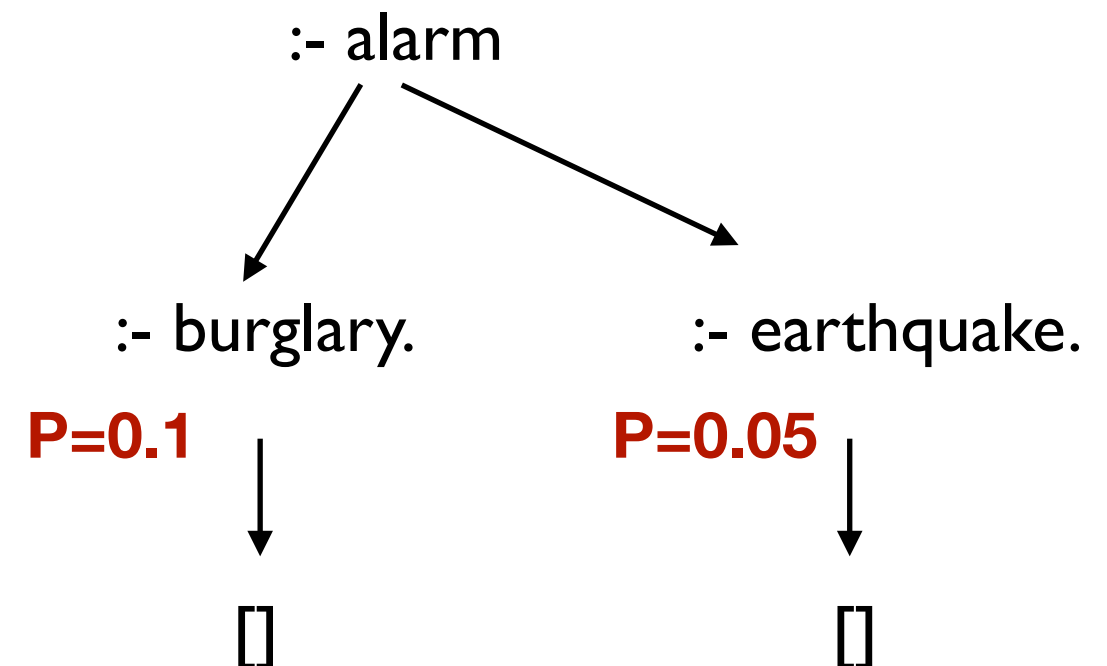
alarm :- earthquake.

alarm :- burglary.

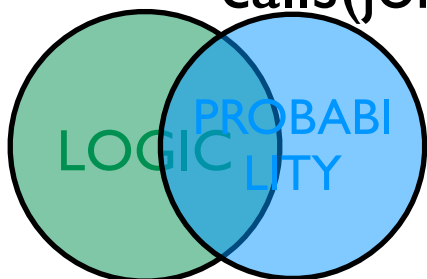
calls(mary) :- alarm, hears_alarm(mary).

calls(john) :- alarm, hears_alarm(john).

Two proofs (by refutation)



Probability of one proof : $\prod_{f: fact \in Proof} P_f$



Probabilistic Logic Programs

as in the probabilistic programming language ProbLog

Propositional logic program

0.1 :: burglary.
0.3 :: hears_alarm(mary).

0.05 :: earthquake.
0.6 :: hears_alarm(john).

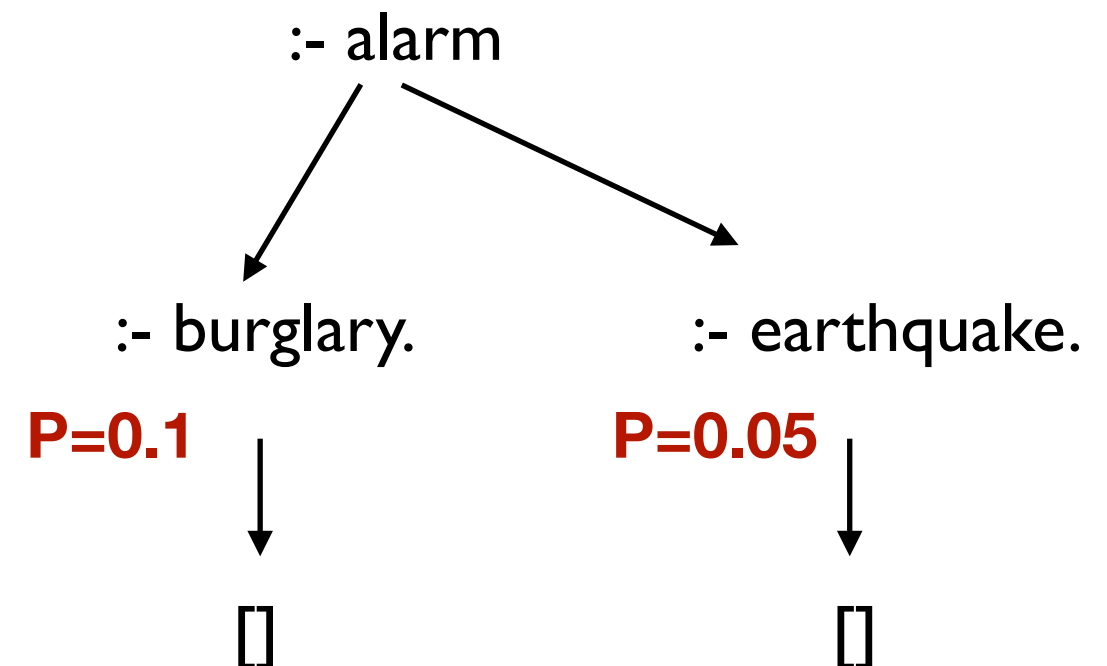
alarm :- earthquake.

alarm :- burglary.

calls(mary) :- alarm, hears_alarm(mary).

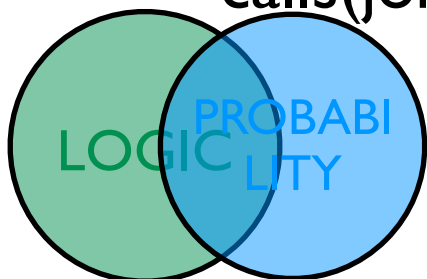
calls(john) :- alarm, hears_alarm(john).

Disjoint sum problem



Probability of one proof : $\prod_{f: fact \in Proof} P_f$

$$\begin{aligned} P(\text{alarm}) &= P(\text{burg OR earth}) \\ &= P(\text{burg}) + P(\text{earth}) - P(\text{burg AND earth}) \\ &\neq P(\text{burg}) + P(\text{earth}) \end{aligned}$$



Distribution Semantics

(with probabilistic facts)

[Sato, ICLP 95]

query

sum over possible worlds
where Q is true

$$P(Q) = \sum_{F \cup R \models Q} \prod_{f \in F} p(f) \prod_{f \notin F} 1 - p(f)$$

subset of
probabilistic
facts

Prolog
rules

probability of
possible world

Probabilistic Logic Semantics

Problog

e.g. in Problog:

B	E	H	p(B,E,H)
F	F	F	0,342
F	F	T	0,513
F	T	F	0,018
F	T	T	0,027
T	F	F	0,038
T	F	T	0,057
T	T	F	0,002
T	T	T	0,003

0.1 :: burglary. (B)

0.05 :: earthquake. (E)

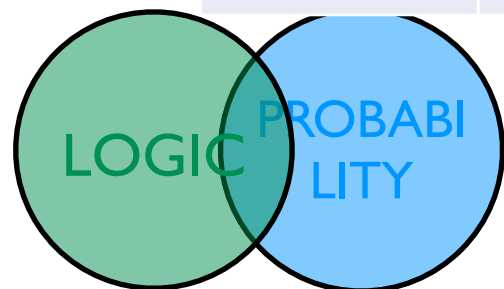
0.6 :: hears_alarm(john). (H)

alarm :- earthquake.

alarm :- burglary.

calls(john) :- alarm, hears_alarm(john)

$$0.1 \times 0.05 \times (1 - 0.6)$$



Probabilistic Logic Semantics

For example:

B	E	H	p(B,E,H)
F	F	F	0,342
F	F	T	0,513
F	T	F	0,018
F	T	T	0,027
T	F	F	0,038
T	F	T	0,057
T	T	F	0,002
T	T	T	0,003

0.1 :: burglary. (B)

0.05 :: earthquake. (E)

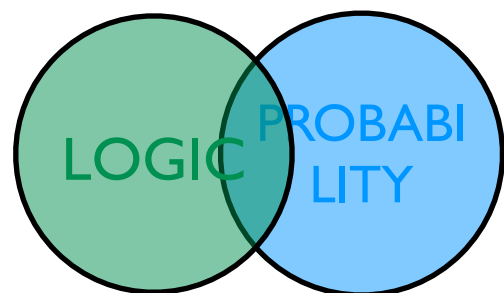
0.6 :: hears_alarm(john). (H)

alarm :- earthquake.

alarm :- burglary.

$$Alarm = (B \vee E)$$

$$P(Alarm) = 1 - 0,513 - 0,342$$



Probabilistic Logic Semantics

For example:

B	E	H	p(B,E,H)
F	F	F	0,342
F	F	T	0,513
F	T	F	0,018
F	T	T	0,027
T	F	F	0,038
T	F	T	0,057
T	T	F	0,002
T	T	T	0,003

0.1 :: burglary. (B)

0.05 :: earthquake. (E)

0.6 :: hears_alarm(john). (H)

alarm :- earthquake.

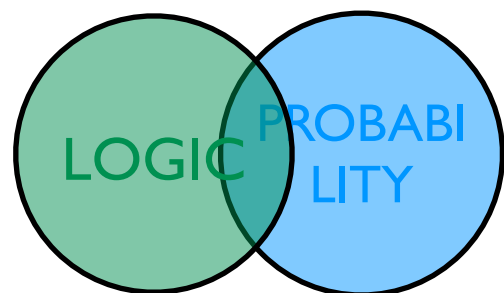
alarm :- burglary.

calls(john) :- alarm, hears_alarm(john)

Query = burglary ^ hears_alarm(john)

$$Q = B \wedge H$$

$$p(Q) = 0.06$$



Probabilistic Logic Semantics

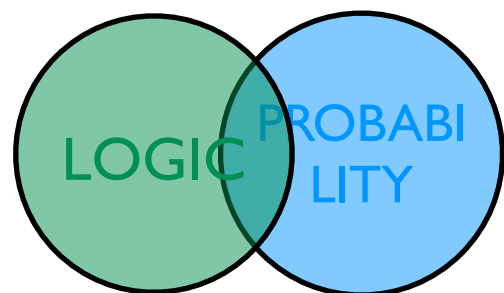
For example:

B	E	H	p(B,E,H)
F	F	F	0,342
F	F	T	0,513
F	T	F	0,018
F	T	T	0,027
T	F	F	0,038
T	F	T	0,057
T	T	F	0,002
T	T	T	0,003

0.1 :: burglary. (B)
 0.05 :: earthquake. (E)
 0.6 :: hears_alarm(john). (H)
 alarm :- earthquake.
 alarm :- burglary.

$$Q = (B \wedge H) \vee E$$

$$P(Q) = 0.105$$



Probabilistic Logic Program Semantics

earthquake.

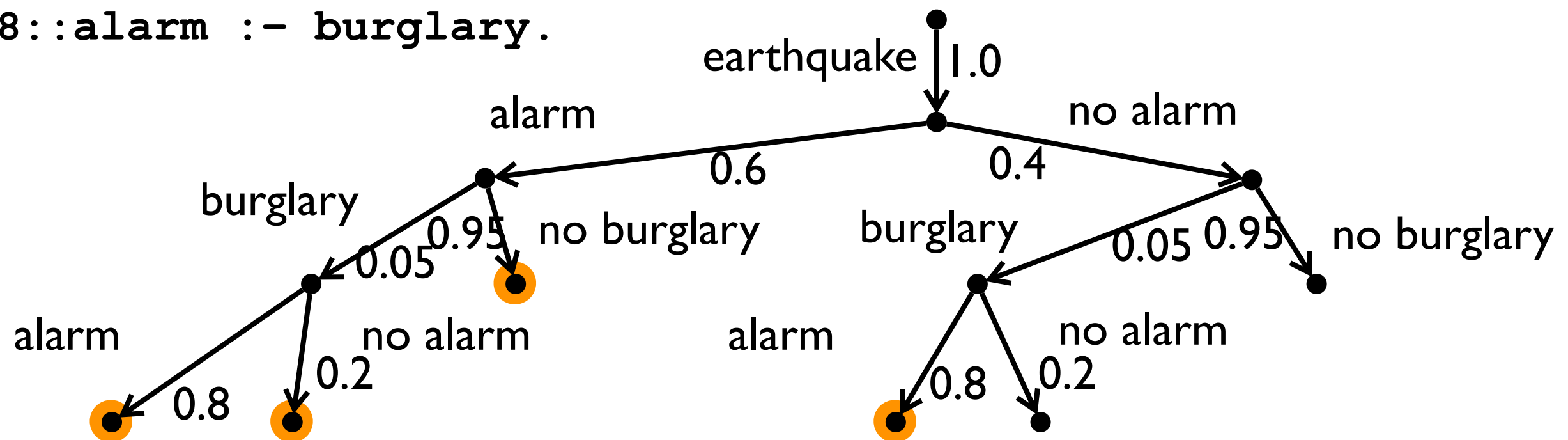
[Vennekens et al, ICLP 04]

0.05::burglary.

0.6::alarm :- earthquake.

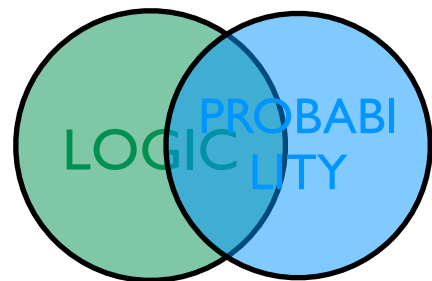
0.8::alarm :- burglary.

**Annotated disjunction
aka conditional probability**



$$P(\text{alarm}) = 0.6 \times 0.05 \times 0.8 + 0.6 \times 0.05 \times 0.2 + 0.6 \times 0.95 + 0.4 \times 0.05 \times 0.8$$

Inference Idea



Inference / Reasoning

- Most of the work in PP and StarAI is on inference
 - It is hard (complexity wise)
 - Many inference methods
 - exact, approximate, sampling and lifted ...
- Inference is the key to learning

ProbLog Inference

Answering a query in a ProbLog program happens in four steps

1. Grounding the program w.r.t. the query
- 2. Rewrite the ground logic program into a propositional logic formula**
3. Compile the formula into an arithmetic circuit
4. Evaluate the arithmetic circuit

0.1 :: burglary.

0.5 :: hears_alarm(mary).

0.2 :: earthquake.

0.4 :: hears_alarm(john).

alarm :- earthquake.

alarm :- burglary.

calls(mary) :- alarm, hears_alarm(mary).

calls(john) :- alarm, hears_alarm(john).

calls(mary)

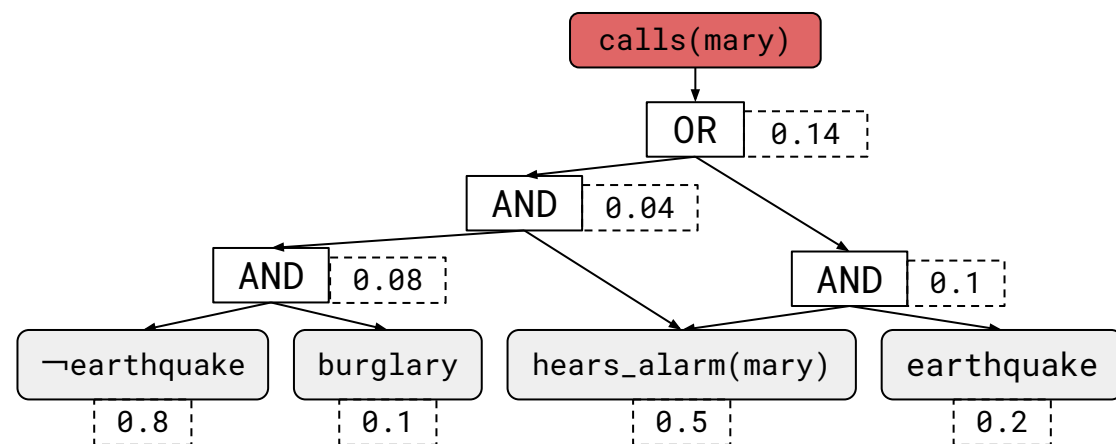
$\leftrightarrow$

$\text{hears_alarm(mary)} \wedge (\text{burglary} \vee \text{earthquake})$

ProbLog Inference

Answering a query in a ProbLog program happens in four steps

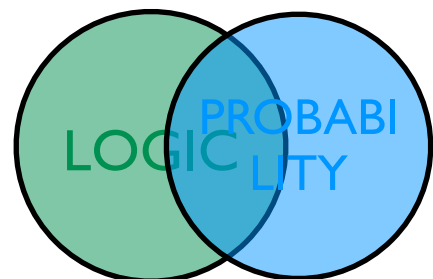
1. Grounding the program w.r.t. the query
2. Rewrite the ground logic program into a propositional logic formula
3. **Compile the formula into an arithmetic circuit (knowledge compilation)**
4. Evaluate the arithmetic circuit



`calls(mary)`

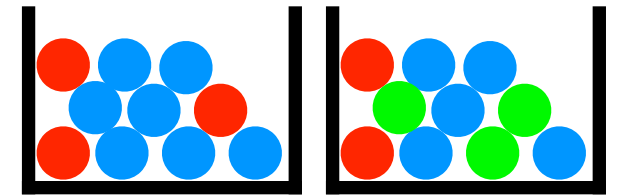
$\leftrightarrow$

$\text{hears_alarm(mary)} \wedge (\text{burglary} \vee \text{earthquake})$



Exercise

ProbLog by example:



A bit of gambling

- toss (biased) coin & draw ball from each urn
- win if (heads and a red ball) or (two balls of same color)

```
0.4 :: heads.
```

probabilistic fact: heads is true with probability 0.4 and false with 0.6)
annotated disjunction: first ball is red with probability 0.3 and blue with 0.7

```
0.3 :: col(1,red) ; 0.7 :: col(1,blue) .
```

```
0.2 :: col(2,red) ; 0.3 :: col(2,green) ;
```

```
0.5 :: col(2,blue) .
```

annotated disjunction: second ball is red with probability 0.2, green with 0.3, and blue with 0.5

```
win :- heads, col(1,red) .  
win :- col(1,C) , col(2,C) .
```

logical rule encoding background knowledge
consequences

Questions

```
0.4 :: heads.
```

```
0.3 :: col(1,red) ; 0.7 :: col(1,blue).
```

```
0.2 :: col(2,red) ; 0.3 :: col(2,green) ; 0.5 :: col(2,blue).
```

```
win :- heads, col(_,red).
```

```
win :- col(1,C), col(2,C).
```

marginal probability

- Probability of **win**

query

conditional probability

- Probability of **win** given **col(2,green)**?

evidence

- Most probable world where **win** is true?

MPE inference

Possible Worlds

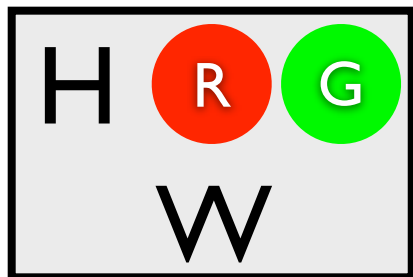
```
0.4 :: heads.
```

```
0.3 :: col(1,red); 0.7 :: col(1,blue)
```

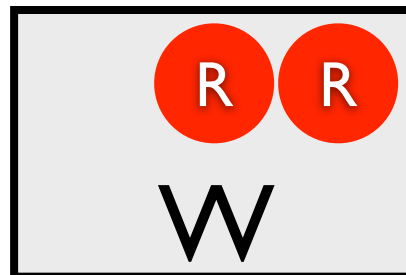
```
0.2 :: col(2,red); 0.3 :: col(2,green); 0.5 :: col(2,blue).
```

```
win :- heads, col(_,red).  
win :- col(1,C), col(2,C).
```

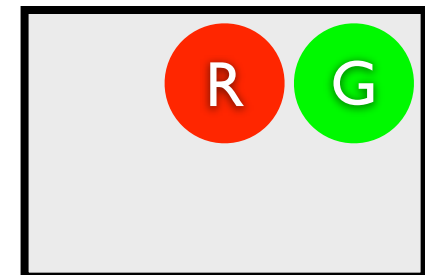
$$0.4 \times 0.3 \times 0.3$$



$$(1-0.4) \times 0.3 \times 0.2$$

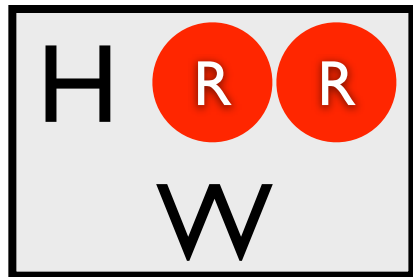


$$(1-0.4) \times 0.3 \times 0.3$$

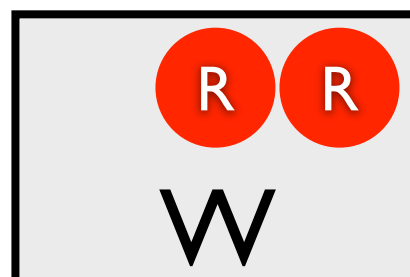


All Possible Worlds

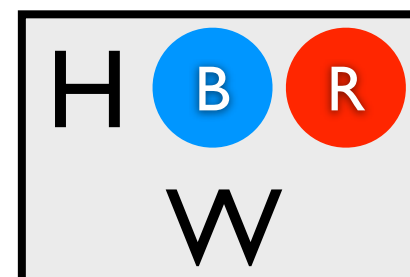
0.024



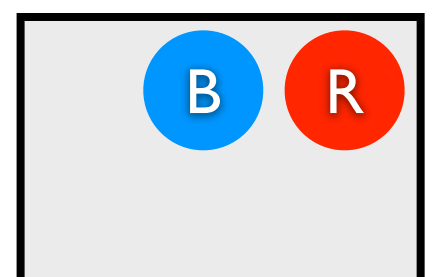
0.036



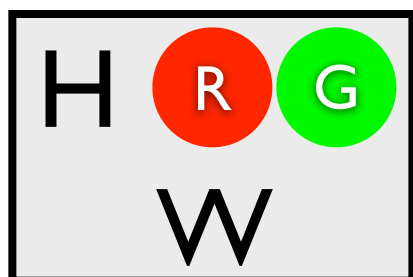
0.056



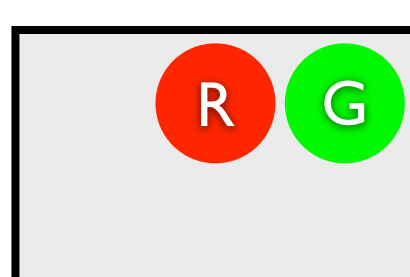
0.084



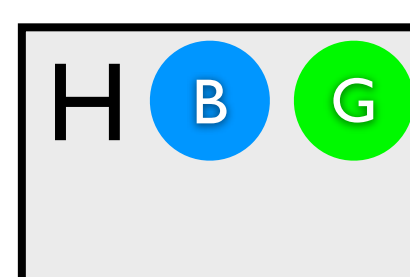
0.036



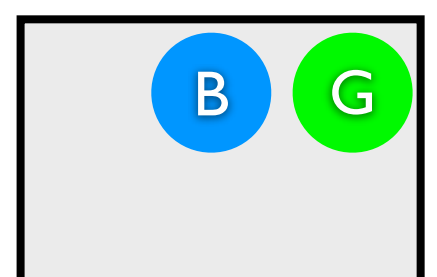
0.054



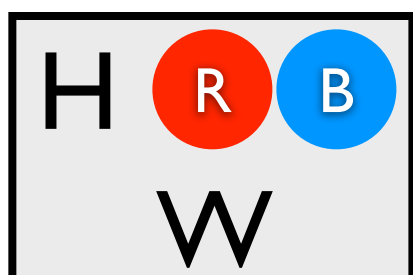
0.084



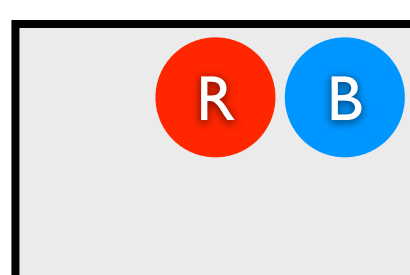
0.126



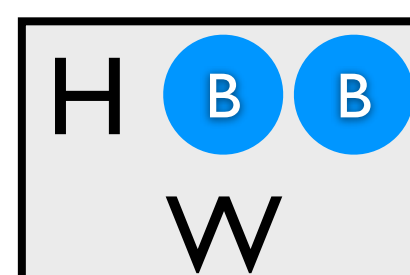
0.060



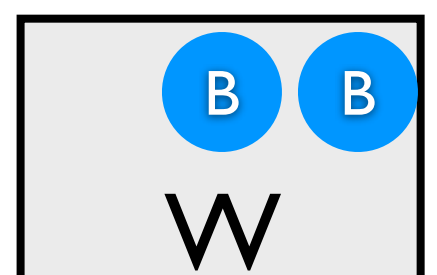
0.090



0.140



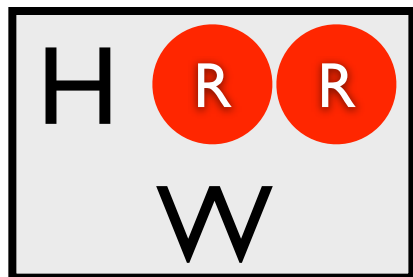
0.210



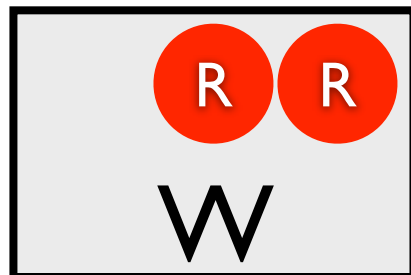
Most likely world where *win* is true?

MPE
Inference

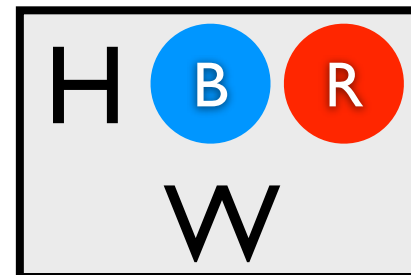
0.024



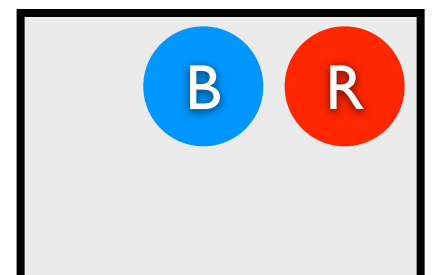
0.036



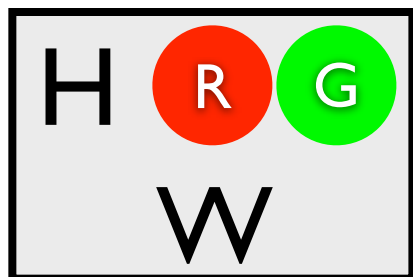
0.056



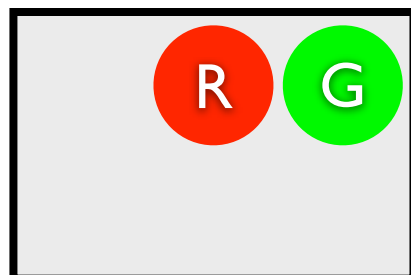
0.084



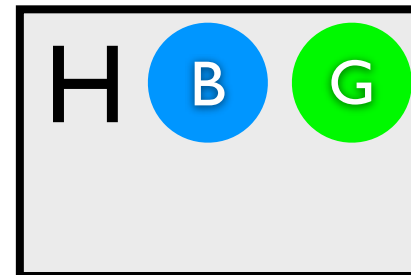
0.036



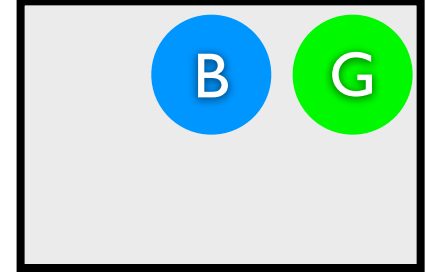
0.054



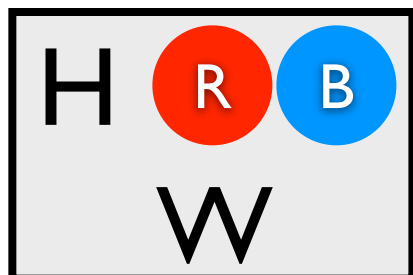
0.084



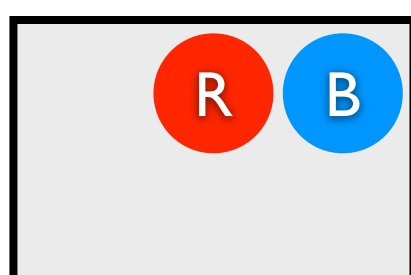
0.126



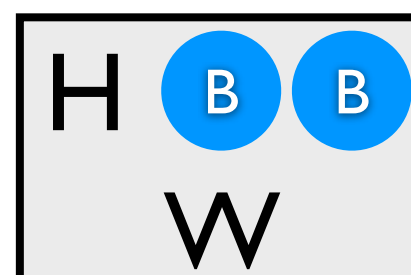
0.060



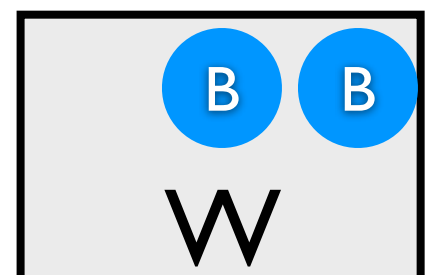
0.090



0.140



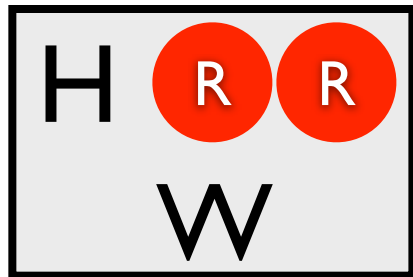
0.210



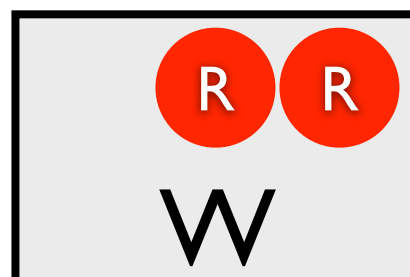
Most likely world where `col(2, blue)` is false?

MPE
Inference

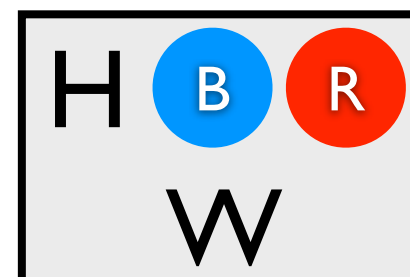
0.024



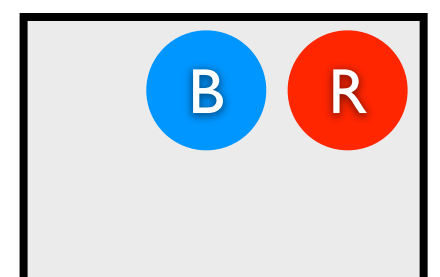
0.036



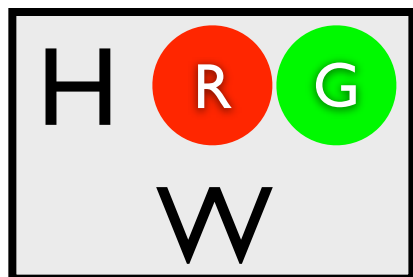
0.056



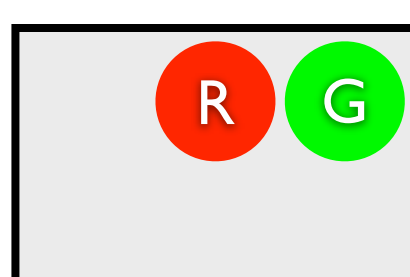
0.084



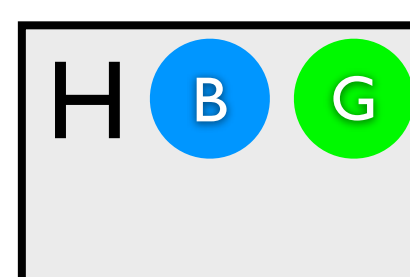
0.036



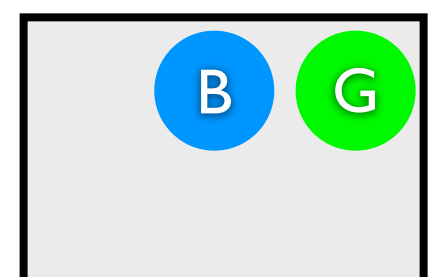
0.054



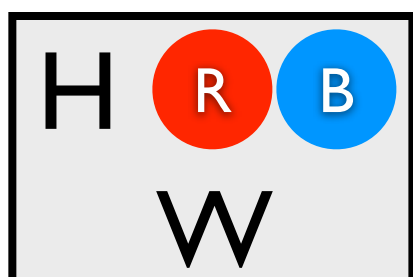
0.084



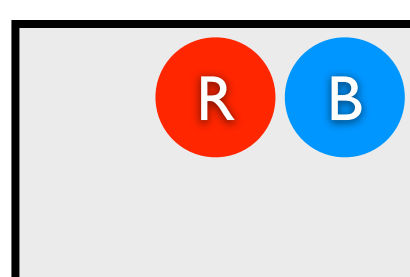
0.126



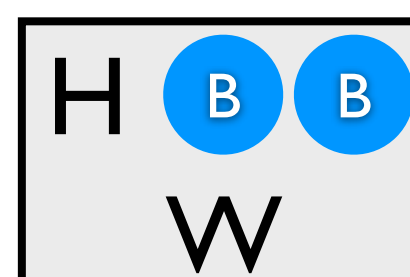
0.060



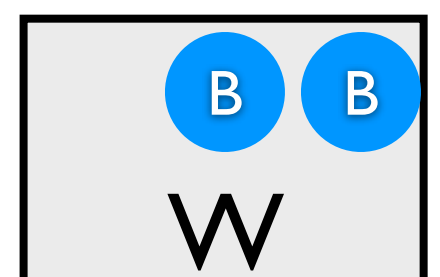
0.090



0.140



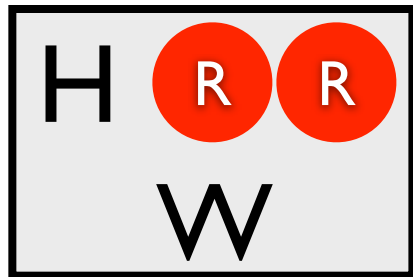
0.210



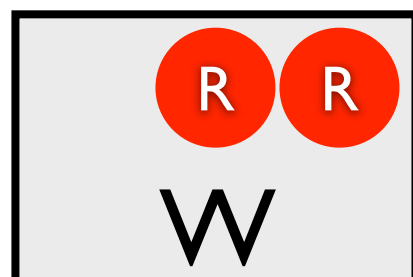
$$P(\text{win}) = \sum = 0.562$$

Marginal
Probability

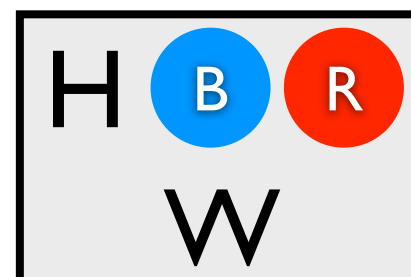
0.024



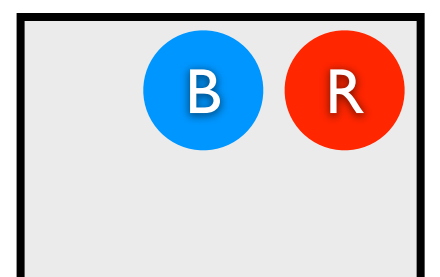
0.036



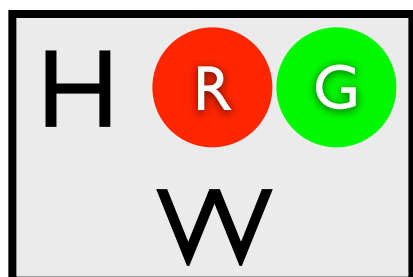
0.056



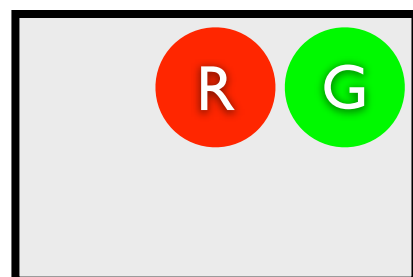
0.084



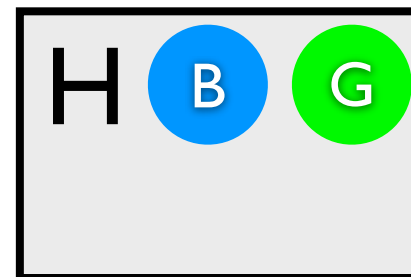
0.036



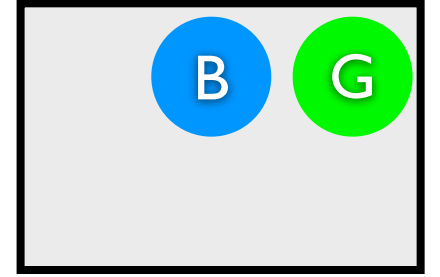
0.054



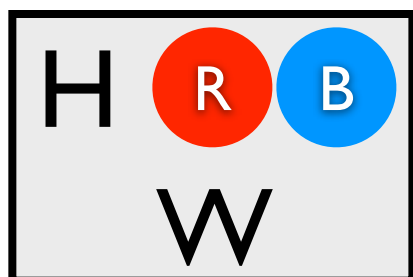
0.084



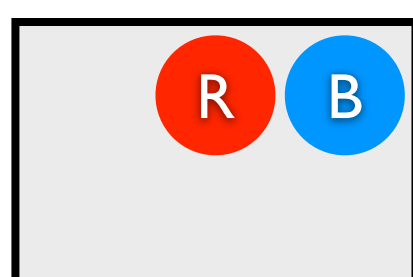
0.126



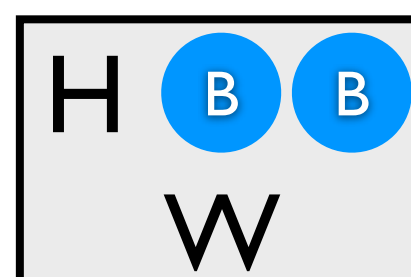
0.060



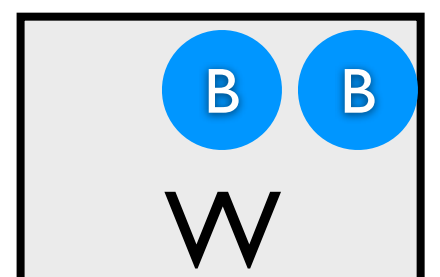
0.090



0.140



0.210

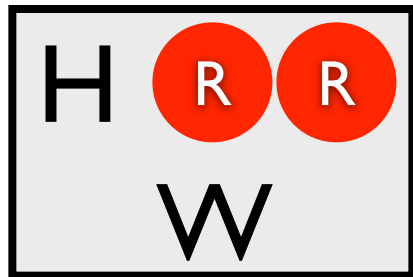


$$P(\text{win} | \text{col}(2, \text{green})) = ? / \Sigma$$

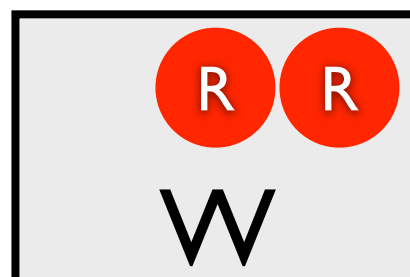
$$= P(\text{win} \wedge \text{col}(2, \text{green}) = 0.036 / P(\text{col}(2, \text{green}) = 0.3)$$

Conditional
Probability

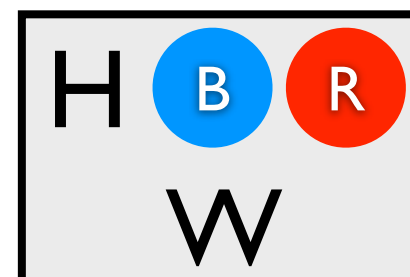
0.024



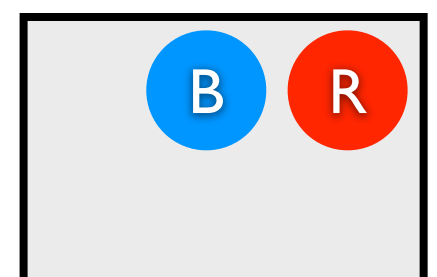
0.036



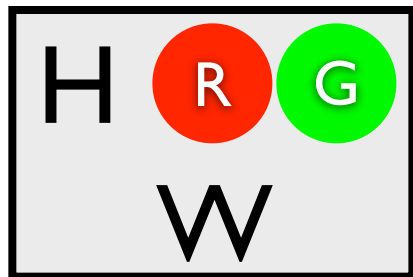
0.056



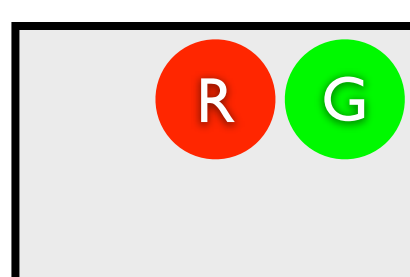
0.084



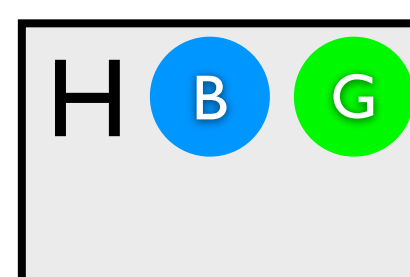
0.036



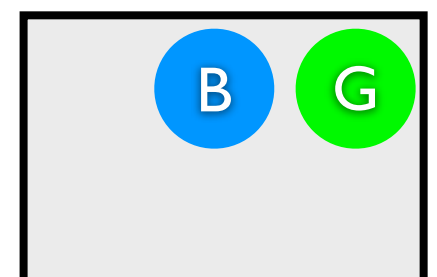
0.054



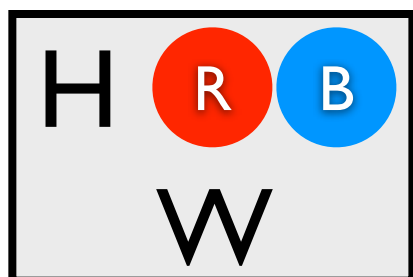
0.084



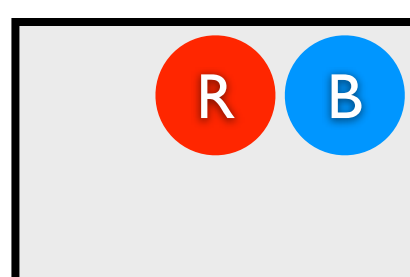
0.126



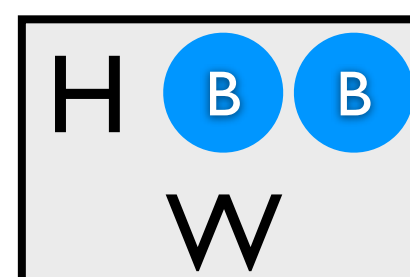
0.060



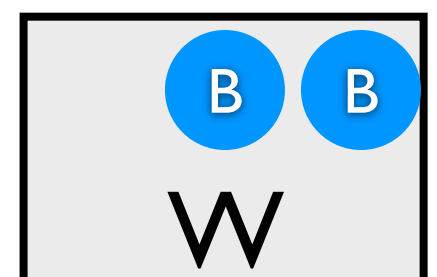
0.090



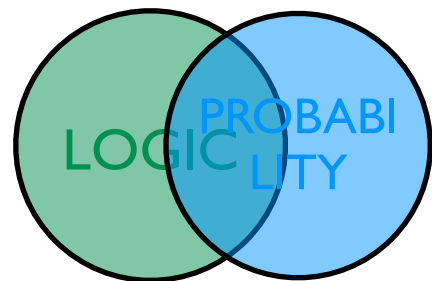
0.140



0.210



Probabilistic Programming and Statistical Relational AI



Probabilistic Logic Program Semantics

Propositional logic program

0.1 :: burglary.

0.05 :: earthquake.

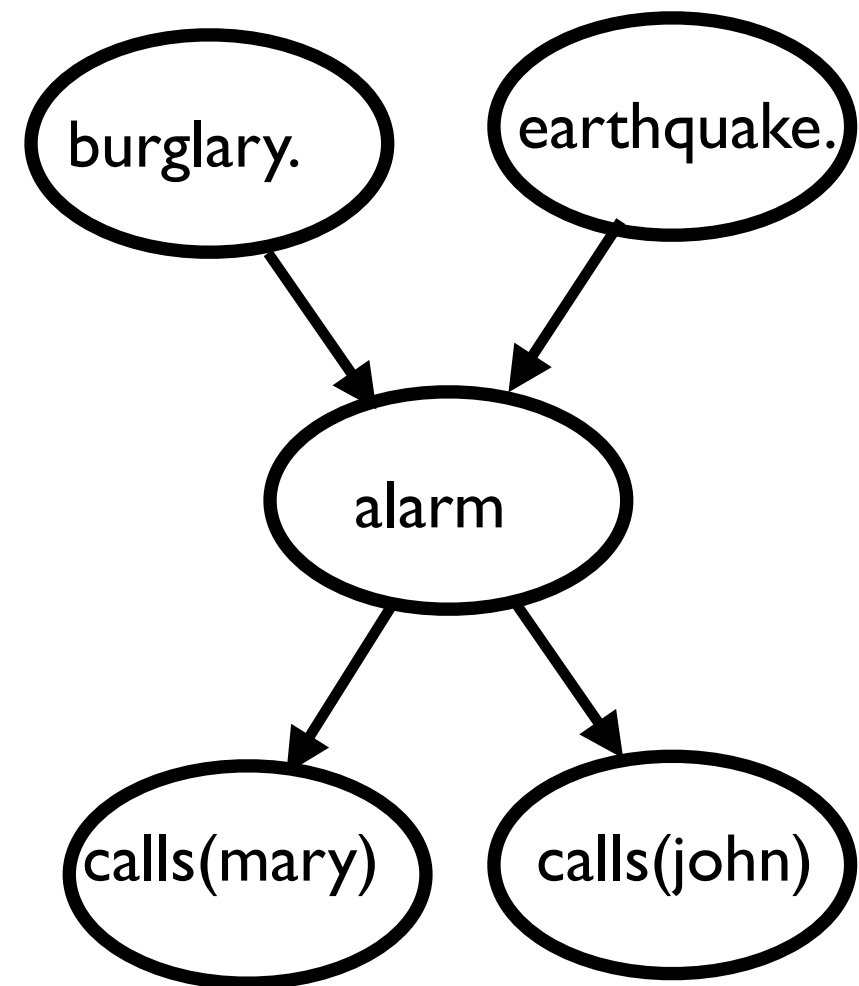
alarm :- earthquake.

alarm :- burglary.

0.7::calls(mary) :- alarm.

0.6::calls(john) :- alarm.

Bayesian Network

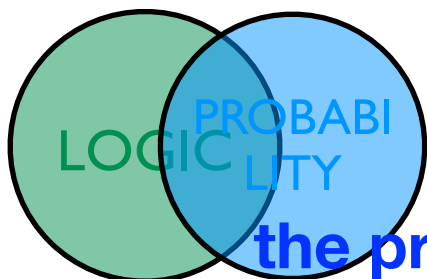


Bayesian net encoded as Probabilistic Logic Program

PLPs correspond to directed graphical models

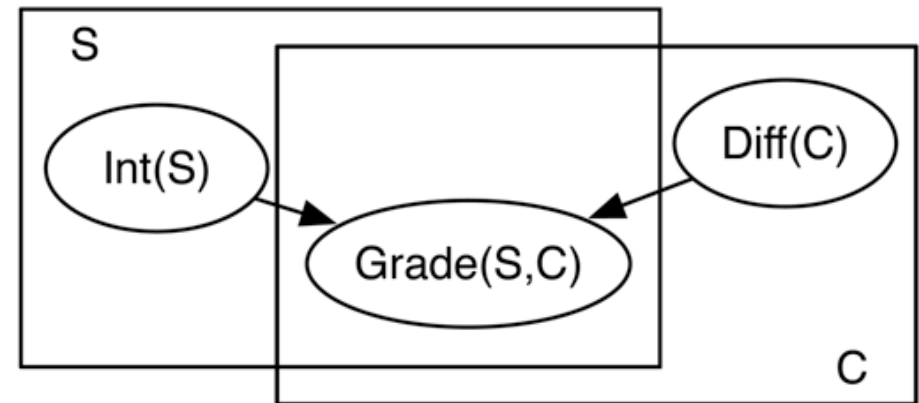
ProbLog has both (directed) probabilistic graphic models,

the programming language Prolog (and probabilistic databases) as special case



Flexible and Compact Relational Model for Predicting Grades

PLATE NOTATION



“Program” Abstraction:

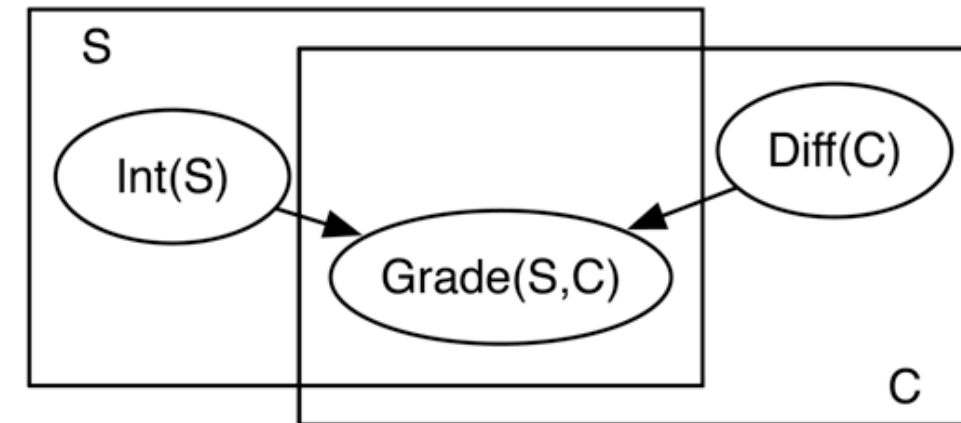
- S, C **logical variable** representing students, courses
- the set of individuals of a type is called a **population**
- Int(S), Grade(S, C), D(C) are **parametrized random variables**

Grounding:

- for every student s, there is a random variable Int(s)
- for every course c, there is a random variable Di(c)
- for every s, c pair there is a random variable Grade(s,c)
- all instances share the same structure and parameters

idea of parameter tying

ProbLog by example: Grading



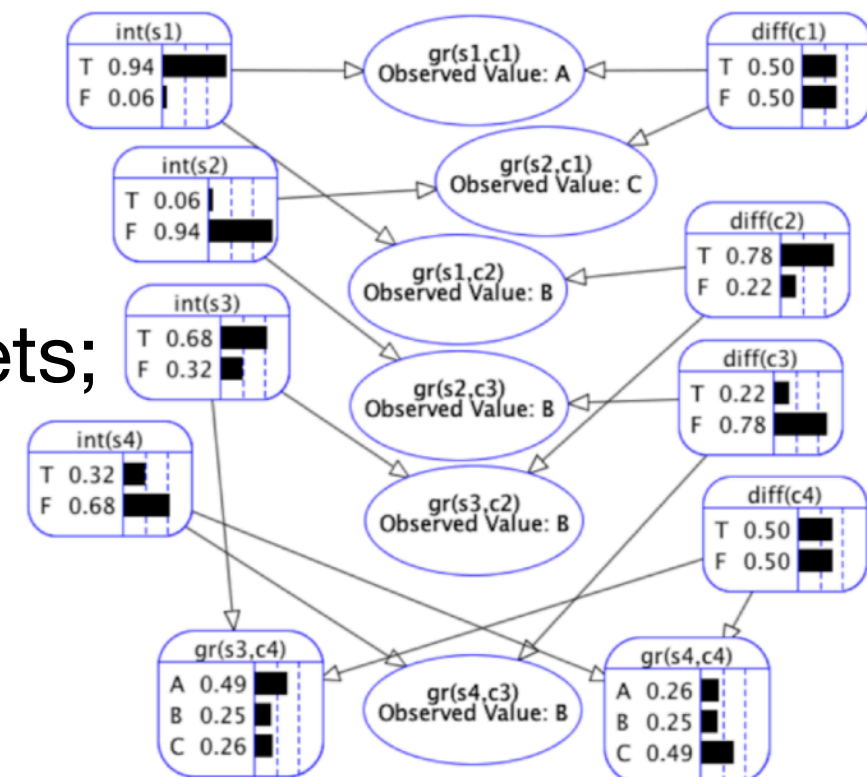
Shows relational structure

- grounded model: replace variables by constants

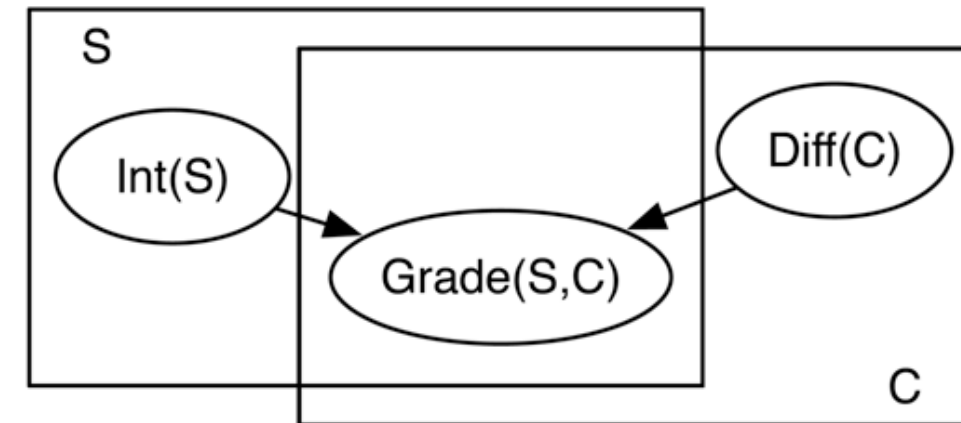
Works for any number of students / classes (for 1000 students and 100 classes, you get 101100 random variables); still only few parameters

With SRL / PP

- build and learn compact models,
- from one set of individuals - > other sets;
- reason also about exchangeability,
- build even more complex models,
- incorporate background knowledge



ProbLog by example: Grading



Shows relational structure

- grounded model: replace variables by constants

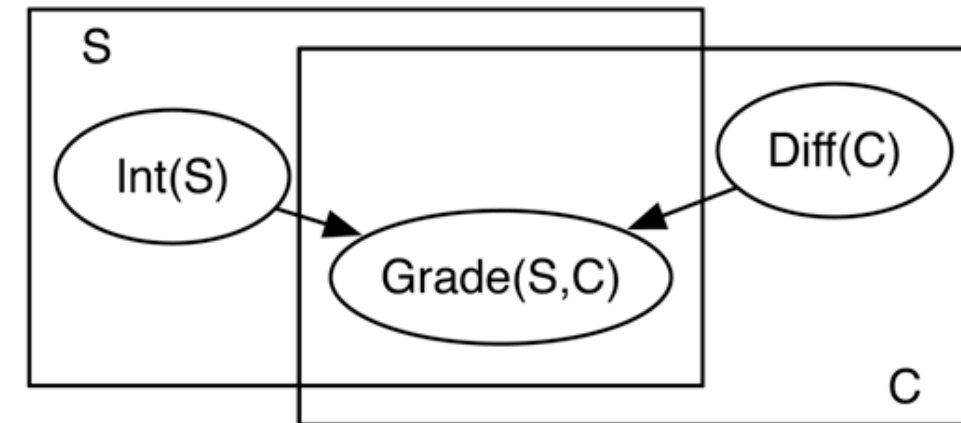
Works for any number of students / classes (for 1000 students and 100 classes, you get 101100 random variables); still only few parameters

With SRL / PP

- build and learn compact models,
- from one set of individuals - > other sets;
- reason also about exchangeability,
- build even more complex models,
- incorporate background knowledge

<i>Student</i>	<i>Course</i>	<i>Grade</i>
s_1	c_1	A
s_2	c_1	C
s_1	c_2	B
s_2	c_3	B
s_3	c_2	B
s_4	c_3	B
s_3	c_4	$?$
s_4	c_4	$?$

ProbLog by example: Grading



```
0.4 :: int(S) :- student(S) .
0.5 :: diff(C) :- course(C) .
```

one rule for every row in a CPT

```
student(john) . student(anna) . student(bob) .
course(ai) .      course(ml) .      course(cs) .
```

```
gr(S,C,a) :- int(S), not diff(C) .
0.3 :: gr(S,C,a) ; 0.5 :: gr(S,C,b) ; 0.2 :: gr(S,C,c) :-
    int(S), diff(C) .
0.1 :: gr(S,C,b) ; 0.2 :: gr(S,C,c) ; 0.2 :: gr(S,C,f) :-
    student(S), course(C),
    not int(S), not diff(C) .
0.3 :: gr(S,C,c) ; 0.2 :: gr(S,C,f) :-
    not int(S), diff(C) .
```

ProbLog by example: Grading

```
unsatisfactory(S) :- student(S), grade(S,C,f).
```

```
excellent(S) :- student(S), not(grade(S,C1,G),below(G,a)),  
                grade(S,C2,a).
```

```
0.4 :: int(S) :- student(S).
```

```
0.5 :: diff(C) :- course(C).
```

```
student(john). student(anna). student(bob).  
course(ai).    course(ml).    course(cs).
```

```
gr(S,C,a) :- int(S), not diff(C).
```

```
0.3 :: gr(S,C,a) ; 0.5 :: gr(S,C,b) ; 0.2 :: gr(S,C,c) :-  
                int(S), diff(C).
```

```
0.1 :: gr(S,C,b) ; 0.2 :: gr(S,C,c) ; 0.2 :: gr(S,C,f) :-  
                student(S), course(C),  
                not int(S), not diff(C).
```

```
0.3 :: gr(S,C,c) ; 0.2 :: gr(S,C,f) :-  
                not int(S), diff(C).
```

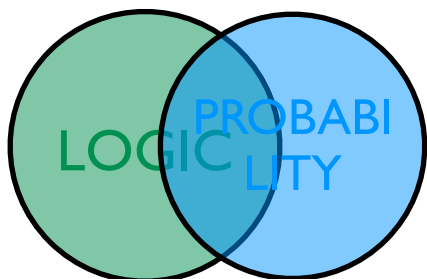
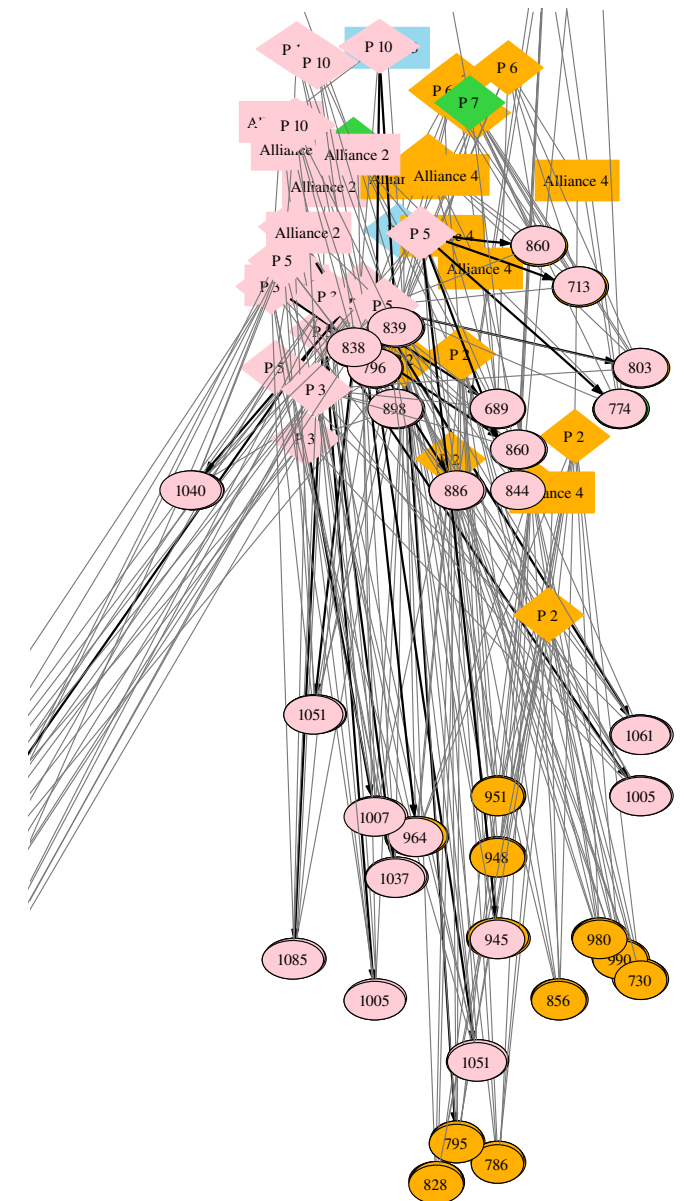
Dynamic networks



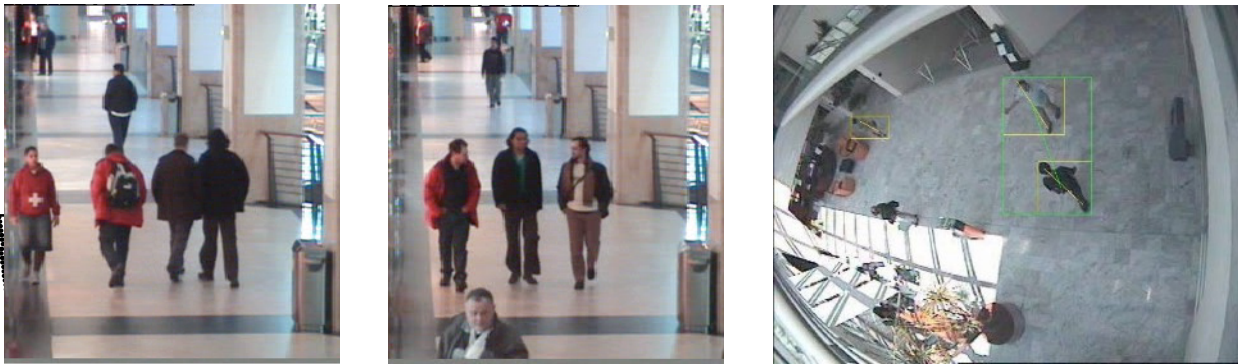
Travian: A massively multiplayer real-time strategy game

Can we build a model
of this world ?

Can we use it for playing
better ?

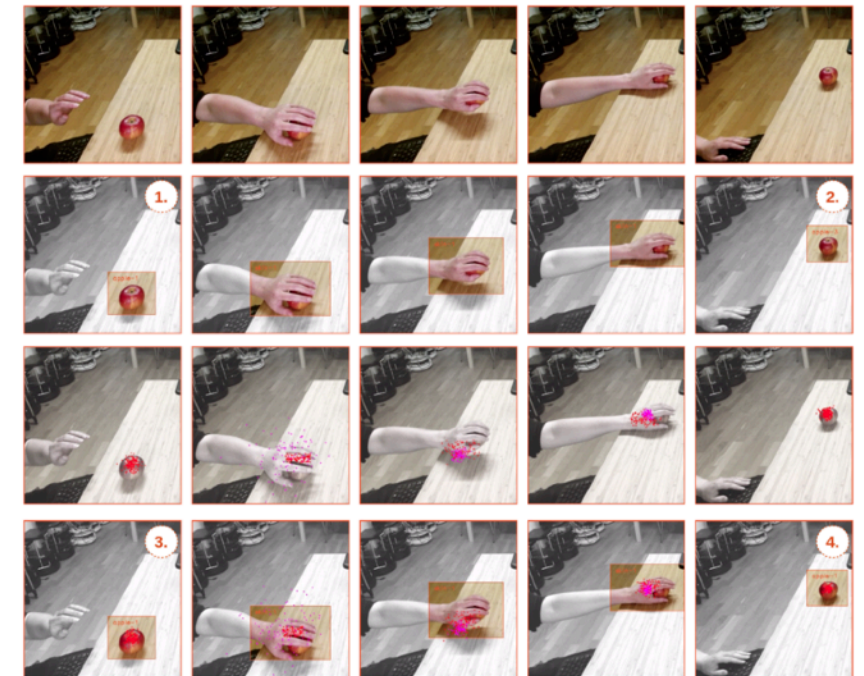


Activity analysis and tracking video analysis

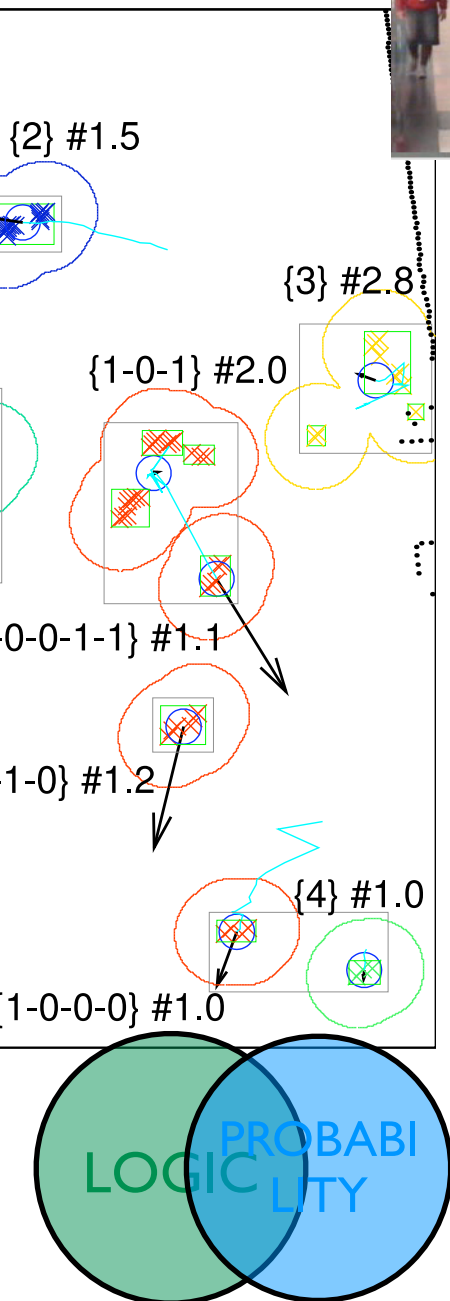


- Track people or objects over time? Even if temporarily hidden?
- Recognize activities?
- Infer object properties?

[Skarlatidis et al, TPLP 14;
Nitti et al, IROS 13, ICRA 14,
MLJ 16]



[Persson et al, IEEE Trans on
Cogn. & Dev. Sys. 19;
IJCAI 20]



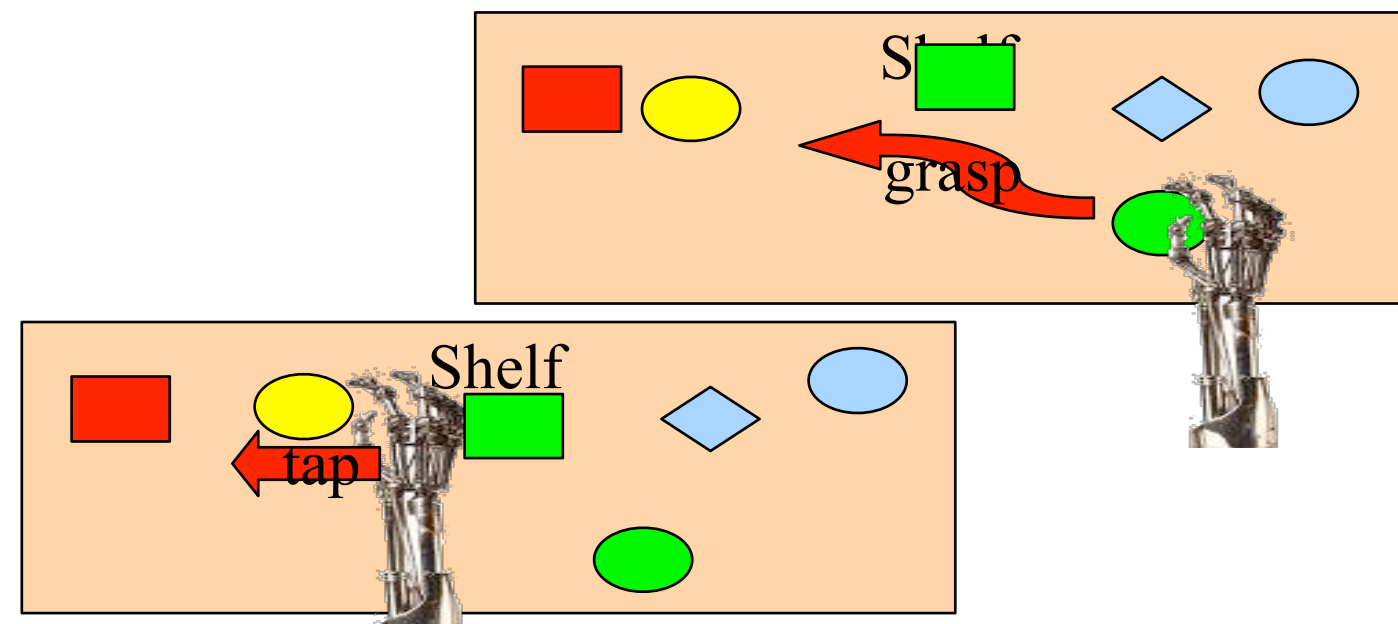
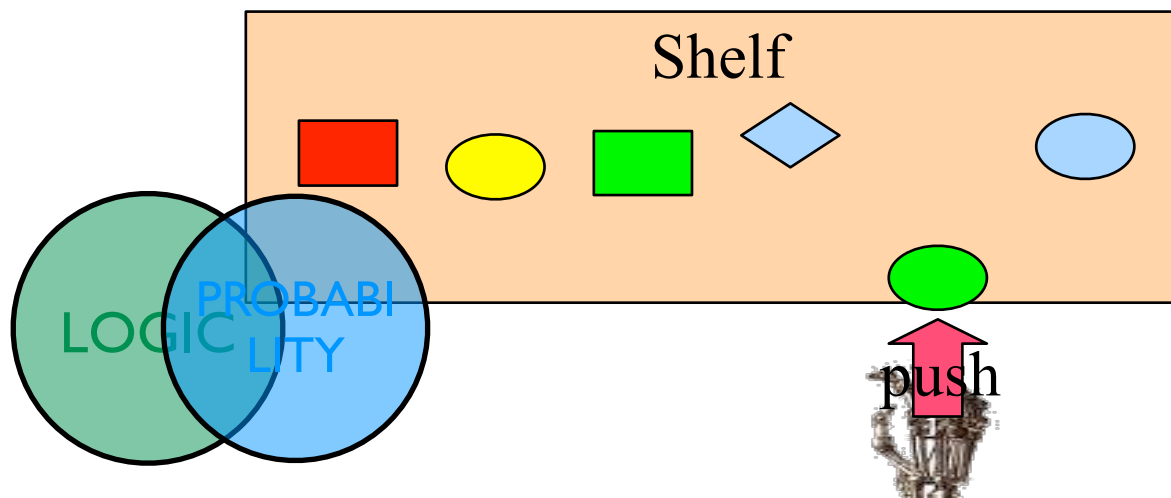
Learning relational affordances



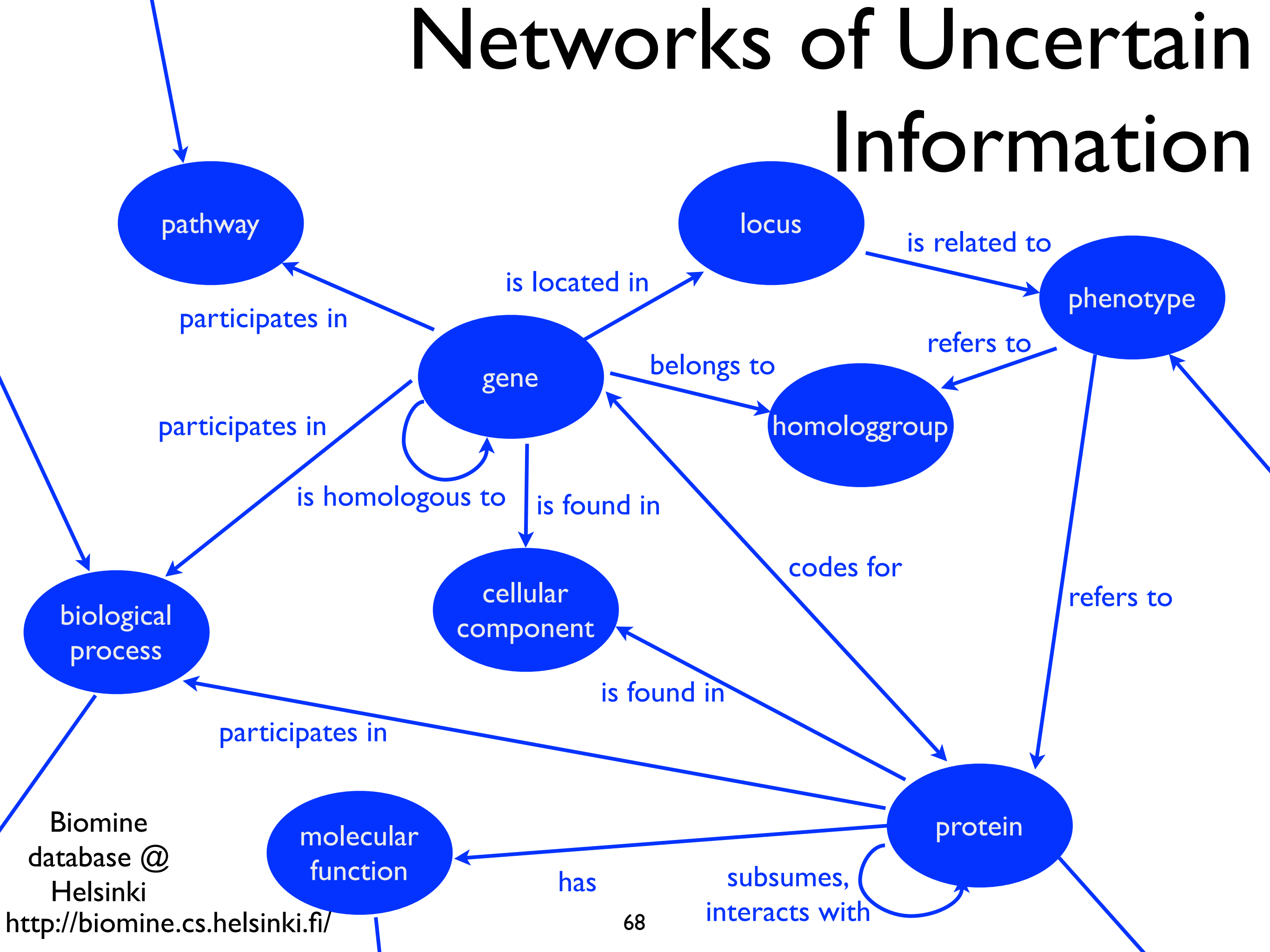
Learning relational
affordances
between
two objects
(learnt by experience)

similar to probabilistic Strips
(with continuous distributions)

Moldovan et al. ICRA 12, 13, 14; Auton. Robots 18



Networks of Uncertain Information



Biology

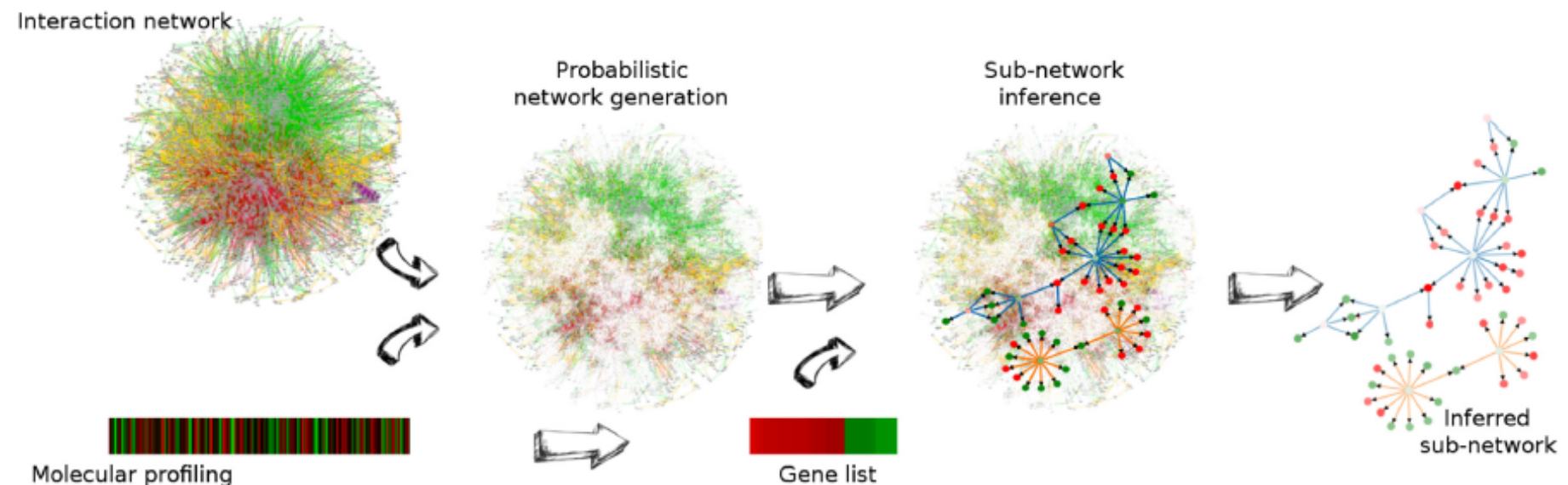
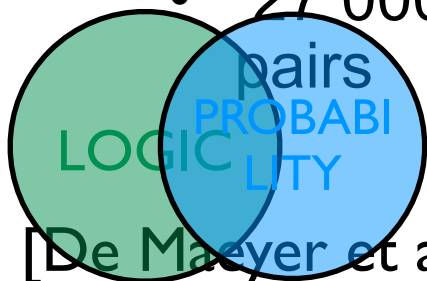
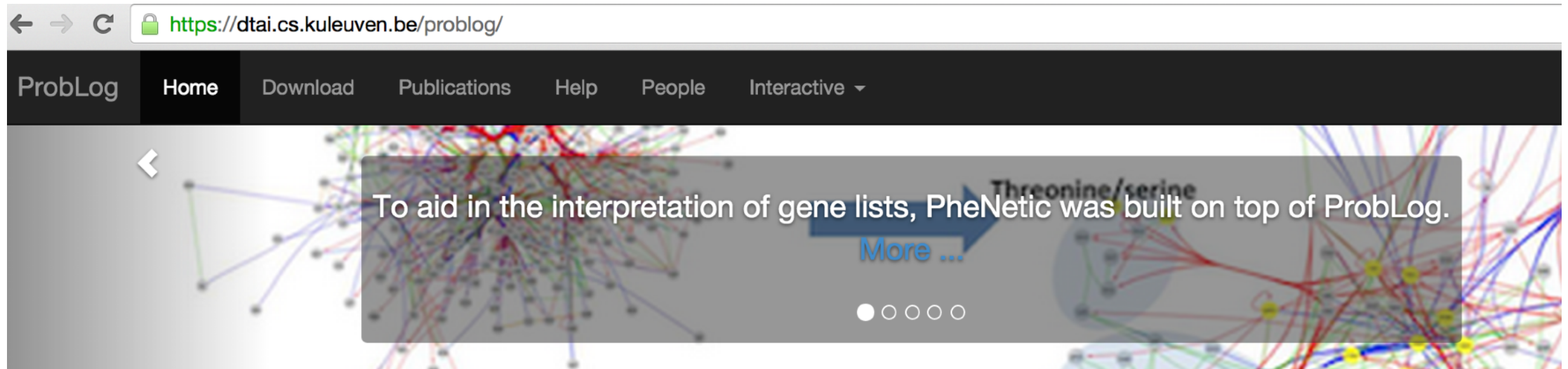


Figure 1. Overview of PheNetic, a web service for network-based interpretation of ‘omics’ data. The web service uses as input a genome wide interaction network for the organism of interest, a user generated molecular profiling data set and a gene list derived from these data. Interaction networks for a wide variety of organisms are readily available from the web server. Using the uploaded user-generated molecular data the interaction network is converted into a probabilistic network: edges receive a probability proportional to the levels measured for the terminal nodes in the molecular profiling data set. This probabilistic interaction network is used to infer the sub-network that best links the genes from the gene list. The inferred sub-network provides a trade-off between linking as many genes as possible from the gene list and selecting the least number of edges.

- Causes: Mutations
 - All related to similar phenotype
 - Effects: Differentially expressed genes
 - 27 000 cause effect pairs
- Interaction network:
 - 3063 nodes
 - Genes
 - Proteins
 - 16794 edges
 - Molecular interactions
 - Uncertain
- Goal: connect causes to effects through common subnetwork
 - = Find mechanism
- Techniques:
 - DTPProbLog
 - Approximate inference





Introduction.

Probabilistic logic programs are logic programs in which some of the facts are annotated with probabilities.

ProbLog is a tool that allows you to intuitively build programs that do not only encode **complex interactions** between a large sets of **heterogenous components** but **uncertainties** that are present in real-life situations.

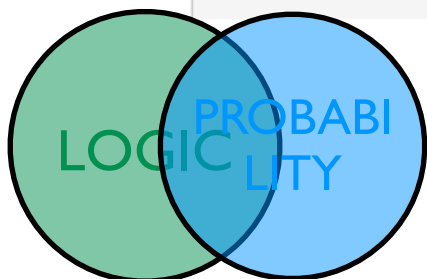
The engine tackles several tasks such as computing the marginals given evidence and learning from (partial) interpretations. ProbLog is a suite of efficient algorithms tasks. It is based on a conversion of the program and the queries and evidence to a weighted Boolean formula. This allows us to reduce the inference tasks to well-s weighted model counting, which can be solved using state-of-the-art methods known from the graphical model and knowledge compilation literature.

The Language. Probabilistic Logic Programming.

ProbLog makes it easy to express complex, probabilistic models.

```
0.3::stress(X) :- person(X).
0.2::influences(X,Y) :- person(X), person(Y).

smokes(X) :- stress(X).
smokes(X) :- friend(X,Y), influences(Y,X), smokes(Y).
```



Probabilistic Programming

Languages outside LP

- IBAL [Pfeffer 01]
- Figaro [Pfeffer 09]
- Church [Goodman et al 08]
- BLOG [Milch et al 05]
- Stan & Edward & Anglican
- and many more appearing recently such

Church

probabilistic functional
programming

[Goodman et al, UAI 08]

**several
possible
executions**

```
(define randplus5  
  (lambda (x) (if (flip 0.6)  
                  (+ x 5)  
                  x)))  
  
(map randplus5 '(1 2 3))
```

Dealing with
primitives

probabilistic primitives + functional program
→ distribution over possible executions

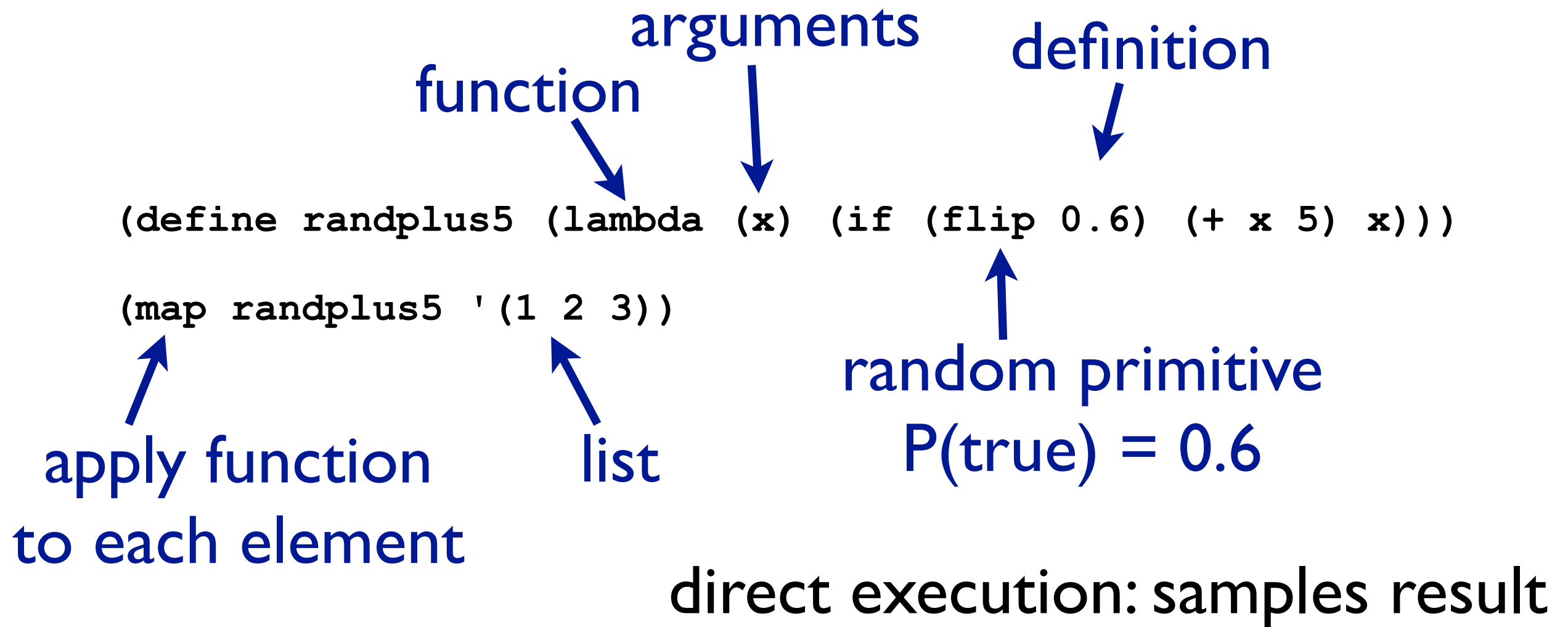
Reasoning with
probabilistic data

one execution

```
(define plus5 (lambda (x) (+ x 5)))  
  
(map plus5 '(1 2 3))
```

Learning

Church Example



sampling also supports continuous RVs, e.g.,
`(* (gaussian 0 1) (gaussian 0 1))`

Probabilistic Programming Ingredients

inject random variables in programming language

- random var = simplest boolean concept in language
- extensions for continuous variables exist

define **semantics** :

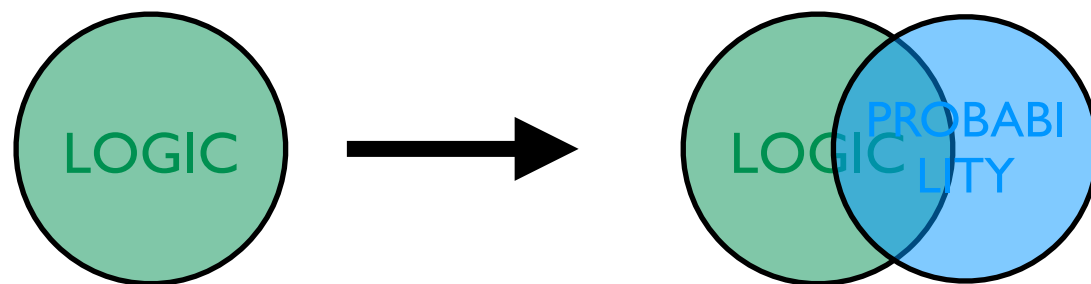
- possible worlds for probabilistic logics / databases / StarAI
- trace-based semantics for functional / imperative languages
 - define probability of execution, whenever random variable encountered -> multiply by probability

inference to answer (conditional) queries

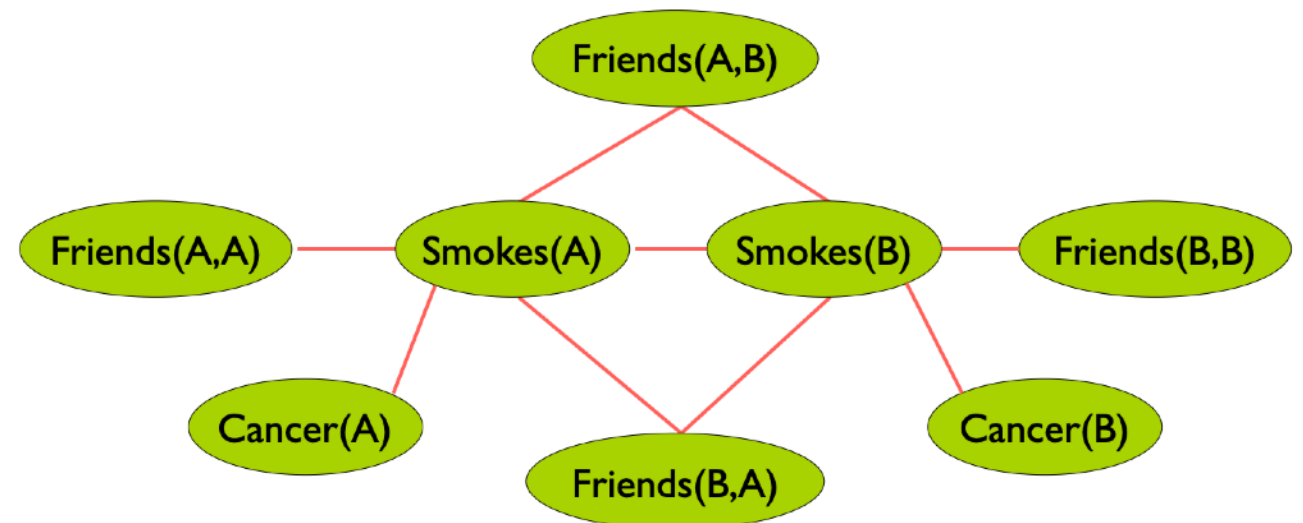
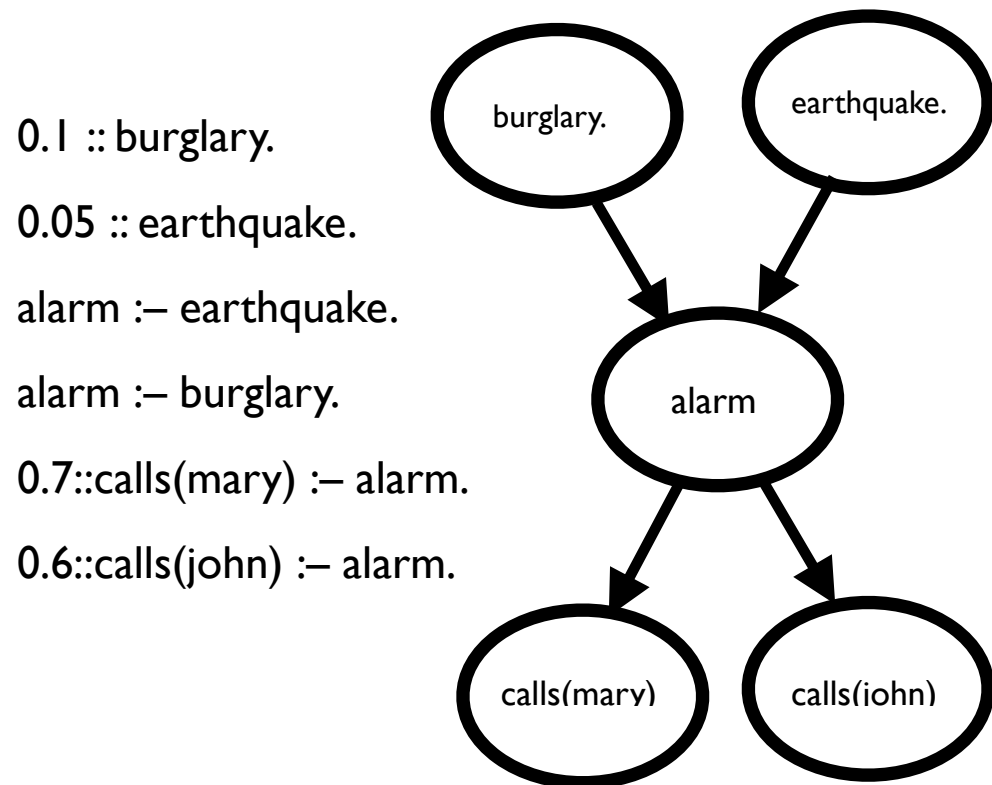
learning

- Bayesian inference : conditioning on observations and querying for distributions (typical for functional and imperative languages)
- Expected Maximisation (typical for Probabilistic Logics & StarAI)
see also [Russell, CACM 15]

Dimension 2: Directed vs Undirected Graphical Model



2. Directed vs Undirected the PGM / StarAI dimension



$$1.5 \quad \forall x \text{ Smokes}(x) \Rightarrow \text{Cancer}(x)$$

$$1.1 \quad \forall x, y \text{ Friends}(x, y) \Rightarrow (\text{Smokes}(x) \Leftrightarrow \text{Smokes}(y))$$

Probabilistic Logic Programs

ProbLog

directed Bayesian Net

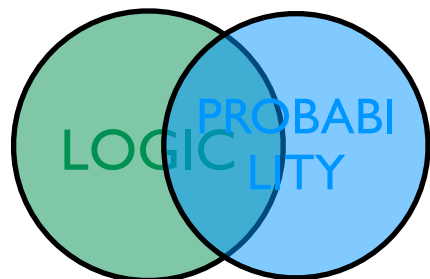
Markov Logic

undirected

Markov Net

model theoretic

key representatives



Markov Logic: Intuition

- *Undirected* graphical model
- A logical KB is a set of **hard constraints** on the set of possible worlds
- Let's make them **soft constraints**:
When a world violates a formula,
it becomes less probable, not impossible
- Give each formula a **weight**
(Higher weight $\Rightarrow$ Stronger constraint)

$$P(\text{world}) \propto \exp\left(\sum \text{weights of formulas it satisfies}\right)$$

A possible worlds view

Say we have two domain elements **Anna** and **Bob** as well as two predicates **Friends** and **Happy**

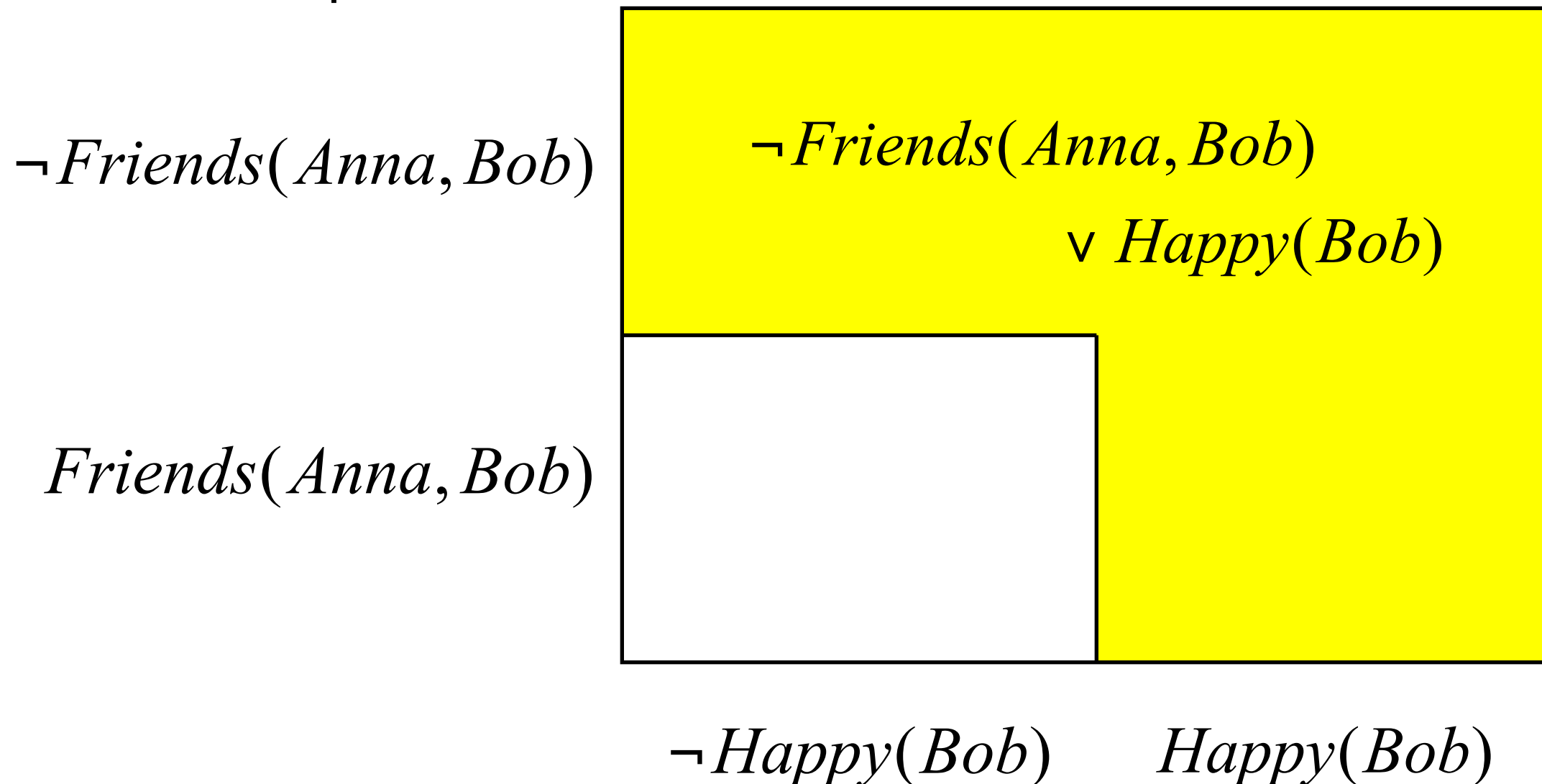
$\neg \text{Friends}(\text{Anna}, \text{Bob})$		
$\text{Friends}(\text{Anna}, \text{Bob})$		
	$\neg \text{Happy}(\text{Bob})$	$\text{Happy}(\text{Bob})$

A possible worlds view

Logical formulas such as

not Friends(Anna,Bob) or Happy(Bob)

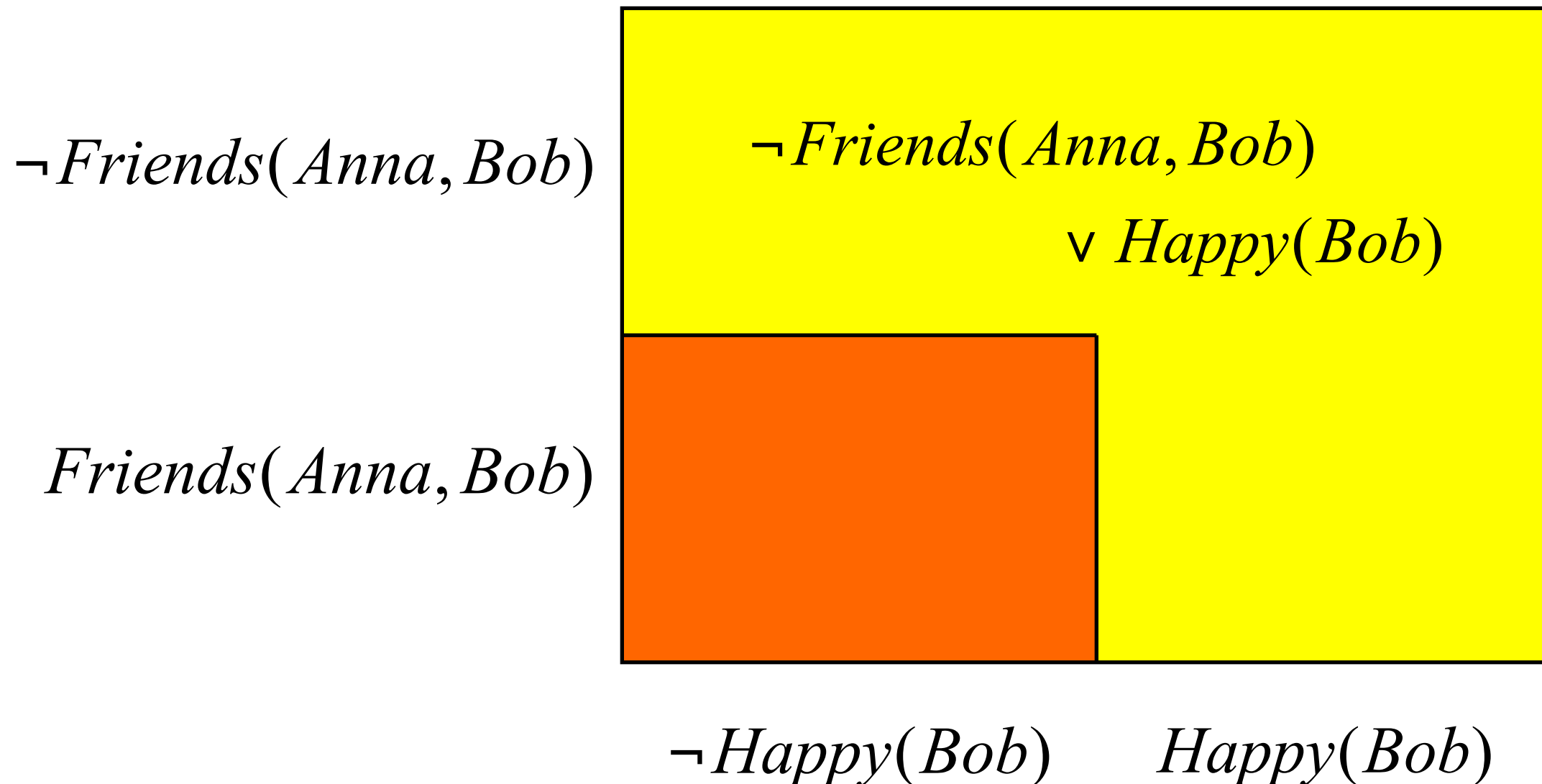
exclude possible worlds



A possible worlds view

Instead of excluding worlds, we want them to become less likely,
e.g.

$$P(\neg \textit{Friends}(\textit{Anna}, \textit{Bob}) \vee \textit{Happy}(\textit{Bob})) = 0.8$$



A possible worlds view

four times as likely that rule holds

$$\Phi(\neg \textit{Friends}(\textit{Anna}, \textit{Bob}) \vee \textit{Happy}(\textit{Bob})) = 1$$

$$\Phi(\textit{Friends}(\textit{Anna}, \textit{Bob}) \wedge \neg \textit{Happy}(\textit{Bob})) = 0.75$$

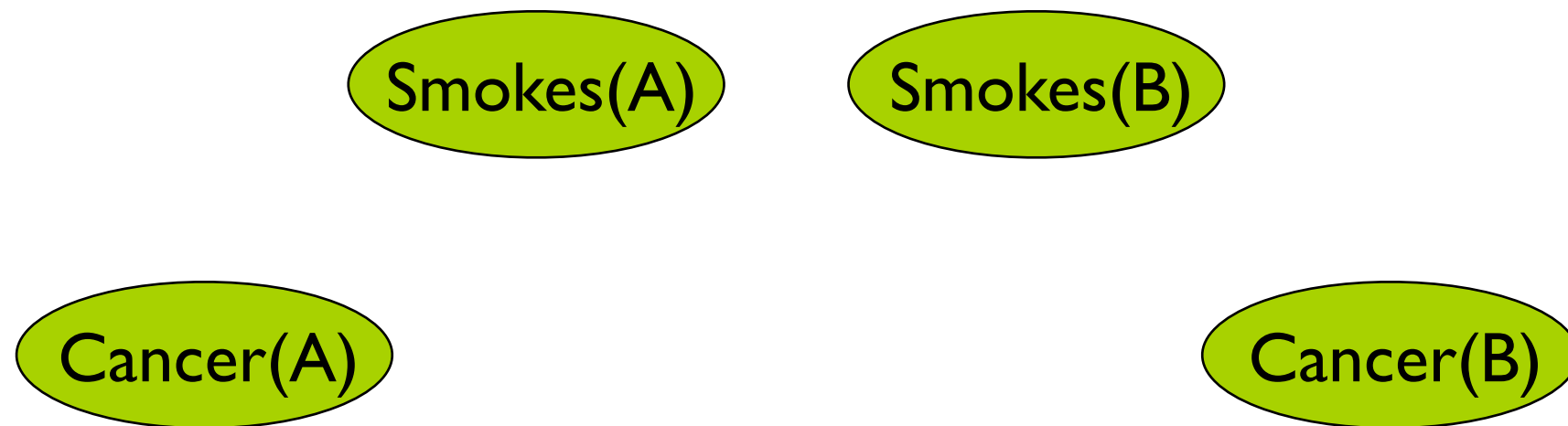
$\neg \textit{Friends}(\textit{Anna}, \textit{Bob})$	1	1
$\textit{Friends}(\textit{Anna}, \textit{Bob})$	0.75	1
	$\neg \textit{Happy}(\textit{Bob})$	$\textit{Happy}(\textit{Bob})$

Markov Logic

1.5 $\forall x \text{ Smokes}(x) \Rightarrow \text{Cancer}(x)$

1.1 $\forall x, y \text{ Friends}(x, y) \Rightarrow (\text{Smokes}(x) \Leftrightarrow \text{Smokes}(y))$

Suppose we have two constants: **Anna** (A) and **Bob** (B)

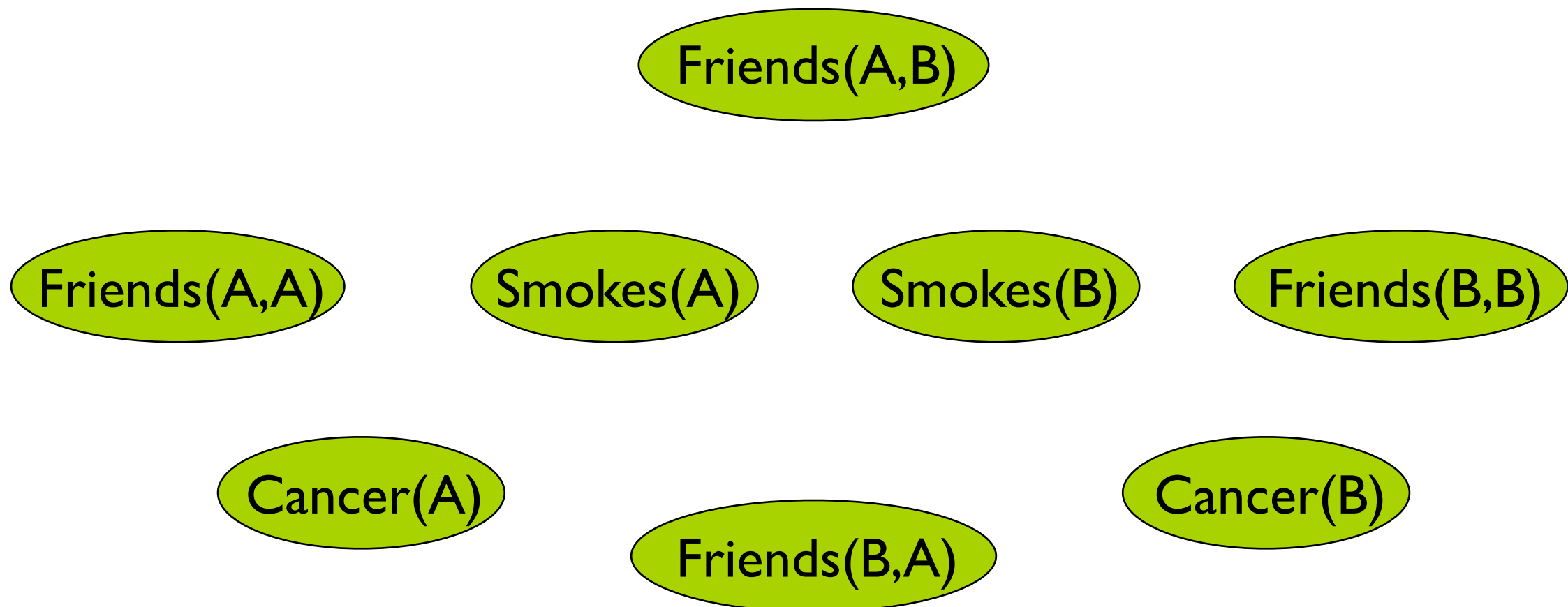


Markov Logic

1.5 $\forall x \text{ Smokes}(x) \Rightarrow \text{Cancer}(x)$

1.1 $\forall x, y \text{ Friends}(x, y) \Rightarrow (\text{Smokes}(x) \Leftrightarrow \text{Smokes}(y))$

Suppose we have two constants: **Anna** (A) and **Bob** (B)

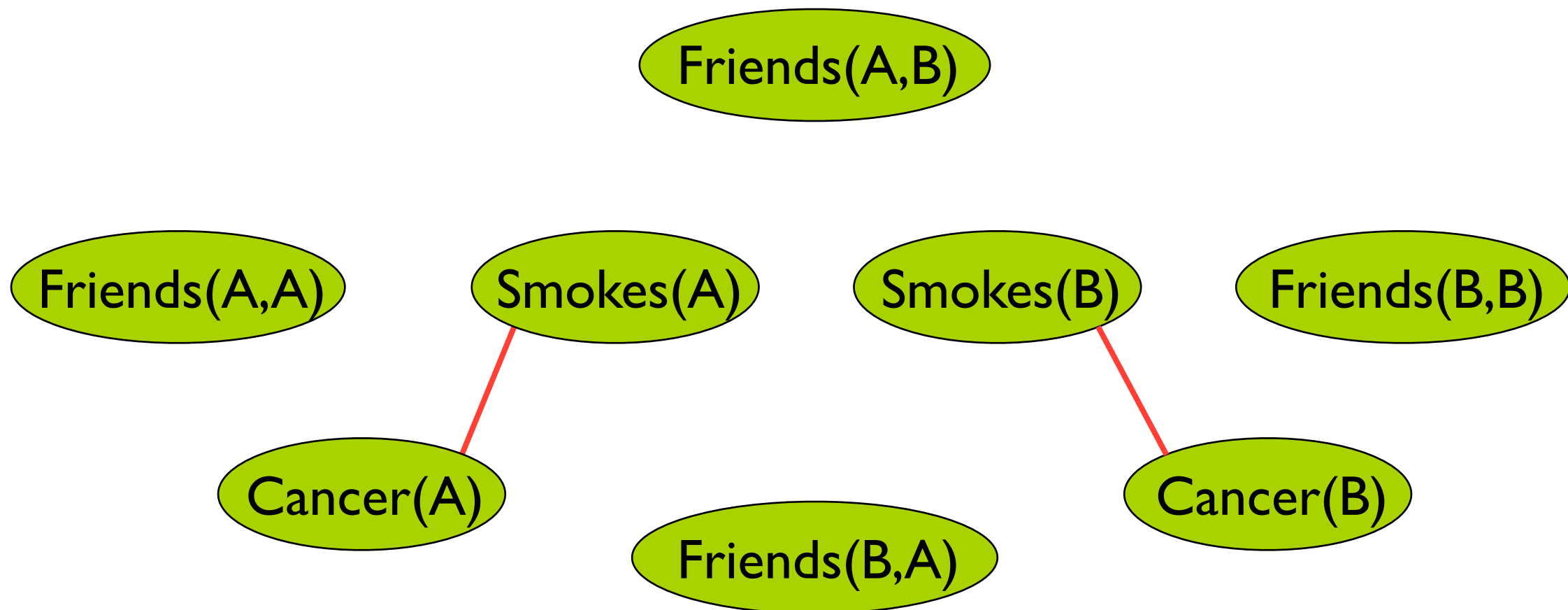


Markov Logic

1.5 $\forall x \text{ Smokes}(x) \Rightarrow \text{Cancer}(x)$

1.1 $\forall x, y \text{ Friends}(x, y) \Rightarrow (\text{Smokes}(x) \Leftrightarrow \text{Smokes}(y))$

Suppose we have two constants: **Anna** (A) and **Bob** (B)

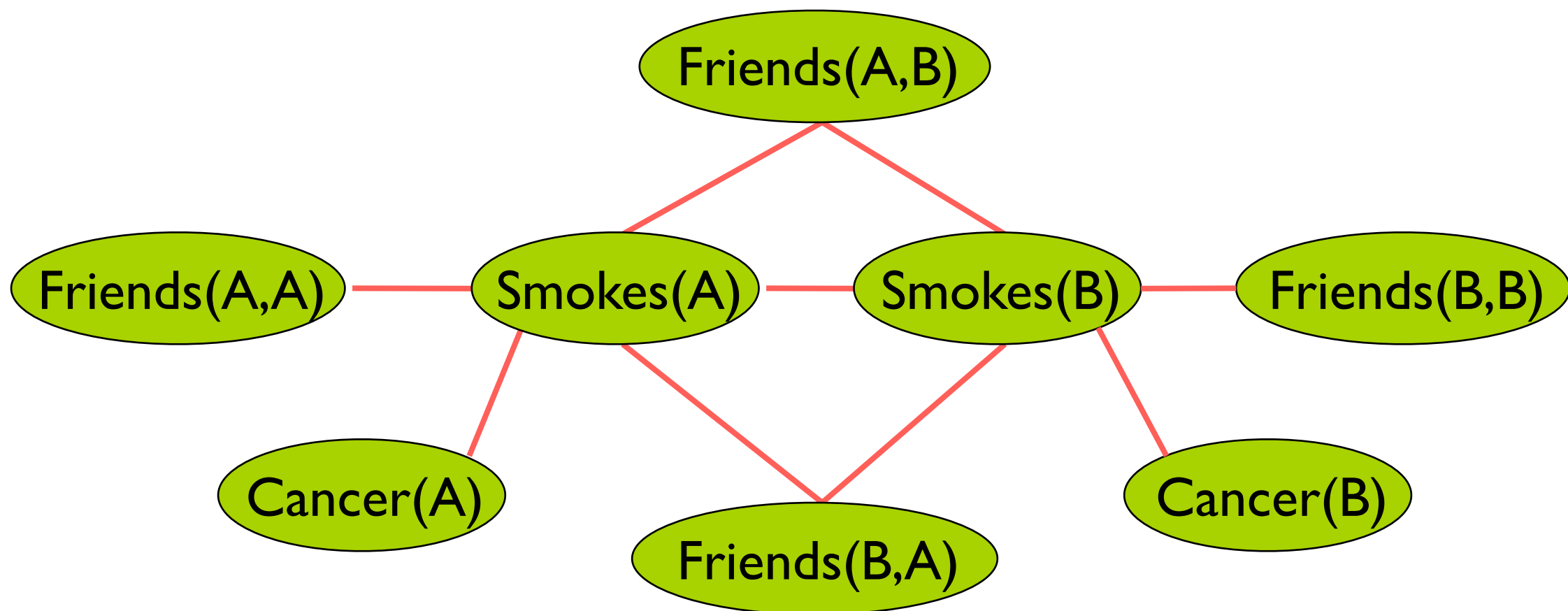


Markov Logic

1.5 $\forall x \text{ Smokes}(x) \Rightarrow \text{Cancer}(x)$

1.1 $\forall x, y \text{ Friends}(x, y) \Rightarrow (\text{Smokes}(x) \Leftrightarrow \text{Smokes}(y))$

Suppose we have two constants: **Anna** (A) and **Bob** (B)



Markov Logic

- A Markov Logic Network (MLN) is a set of pairs (F, w) where
 - F is a formula in first-order logic
 - w is a real number
- An MLN defines a Markov network with
 - One node for each grounding of each predicate in the MLN
 - One feature for each grounding of each formula F in the MLN, with the corresponding weight w
- Probability of a world

$$P(x) = \frac{1}{Z} \exp \left(\sum_i w_i n_i(x) \right)$$

Weight of formula i

No. of true groundings of formula i in x

Possible Worlds

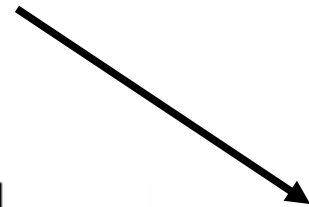
A vocabulary

Smokes(Alice)	Smokes(Bob)	Friends(Alice,Bob)	Friends(Bob,Alice)
0	0	0	0
⋮	⋮	⋮	⋮
1	0	1	0
⋮	⋮	⋮	⋮
1	1	1	1

Possible worlds
Logical interpretations

Possible Worlds

A logical theory



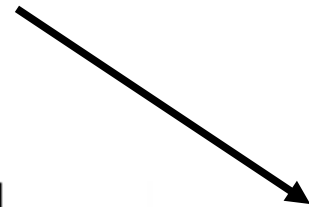
$$\forall x,y, \text{Smokes}(x) \wedge \text{Friends}(x,y) \Rightarrow \text{Smokes}(y)$$

Smokes(Alice)	Smokes(Bob)	Friends(Alice,Bob)	Friends(Bob,Alice)	theory
0	0	0	0	1
⋮	⋮	⋮	⋮	⋮
1	0	1	0	0
⋮	⋮	⋮	⋮	⋮
1	1	1	1	1

Interpretations that
satisfy the theory
Models

First-Order Model Counting

A logical theory



$$\forall x,y, \text{Smokes}(x) \wedge \text{Friends}(x,y) \Rightarrow \text{Smokes}(y)$$

Smokes(Alice)	Smokes(Bob)	Friends(Alice,Bob)	Friends(Bob,Alice)	theory
0	0	0	0	1
⋮	⋮	⋮	⋮	⋮
1	0	1	0	0
⋮	⋮	⋮	⋮	⋮
1	1	1	1	1

First-order model count
 $\sim \#SAT$

Markov Logic

- MLNs are a template for ground Markov Networks
- Probability of a world/interpretation
- If $n_i = 0$ then $P(x) = \frac{1}{Z}$

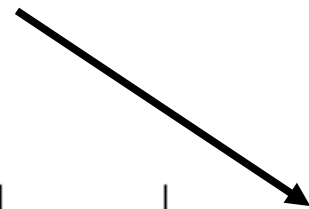
$$P(x) = \frac{1}{Z} \exp \left(\sum_i w_i n_i(x) \right)$$

Weight of formula i

No. of true groundings of formula i in x

Markov Logic

A Markov Logic theory



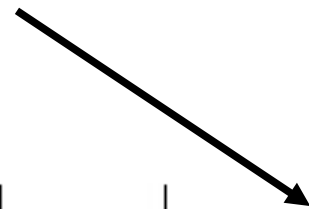
Smokes(Alice)	Smokes(Bob)	Friends(Alice,Bob)	Friends(Bob,Alice)	theory
0	0	0	0	$\frac{1}{Z} \exp(1.5 * 2)$
⋮	⋮	⋮	⋮	
1	0	1	0	$\frac{1}{Z} \exp(1.5 * 1)$
⋮	⋮	⋮	⋮	
1	1	1	1	$\frac{1}{Z} \exp(1.5 * 2)$

$$1.5 \forall x,y, \text{Smokes}(x) \wedge \text{Friends}(x,y) \Rightarrow \text{Smokes}(y)$$

counting only substitutions for which $X \neq Y$
 $X=\text{Alice}, Y=\text{Bob}$
 $X=\text{Bob}, Y=\text{Alice}$

Markov Logic

A Markov Logic theory



Smokes(Alice)	Smokes(Bob)	Friends(Alice,Bob)	Friends(Bob,Alice)	theory
0	0	0	0	$\frac{1}{Z} \exp(1.5 * 2)$
⋮	⋮	⋮	⋮	
1	0	1	0	$\frac{1}{Z} \exp(1.5 * 1)$
⋮	⋮	⋮	⋮	
1	1	1	1	$\frac{1}{Z} \exp(1.5 * 2)$

$1.5 \forall x,y, \text{Smokes}(x) \wedge \text{Friends}(x,y) \Rightarrow \text{Smokes}(y)$

{ Σ

Z
partition function

Markov Logic

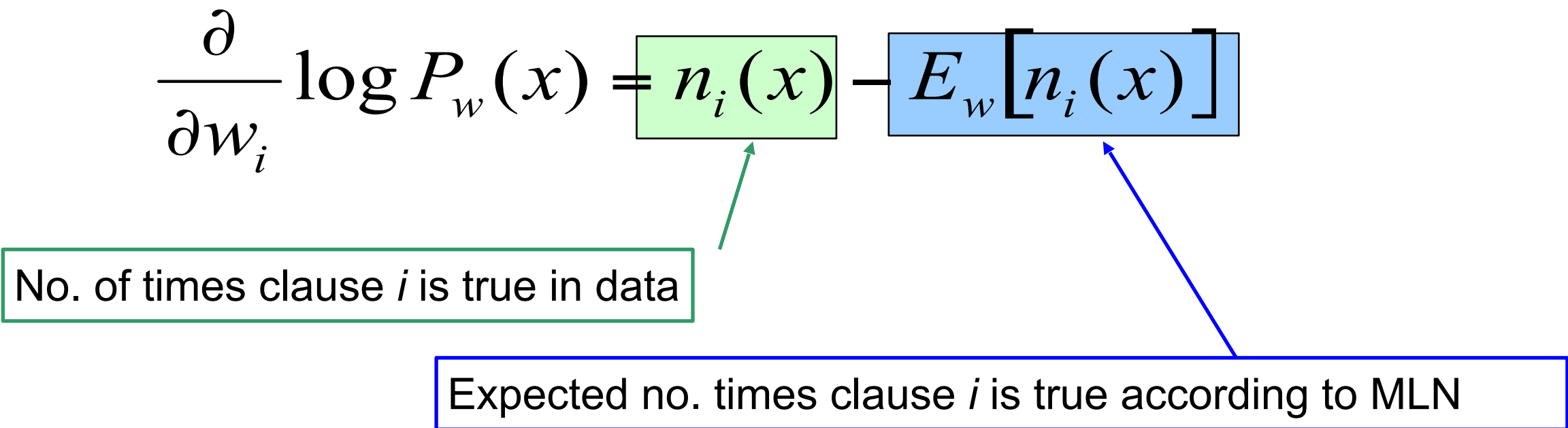
- A Markov Logic Network (MLN) is a set of pairs (F, w) where
 - F is a formula in first-order logic
 - w is a real number
- An MLN defines a Markov network with
 - One node for each grounding of each predicate in the MLN
 - One feature for each grounding of each formula F in the MLN, with the corresponding weight w
- Probability of a world

$$P(x) = \frac{1}{Z} \exp \left(\sum_i w_i n_i(x) \right)$$

Weight of formula i

No. of true groundings of formula i in x

Parameter Learning

$$\frac{\partial}{\partial w_i} \log P_w(x) = \boxed{n_i(x)} - \boxed{E_w[n_i(x)]}$$


No. of times clause i is true in data

Expected no. times clause i is true according to MLN

Has been used for generative learning (Pseudolikelihood);
Many variations (also discriminative);
applications in networks, NLP, bioinformatics, ...

Applications

- Natural language processing, Collective Classification, Social Networks, Activity Recognition, ...

Alchemy: Open Source AI

Tutorial

Mailing Lists

[Alchemy](#)

[Alchemy-announce](#)

[Alchemy-update](#)

[Alchemy-discuss](#)

Repositories

[Code](#)

[Datasets](#)

[MLNs](#)

[Publications](#)

Related Links

Welcome to the Alchemy system! Alchemy is a software package providing a series of algorithms for statistical relational learning and probabilistic logic inference, based on the Markov logic representation. Alchemy allows you to easily develop a wide range of AI applications, including:

- Collective classification
- Link prediction
- Entity resolution
- Social network modeling
- Information extraction

Choose a version of Alchemy:

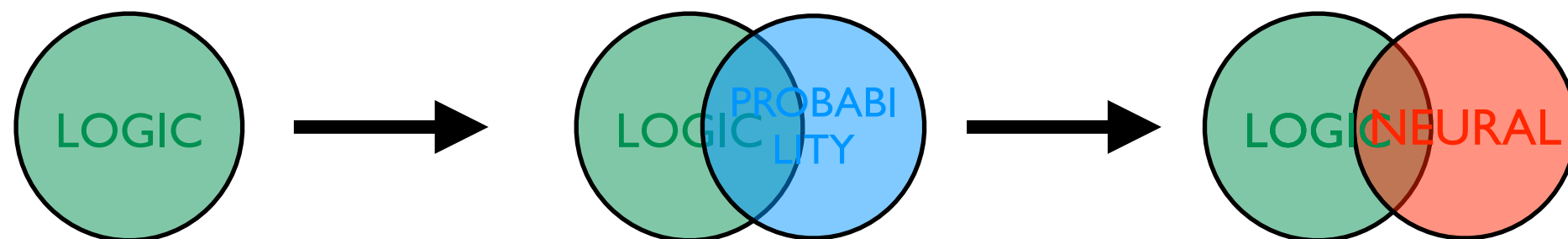
[Alchemy Lite](#)

Alchemy Lite is a software package for inference in Tractable Markov Logic (TML), the first tractable first-order probabilistic logic. Alchemy Lite allows for fast, exact inference for models formulated in TML. Alchemy Lite can be used in batch or interactive mode.

Why StarAI ?

- Reasoning (Probability + Logic) AND Learning
- SRL : Expressive Probabilistic Graphical Models
 - First order logic results supports entities + relationships + background knowledge — abstraction of multiple entities
 - Recursion (e.g. smokers cannot be represented by a plate model)
- PP : Power of a universal Turing machine = a prog. language
 - you can program in it and have builtin expressive prob. models
 - PP can learn -> so bring learning to programming languages
- ProbLog fits both paradigms

From StarAI to NESY

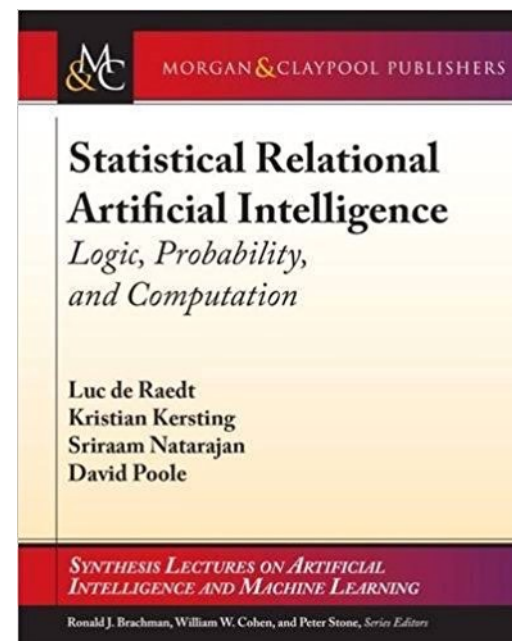
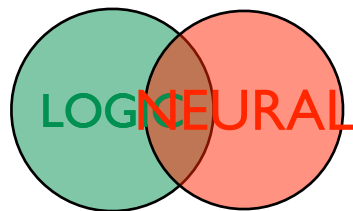


Two types of Neurosymbolic Systems

Logic in the architecture

Logic as a kind
of *neural program*

directed probabilistic
graphical models
(PGMs)

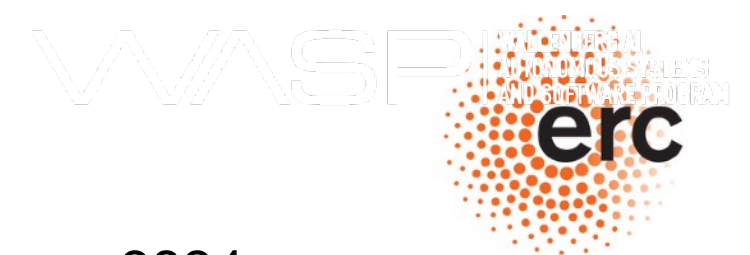


Just like in
StarAI

Logic in the loss

Logic as the
regularizer
(reminiscent of Markov
Logic Networks)

undirected
probabilistic
graphical models
(PGMs)



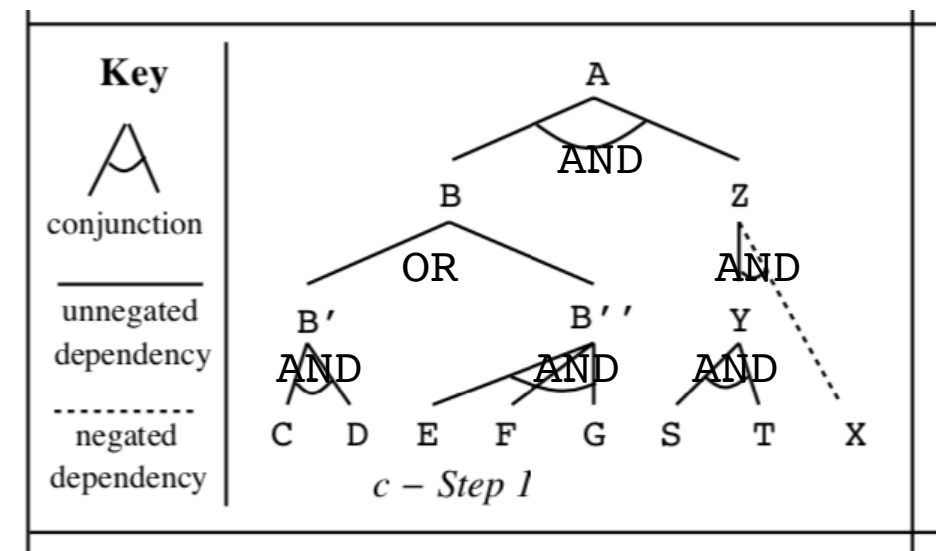
Simple Logic as a neural program

directed PGM approach and logic programs

Logic in the architecture

- KBANN (Towell and Shavlik AIJ 94)
- Turn a (propositional) logic program into a neural network and learn

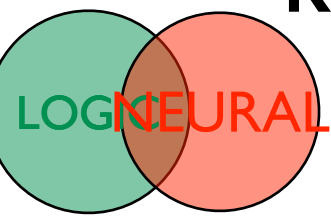
A IF B AND Z.		A IF B AND Z.
B IF C AND D.	→	B IF B'.
B IF E AND F AND G.		B IF B''.
Z IF Y AND not X.		B' IF C AND D.
Y IF S AND T.		B'' IF E AND F AND G.
		Z IF Y AND not X.
		Y IF S AND T.



DATA = (C,D,E,F,G,S,T,X) → A

KNOWLEDGE

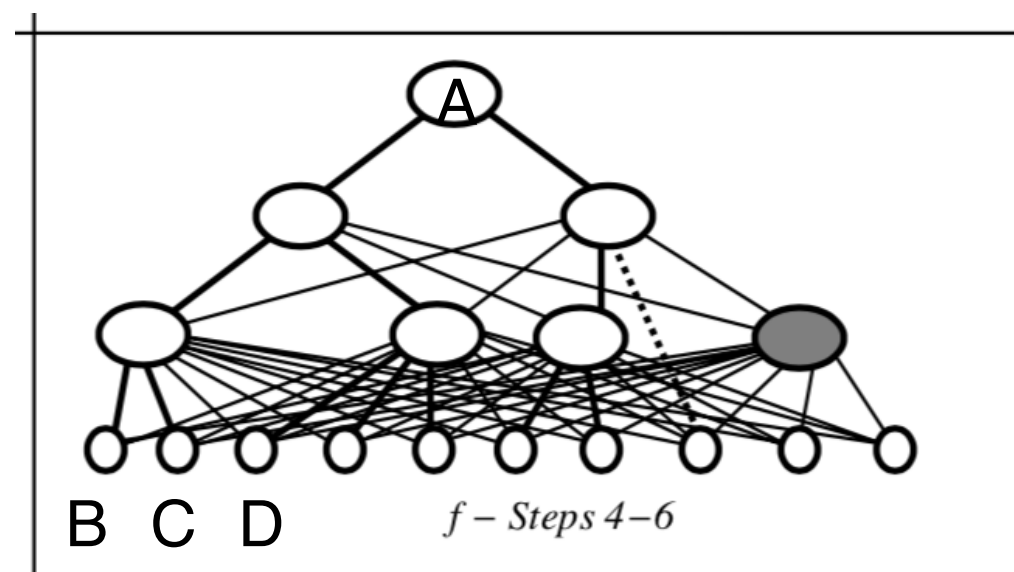
REWRITE



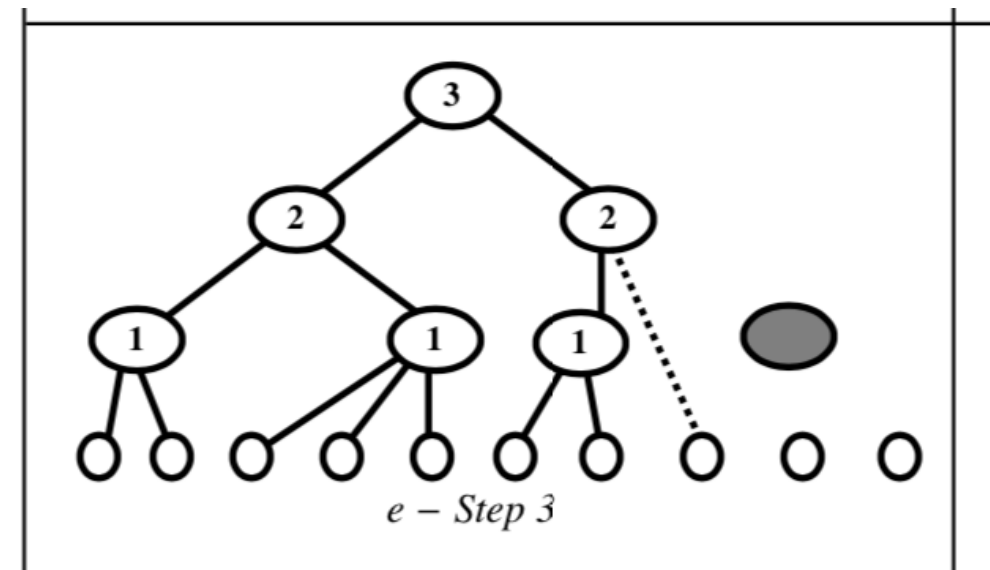
Simple Logic as a neural program

Logic in the architecture

directed PGM approach and logic programs



ADD LINKS — ALSO SPURIOUS ONES



HIDDEN UNIT

and then learn
(Details of activation & loss functions not mentioned)

100

Other Neurosymbolic Systems

Logic in the architecture

directed PGM approach and logic programs

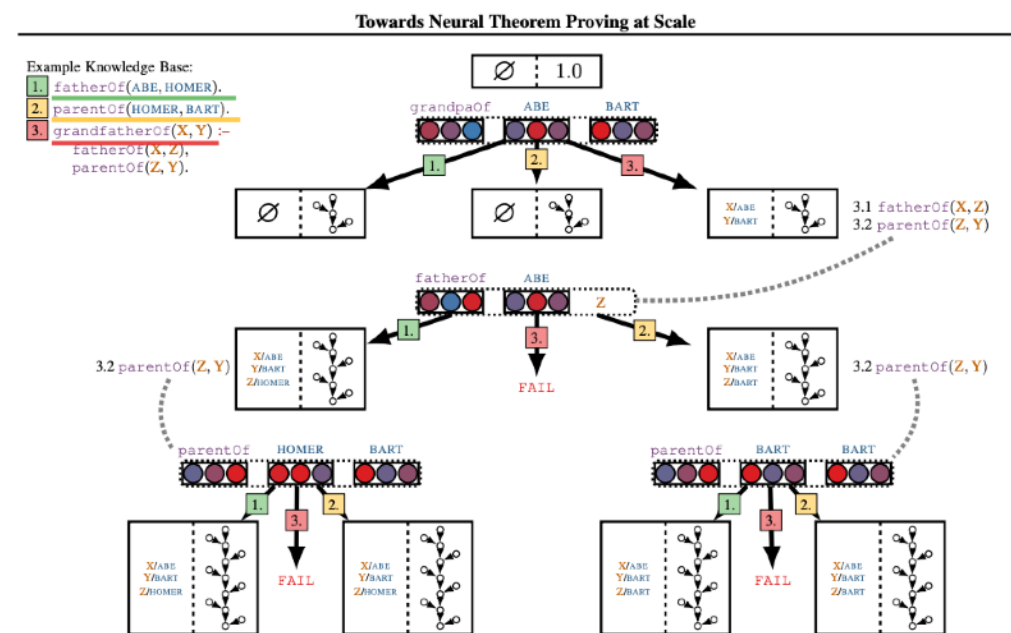
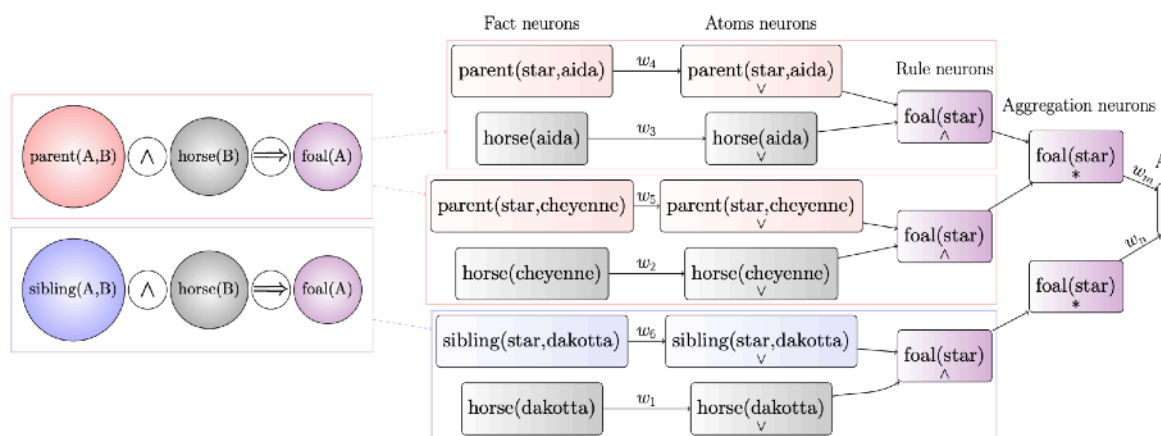
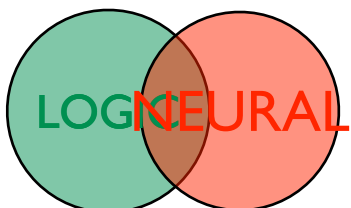


Figure 1. A visual depiction of the NTP' recursive computation graph construction, applied to a toy KB (top left). Dash-separated rectangles denote proof states (left: substitutions, right: proof score -generating neural network). All the non-FAIL proof states are aggregated to obtain the final proof success (depicted in Figure 2). Colours and indices on arrows correspond to the respective KB rule application.

Lifted Relational Neural Nets
[Sourek, Kuzelka, et al JAIR]

The Neural Theorem Prover
[Rocktäschel Riedel, NeurIPS 17; Minervini et al]

the logic becomes the neural net, provides guarantees, but
logic is somehow lost

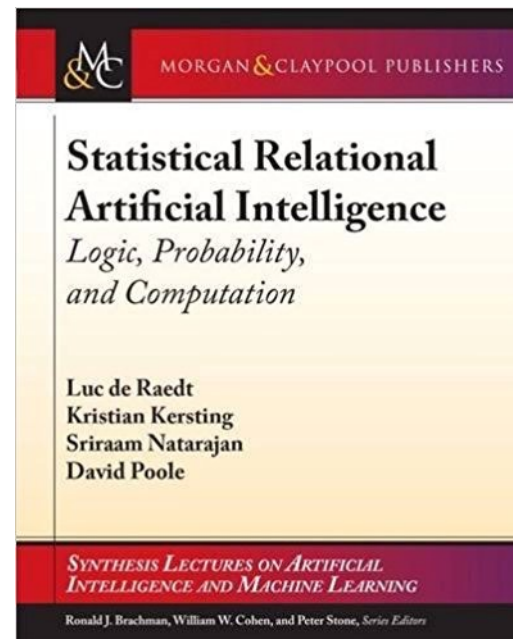
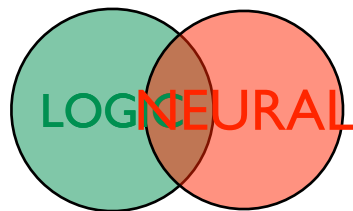


Two types of Neurosymbolic Systems

Logic in the architecture

Logic as a kind
of *neural program*

directed probabilistic
graphical models
(PGMs)

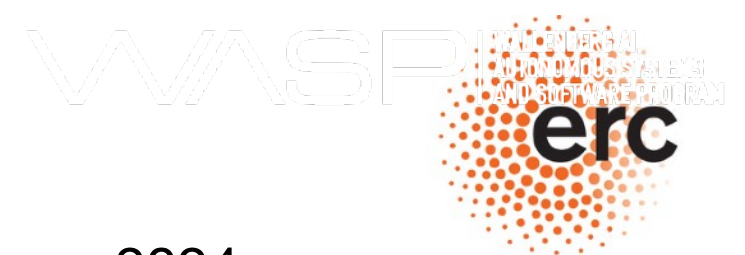


Just like in
StarAI

Logic in the loss

Logic as the
regularizer
(reminiscent of Markov
Logic Networks)

undirected
probabilistic
graphical models
(PGMs)



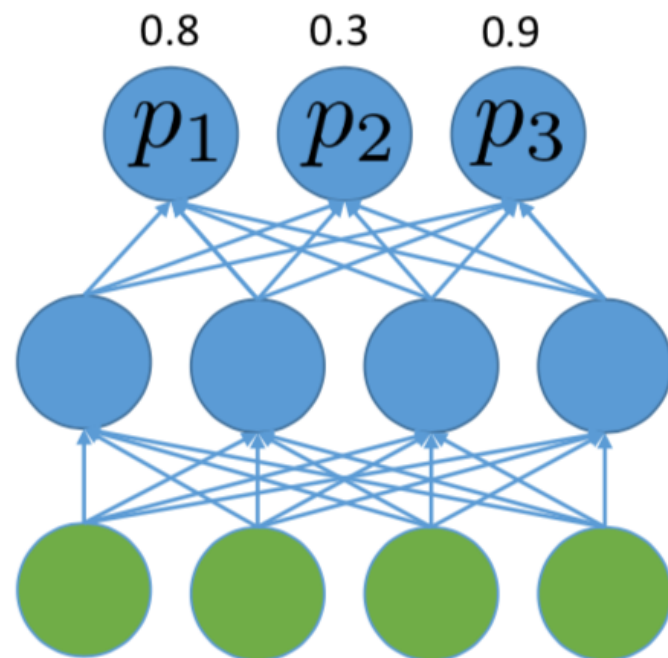
Logic as the regularizer (as in SATisfiability)

Logic in the loss

undirected PGM approach and (soft) constraints



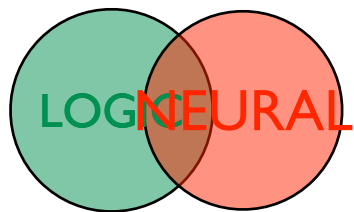
multi-class classification



This constraint should be satisfied

$$\begin{aligned} &(\neg red \wedge \neg orange \wedge green) \vee \\ &(\neg red \wedge orange \wedge \neg green) \vee \\ &(red \wedge \neg orange \wedge \neg green) \end{aligned}$$

KNOWLEDGE



figures and example from Xu et al., ICML 2018

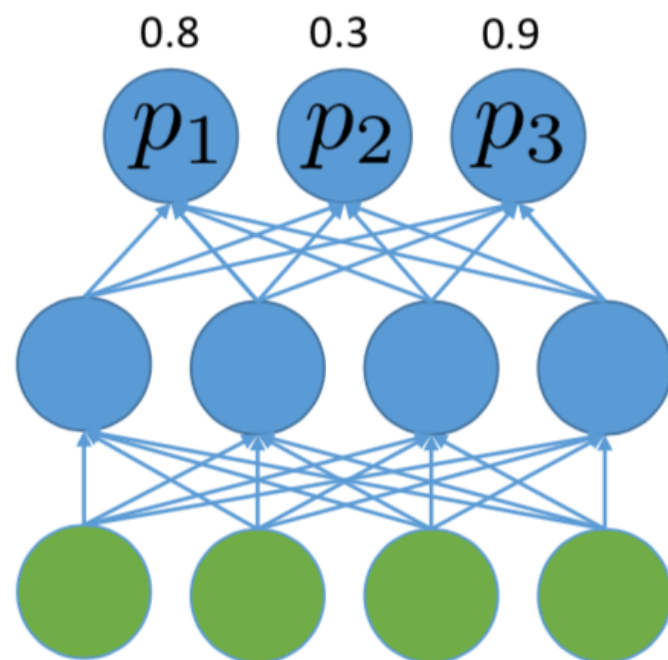
Logic as the regularizer (as in SATisfiability)

Logic in the loss

undirected PGM approach and (soft) constraints



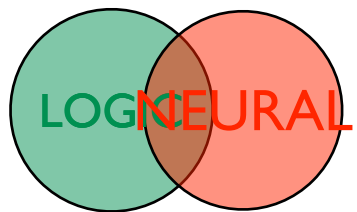
multi-class classification



Probability that constraint is satisfied

$$(1 - red)(1 - orange)green + (1 - red)orange(1 - green) + red(1 - orange)(1 - green)$$

KNOWLEDGE



figures and example from Xu et al., ICML 2018

Logic as the regularizer

Logic in the loss

undirected PGM approach and (soft) constraints

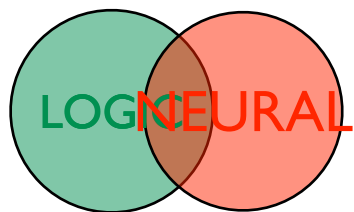
Semantic Loss:

- Use logic as constraints (very much like in SAT examples)
- Semantic loss (- log probability that constraint is satisfied)

$$SLoss(T) \propto -\log \sum_{X \models T} \prod_{x \in X} p_i \prod_{\neg x \in X} (1 - p_i)$$

- Used as regulariser

$$Loss = TraditionalLoss + w.SLoss$$



figures and example from Xu et al., ICML 2018

Logic as the regularizer

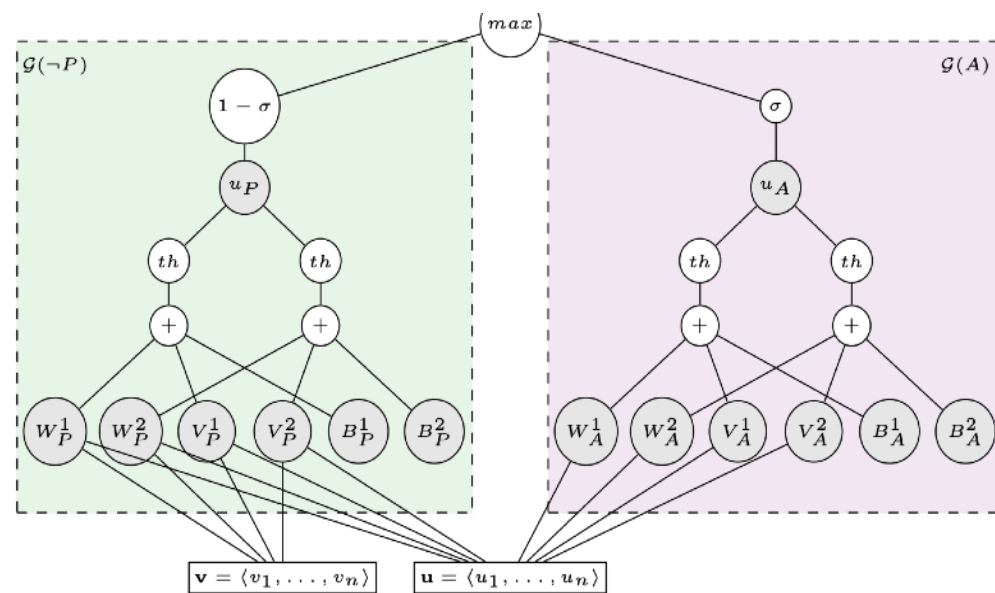
- Semantic Loss can be used with any logical constraint theory
- Examples with semi-supervised learning, where the constraint enforces that each example should have a class
- very nice properties :
 - differentiable, also monotonicity
 - if $\alpha \models \beta$ then $SLoss(\alpha) \geq SLoss(\beta)$

Other Neurosymbolic Systems

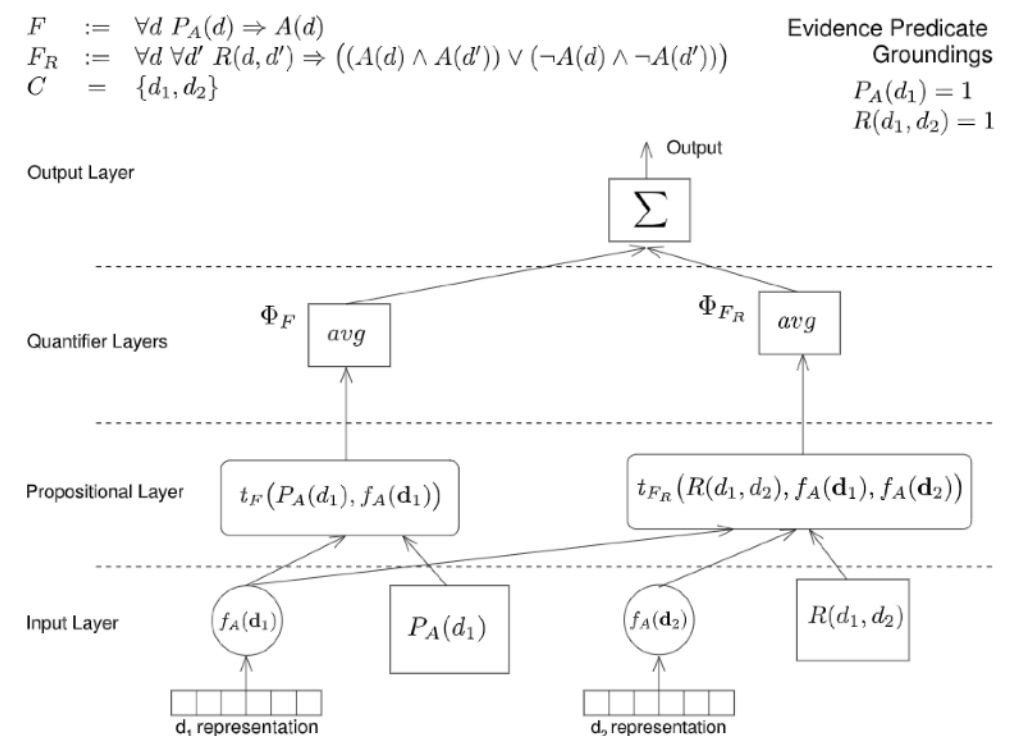
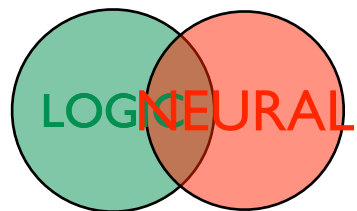
Logic in the loss

undirected PGM approach and (soft) constraints

$P(x, y) \rightarrow A(y)$, with $\mathcal{G}(x) = \mathbf{v}$ and $\mathcal{G}(y) = \mathbf{u}$



Logic Tensor Networks
Serafini & Garcez



Semantic Based Regularization
Diligenti et al. AIJ

the logic becomes a regularizer

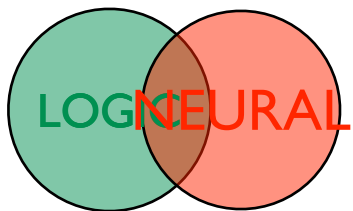
Two types of Neurosymbolic Systems

Logic in the architecture

Logic used at inference time

Hard constraints, come **with guarantees**

Logic built into the model



Logic in the loss

Logic **NOT** used at inference time, only during learning for nudging the neural nerts

Soft constraints, comes **without** guarantees

Logic outside the model



Neurosymbolic AI: Common Principles and Building Blocks

Towards a theory

One more Ingredient

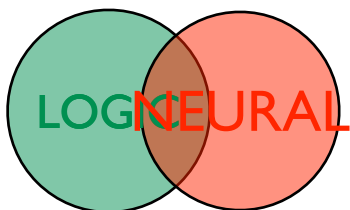
The NEURAL Predicate

perception	???
scalable	?
learning	✓
reasoning	✓
explanations	
knowledge	✓

What about perception ?

How to link the symbol and the subsymbol levels ?

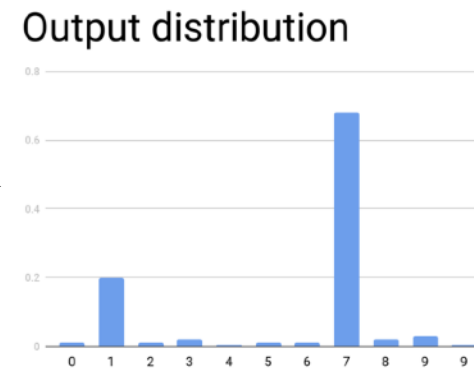
The Neural Predicate



The Neural Predicate



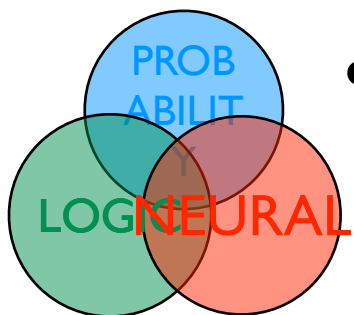
Neural
network



One more
ingredient

MNIST

- Neural networks are functions from input to output
- Input can be (sub)symbolic
- Interpret symbolic output also as a probability/fuzzy distribution
- Neural predicate models the output as probabilistic facts



Key Idea (from DeepProbLog)

unify the basic concepts in
logic and neural networks:

neural predicate ~ neural net

a deep interface between logic
and neural nets

Manhaeve et al. NeurIPS 18, AIJ 21

The Neural Predicate

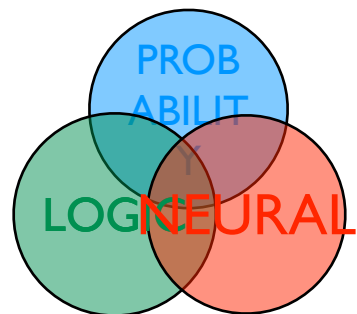
The output of the **digit** neural network is a set of probabilistic facts (DeepProbLog)

Example:

digit(MNIST, Y)
 input NN output NN

Semantics : probabilistic distribution over facts

$P(\text{digit}(\text{1}, 0)) = .04$; $P(\text{digit}(\text{1}, 1)) = .35 \dots P(\text{digit}(\text{1}, 7)) = .53 \dots$
 $P(\text{digit}(\text{1}, 9)) = .14$



needs to be learned end-to-end from DISTANT supervision

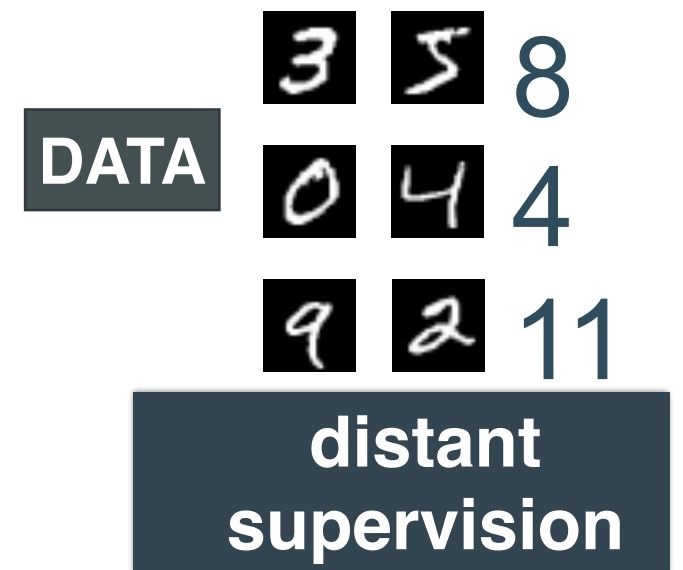


The Neural Predicate Exemplified : MNIST addition

Task: Classify pairs of MNIST digits with their sum

Benefit of Neurosymbolic AI (here DeepProbLog)

- Definition of **addition** known (= knowledge)
- How **to classify digits** is **unknown** and has to be learned



The rule of addition

```
addition(MNIST1,MNIST2,Z) IF  
  digit(MNIST2,N1), digit(MNIST2,N2), Z = N1+N2.  
(how to add to numbers)
```

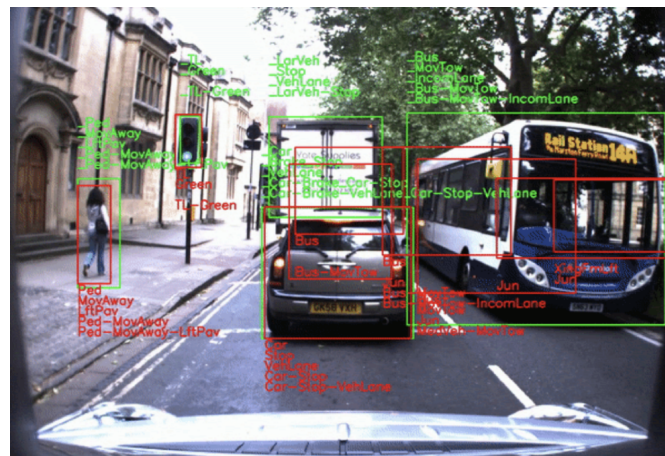
KNOWLEDGE

Examples:

```
addition( 3 , 5 ,8), addition( 0 , 4 ,4), addition( 9 , 2 ,11), ...
```



AI Model = DATA + KNOWLEDGE



Natural Language Explanations

- If an agent pushes an object then it is a pedestrian
- A pedestrian can only push objects, move away, etc.
- Only pedestra~~ins~~ns, cars, cyclists, etc. can cross from left
- Only pedestrians and cyclists can wait to cross
- Only pedestrians, cars, cyclists, etc can stop
- Only pedestrians, cars, cyclists, etc can move
- Only pedestrians, cars, cyclists, etc can move towards
- Only pedestrians, cars, cyclists, etc can move away
- An emergency vehicle can only overtake, move away etc.
- Only emergency vehicles, cars etc. can have hazards lights on
- A bus can only overtake, move away move towards etc.
- A medium vehicle can only overtake, move away, move towards etc.

DATA

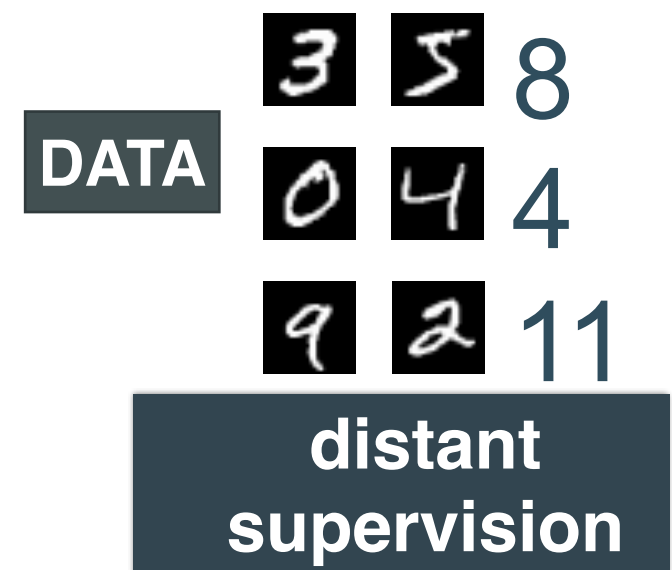
KNOWLEDGE

The Neural Predicate Exemplified : MNIST addition

Task: Classify pairs of MNIST digits with their sum

Benefit of Neurosymbolic AI (here DeepProbLog)

- Definition of **addition** known (= knowledge)
- How **to classify digits** is **unknown** and has to be learned



The rule of addition

```
addition(MNIST1,MNIST2,Z) IF  
  digit(MNIST1,N1), digit(MNIST2,N2), Z = N1+N2.  
(how to add two numbers)
```

KNOWLEDGE

Examples:

```
addition(3, 5, 8), addition(0, 4, 4), addition(9, 2, 11), ...
```



The Neural Predicate Exemplified : MNIST addition

Task: Classify pairs of MNIST digits with their sum

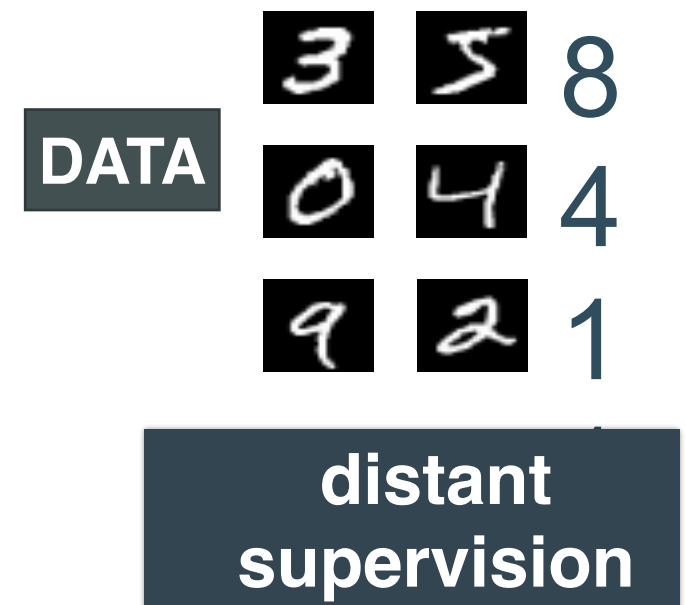
Benefit of Neurosymbolic AI (here DeepProbLog)

- Definition of **addition** known (= knowledge)
- How **to classify digits** is **unknown** and has to be learned

The rule of addition becomes a constraint

```
addition(MNIST1,MNIST2,Z) IF  
  digit(MNIST1,N1), digit(MNIST2,N2), Z = N1+N2. (how to add two numbers)
```

```
addition(3,5,8) IF  
  digit(3,N1), digit(5,N2), 8 = N1+N2. (how to add two numbers)
```

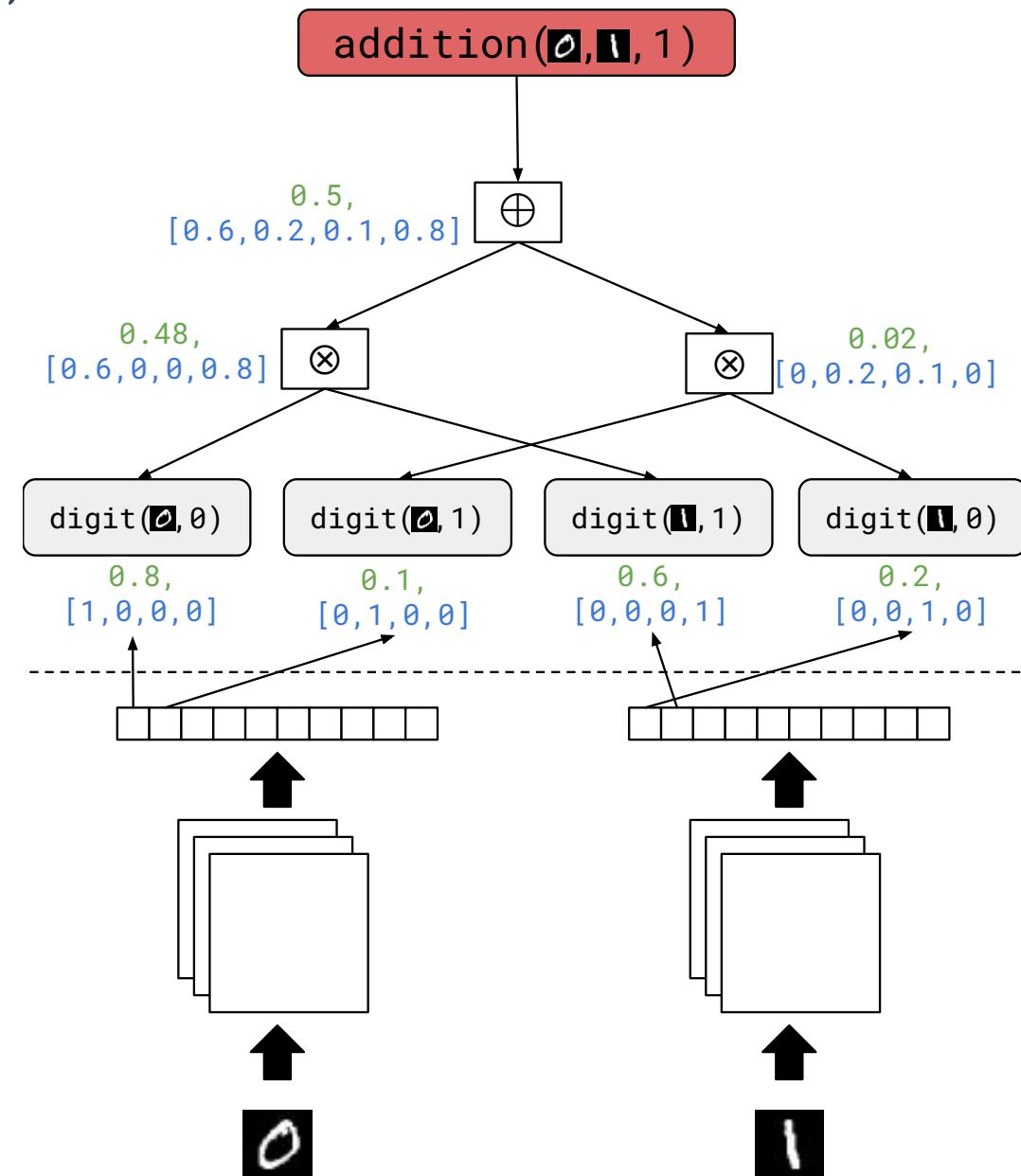
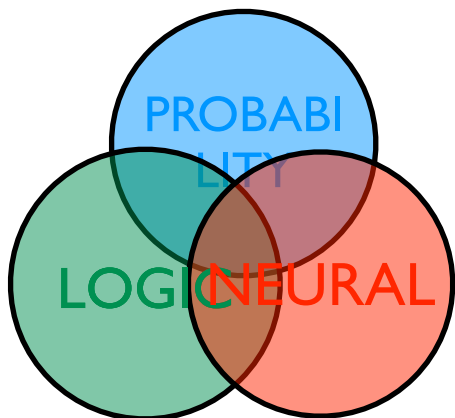


DeepProbLog

```
nn(mnist_net, [X], Y, [0 ... 9] ) ::  
    digit(X,Y).
```

```
addition(X,Y,Z) :-  
    digit(X,N1),  
    digit(Y,N2),  
    Z is N1+N2.
```

The ACs are differentiable and there is an interface with the neural nets



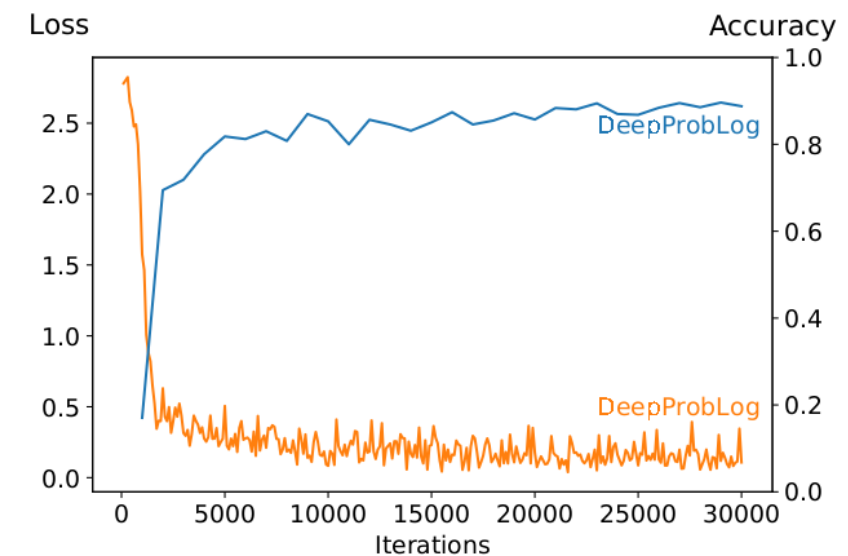
The Neural Predicate Exemplified : MNIST addition

Learn to classify the sum of pairs of MNIST digits

Individual digits not labeled!

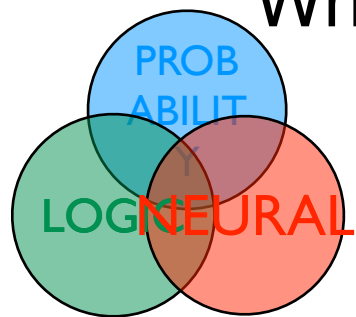
E.g. ( ,  , 8)

Could be done by a CNN: classify the
concatenation of both images into 19 classes



(b) Multi-digit (T2)

What about



     +    = ?

Noisy Addition

```
nn(classifier, [X], Y, [0 .. 9]) :: digit(X,Y).
t(0.2) :: noisy.
```

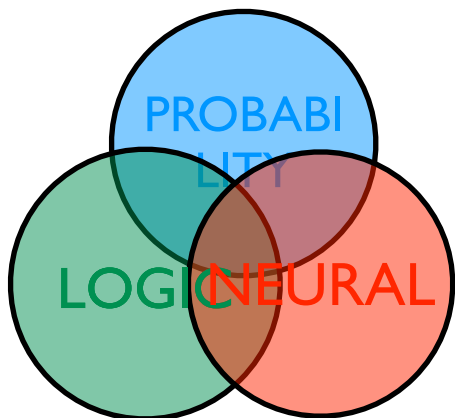
```
1/19 :: uniform(X,Y,0) ; ... ; 1/19 :: uniform(X,Y,18).
```

```
addition(X,Y,Z) :- noisy, uniform(X,Y,Z).
addition(X,Y,Z) :- \+noisy, digit(X,N1), digit(Y,N2), Z is N1+N2.
```

(a) The DeepProbLog program.

	Fraction of noise					
	0.0	0.2	0.4	0.6	0.8	1.0
Baseline	93.46	87.85	82.49	52.67	8.79	5.87
DeepProbLog	97.20	95.78	94.50	92.90	46.42	0.88
DeepProbLog w/ explicit noise	96.64	95.96	95.58	94.12	73.22	2.92
Learned fraction of noise	0.000	0.212	0.415	0.618	0.803	0.985

Table 3: The accuracy on the test set for **T4**.



Program Induction/Sketching

In Neural Symbolic methods

- Rule Induction — work with templates

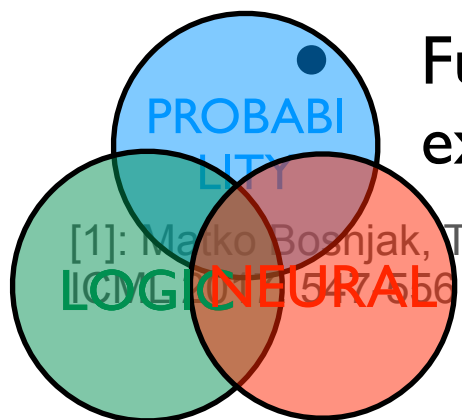
$P(X) \text{ :- } R(X,Y), Q(Y)$

- and have the “predicate” variables / slots P,Q, R determined by the NN
- Simpler form, fill just a few slots / holes

Approach similar to ‘*Programming with a Differentiable Forth Interpreter*’ [1] 24

- Partially defined Forth program with slots / holes
- Slots are filled by neural network (encoder / decoder)

Fully differentiable interpreter: NNs are trained with input / output examples



[1]: Marko Boshnjak, Tim Rocktäschel, Jason Naradowsky, Sebastian Riedel: Programming with a Differentiable Forth Interpreter.

Example DeepProbLog

neural predicate

hole(X,Y,X,Y):-
 swap(X,Y,0).

hole(X,Y,Y,X):-
 swap(X,Y,1).

bubble sort

bubble([X],[],X).
bubble([H1,H2|T],[X1|T1],X):-
 hole(H1,H2,X1,X2),
 bubble([X2|T],T1,X).

bubblesort([],L,L).

bubblesort(L,L3,Sorted) :-
 bubble(L,L2,X),
 bubblesort(L2,[X|L3],Sorted).

sort(L,L2) :- bubblesort(L,[],L2).

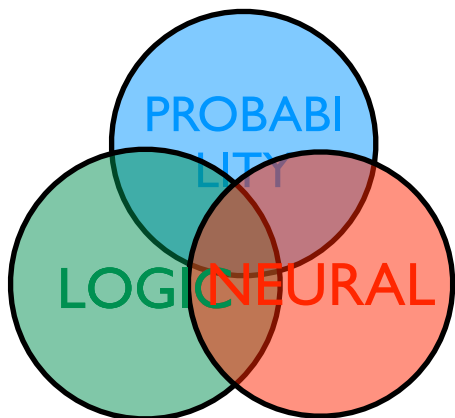
	Test Length	Sorting: Training length					Addition: training length		
		2	3	4	5	6	2	4	8
$\partial 4$ [Bošnjak et al., 2017]	8	100.0	100.0	49.22	–	–	100.0	100.0	100.0
	64	100.0	100.0	20.65	–	–	100.0	100.0	100.0
DeepProbLog	8	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
	64	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

(a) Accuracy on the sorting and addition problems (results for $\partial 4$ reported by Bošnjak et al. [2017]).

Training length $\longrightarrow$	2	3	4	5	6
$\partial 4$ on GPU	42 s	160 s	–	–	–
$\partial 4$ on CPU	61 s	390 s	–	–	–
DeepProbLog on CPU	11 s	14 s	32 s	114 s	245 s

(b) Time until 100% accurate on test length 8 for the sorting problem.

Table 1: Results on the Differentiable Forth experiments



DeepSeaProbLog

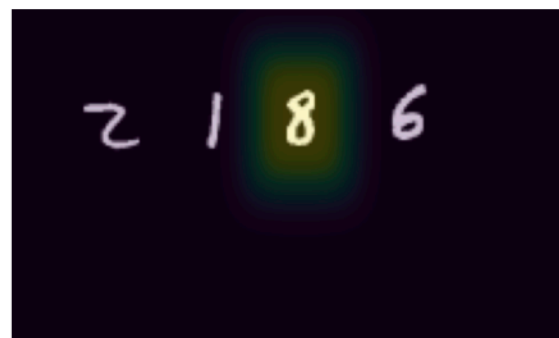
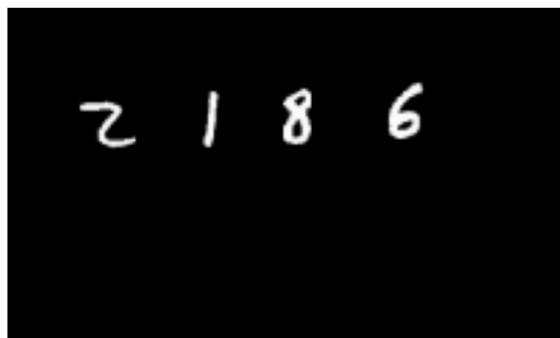
discrete and continuous distributions [De Smet UAI 23]

useful for robotics and perception

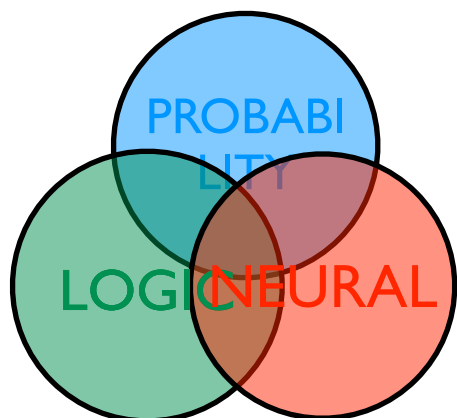
dim is neural net returning parameters of normal distribution.

```
length(Obj) ~ normal(dim(Obj, Image)).
```

```
large(Obj) :- length(Obj) > 100.
```



determining order digits
to determine year



A Recipe for Neurosymbolic AI

Putting knowledge layers in any neural network
Using any type of symbolic formalism

check our survey on AIJ — Marra, Dumancic, Manhaeve & De Raedt, AIJ 24

Neurosymbolic functions

& primitives defining semantics and computational graph

Neural nets

signal-
analysis

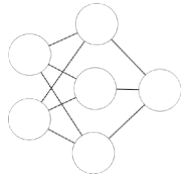
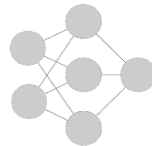
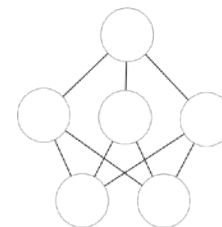


image-
analysis

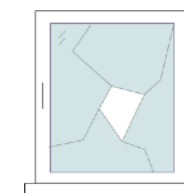
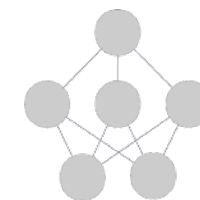


earthquake



sign43

burglary



img55

Neurosymbolic functions

& primitives defining semantics and computational graph

Neural nets

signal-
analysis

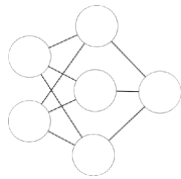
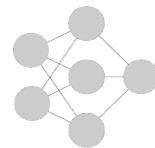


image-
analysis

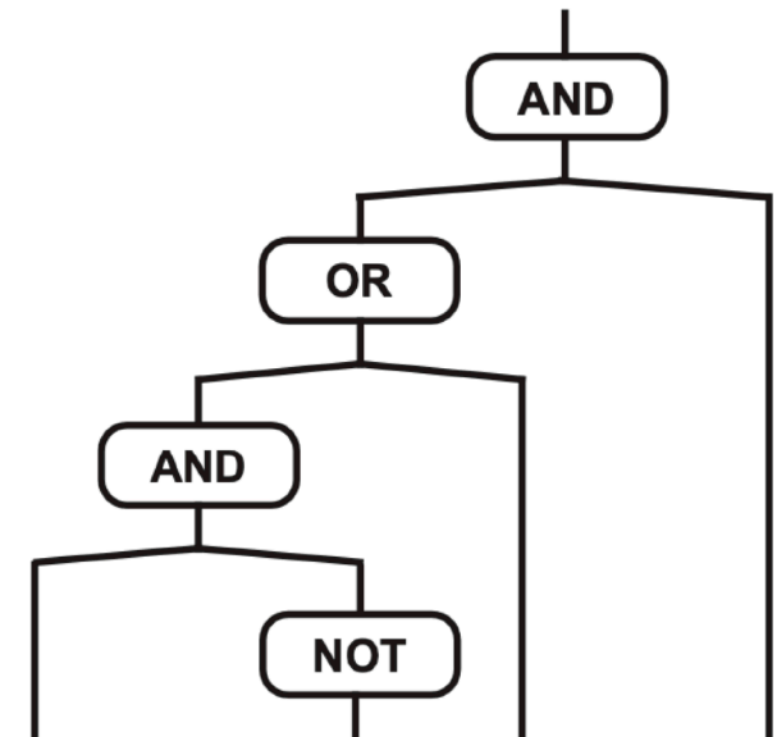


Logic

`alarm(B,E)` IF `earthquake(E)` OR `burglary(B)`
`calls(B,E,P)` IF `alarm(B,E)` AND `hears_alarm(P)`

`hears_alarm(john)`
`hears_alarm(mary)`

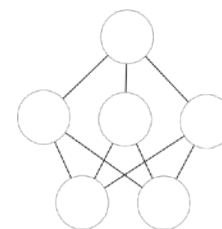
`calls(img55,sign43,john)`



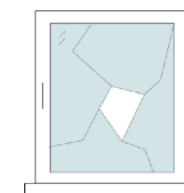
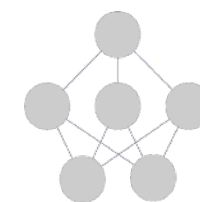
earthquake

burglary

hears_alarm(john)



sign43



img55

Neurosymbolic functions

& primitives defining semantics and computational graph

Can we learn
calls END TO END ?

Neural nets

signal-
analysis

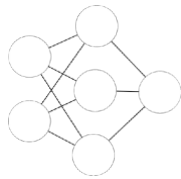
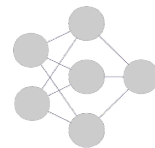


image-
analysis

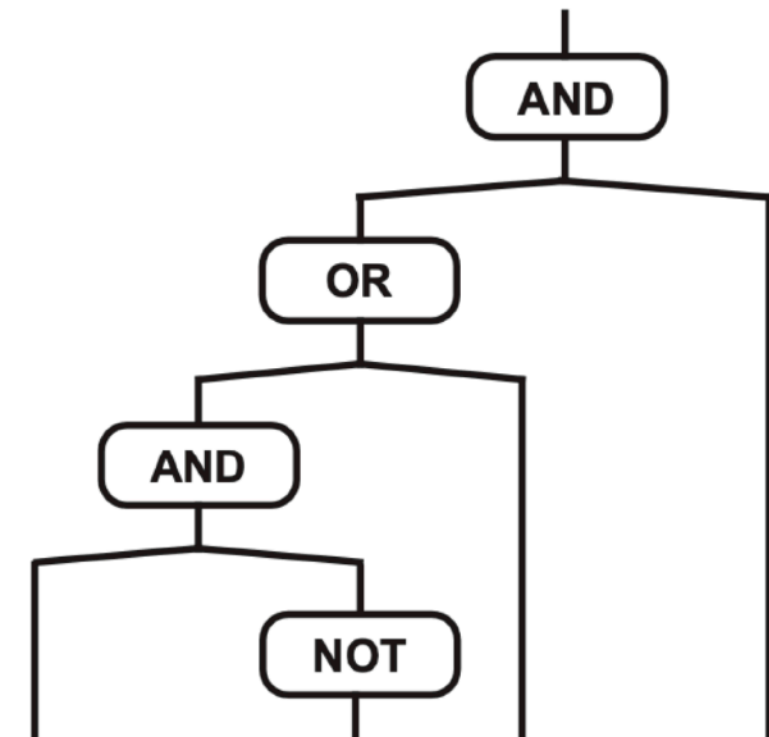


Logic

alarm(B,E) IF earthquake(E) OR burglary(B)
calls(B,E,P) IF alarm(B,E) AND hears_alarm(P)

hears_alarm(john)
hears_alarm(mary)

calls(img55,sign43,john)

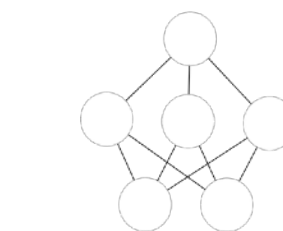


earthquake

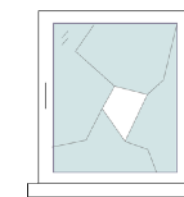
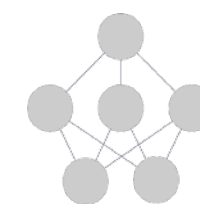
burglary

hears_alarm(john)

Distant Supervision



sign43



img55

← From

Neurosymbolic functions

& primitives defining semantics and computational graph

Neural nets

signal-
analysis

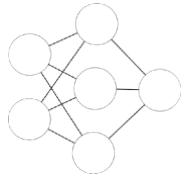
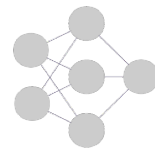


image-
analysis



Logic

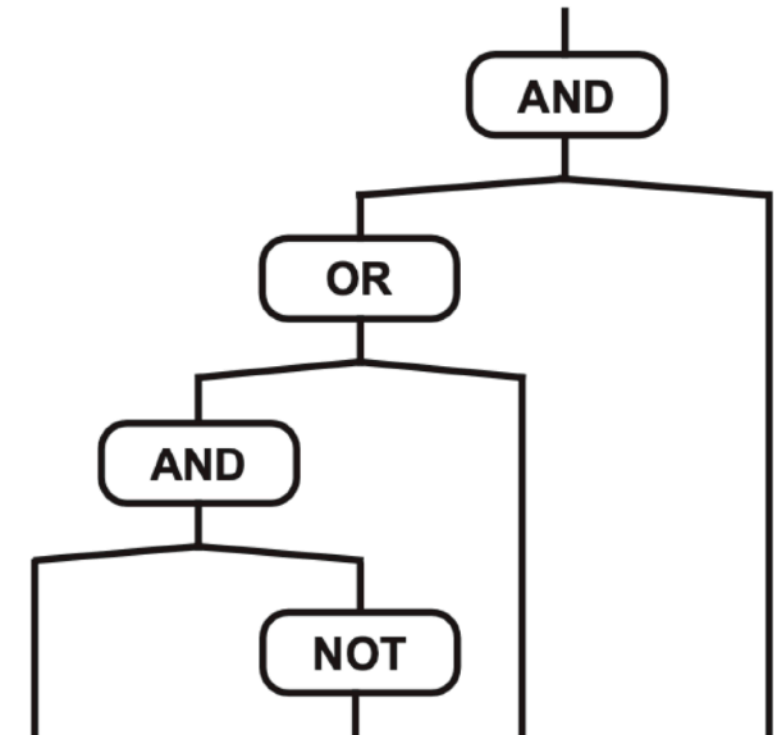
alarm(B,E) IF earthquake(E) OR burglary(B)
calls(B,E,P) IF alarm(B,E) AND hears_alarm(P)

hears_alarm(john)
hears_alarm(mary)

Probability (or fuzzy)

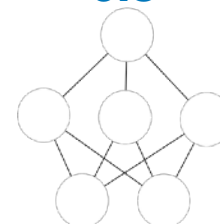
0.8::earthquake
0.01::burglary
0.3::hears_alarm(john)

0.2406 calls(img55,sign43,john)



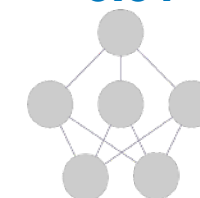
earthquake

0.8



burglary

0.01

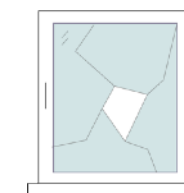


hears_alarm(john)

0.3



sign43



img55

Neurosymbolic functions

& primitives defining semantics and computational graph

0.2406 calls(img55,sign43,john)

Neural nets

signal-
analysis

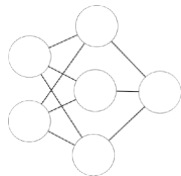
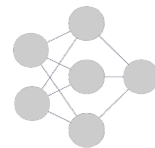


image-
analysis



Logic

alarm(B,E) IF earthquake(E) OR burglary(B)
calls(B,E,P) IF alarm(B,E) AND hears_alarm(P)

hears_alarm(john)
hears_alarm(mary)

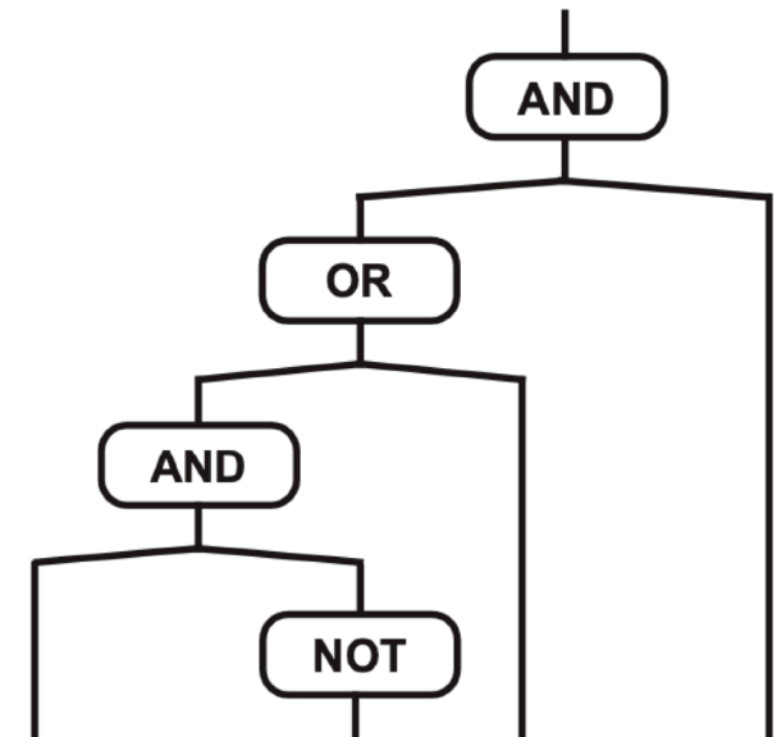
Probability (or fuzzy)

0.8::earthquake
0.01::burglary
0.3::hears_alarm(john)

Deep bidirectional interface

Neurosymbolic primitive = the neural predicate

burglary(B) = neural(image-analysis(B))
earthquake(E) = neural(signal-analysis(E))



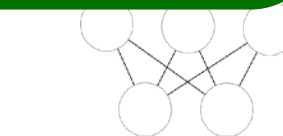
earthquake

burglary

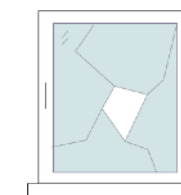
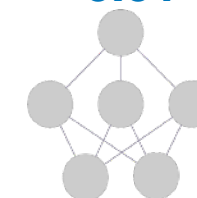
hears_alarm(john)

0.01

0.3



sign43



img55

Neurosymbolic functions

& primitives defining semantics and computational graph

Semantics = Computational Graph

0.2406 calls(img55,sign43,john)

Neural nets

signal-
analysis

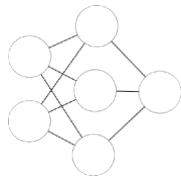
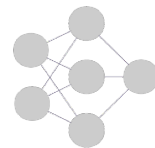


image-
analysis



Logic

alarm(B,E) IF earthquake(E) OR burglary(B)
calls(B,E,P) IF alarm(B,E) AND hears_alarm(P)

hears_alarm(john)
hears_alarm(mary)

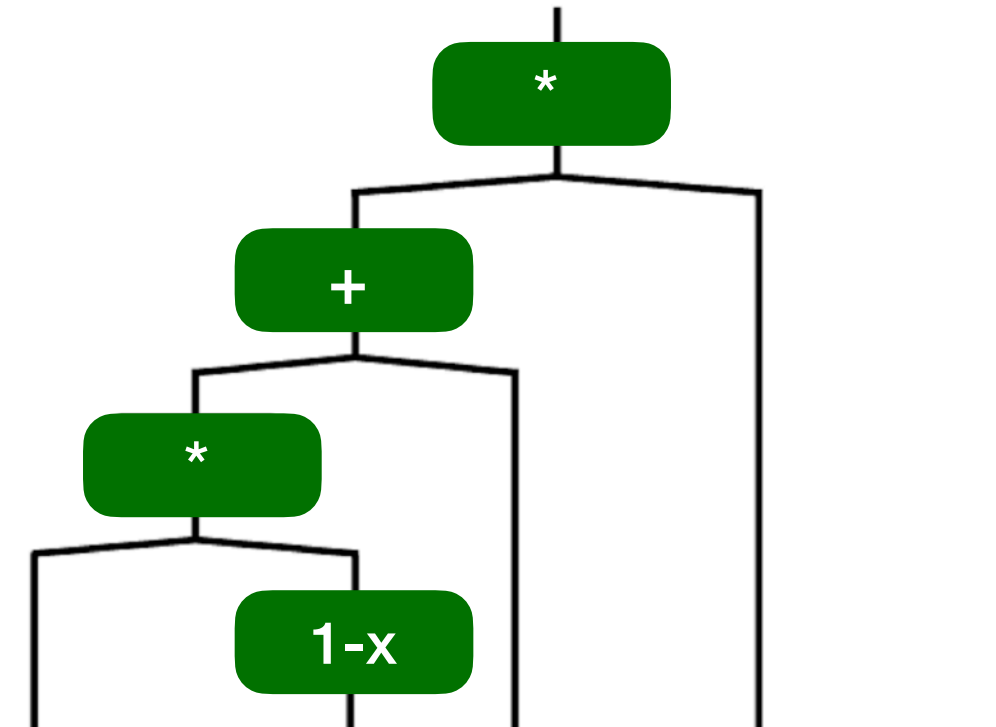
Probability (or fuzzy)

0.8::earthquake
0.01::burglary
0.3::hears_alarm(john)

Deep bidirectional interface

Neurosymbolic primitive = the neural predicate

burglary(B) = neural(image-analysis(B))
earthquake(E) = neural(signal-analysis(E))



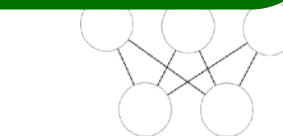
earthquake

burglary

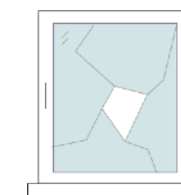
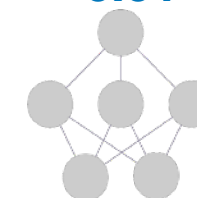
hears_alarm(john)

0.01

0.3



sign43



img55

Semantics = Computational Graph



Inference and learning (end-to-end)

Many variations, many challenges
Standard gradient descent applies



Explanations

due to the use of a probabilistic logic, quite natural
probabilistic abduction (cf David Poole)

if john calls then it is because

alarm

earthquake (and no burglary)

hears_alarm(john)

you can even edit these programs (knowledge)

alarm

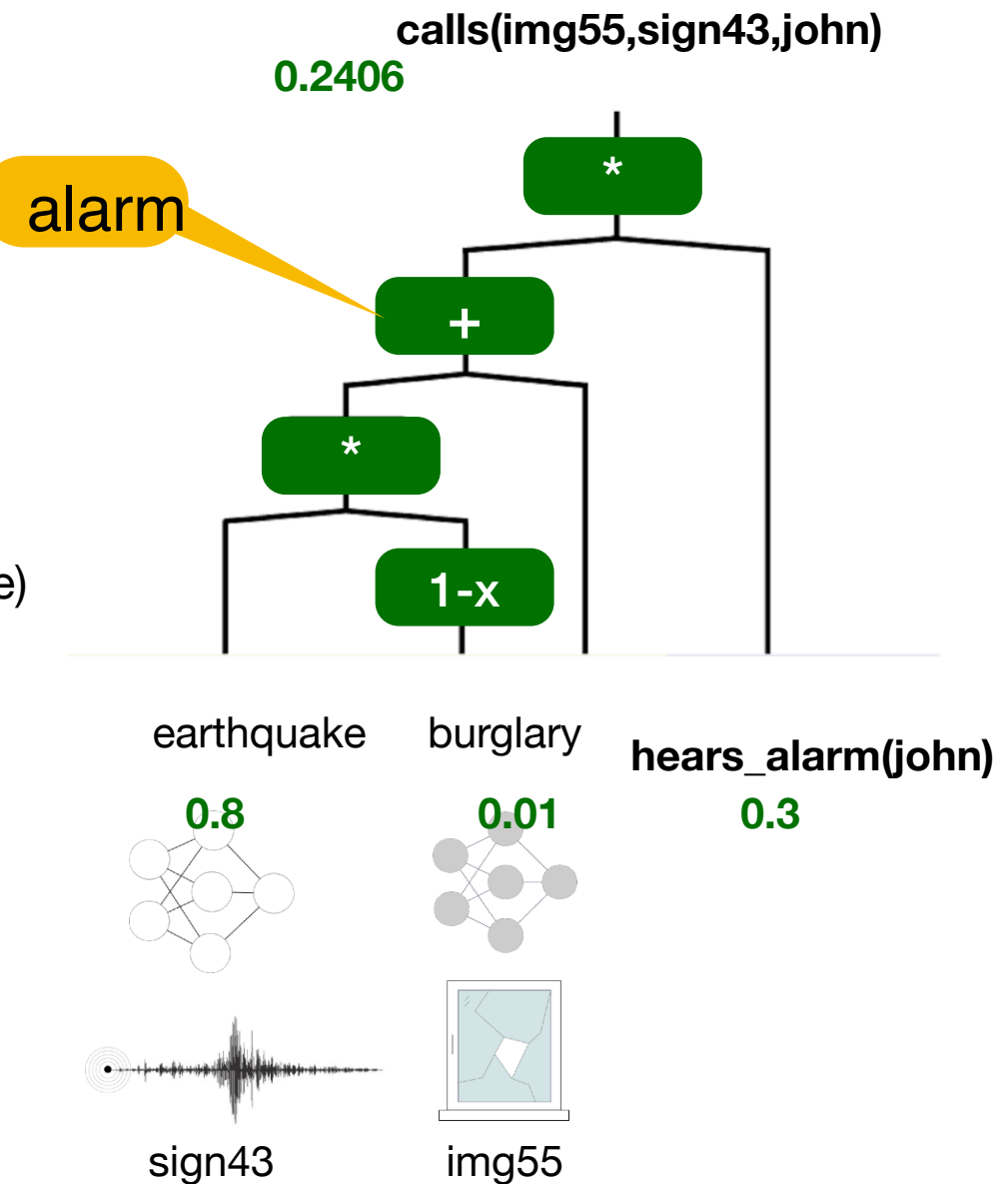
Semantics

How to define semantics ?

Role of components ?

(logic, fuzzy / probability, neural nets)?

Many variations



Semantics = Computational Graph



Inference and learning (end-to-end)

Many variations, many challenges
Standard gradient descent applies



Explanations

due to the use of a probabilistic logic, quite natural
probabilistic abduction (cf David Poole)

if john calls then it is because

alarm

earthquake (and no burglary)

hears_alarm(john)

you can even edit these programs (knowledge)

alarm

Semantics

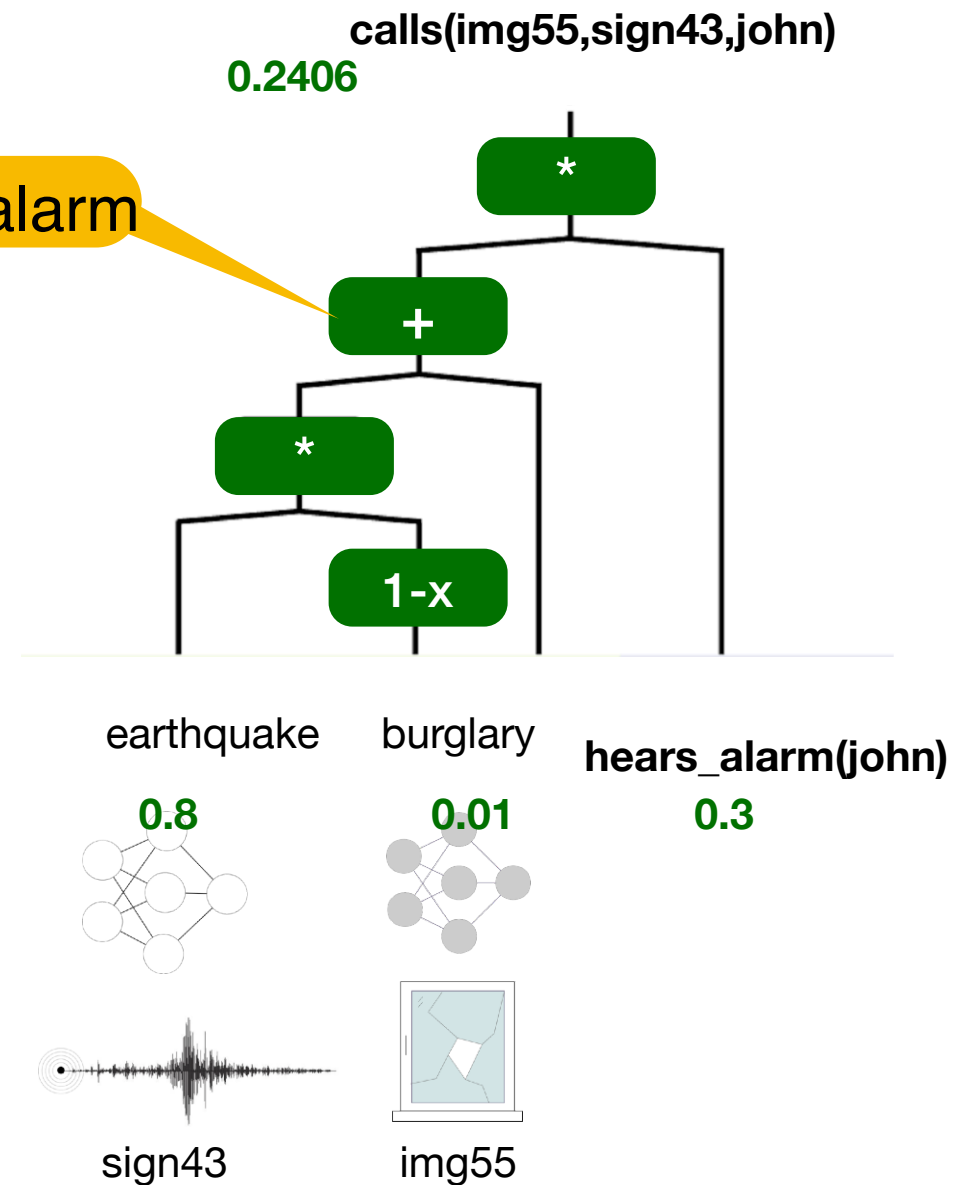
How to define semantics ?

Role of components ?

(logic, fuzzy / probability, neural nets) ?

Many variations

alarm



DeepStochLog

Neural Unification Based

Grammars

An illustration of the RECIPE

Winters et al AAAI 22

Context free grammars

KNOWLEDGE

$E \rightarrow N$

$E \rightarrow E, P, N$

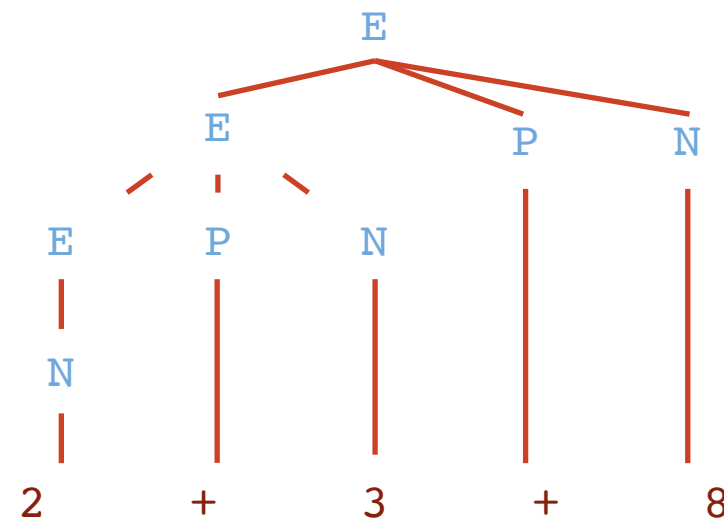
$P \rightarrow ["+"]$

$N \rightarrow ["0"]$

$N \rightarrow ["1"]$

...

$N \rightarrow ["9"]$



DATA

Useful for:

- Is sequence an **element** of the specified language?
- What is the "*part of speech*"-**tag** of a terminal
- **Generate** all elements of language

Probabilistic Context free grammars

0.5 :: E --> N
 0.5 :: E --> E, P, N

1.0 :: P --> ["+"]

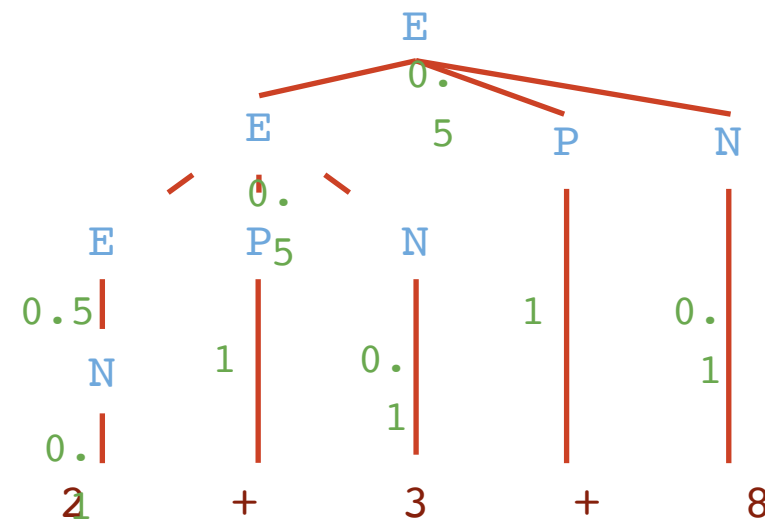
Always sums to 1 per
non-terminal

0.1 :: N --> ["0"]

0.1 :: N --> ["1"]

...

0.1 :: N --> ["9"]



DATA

Probability of this parse = $0.5 * 0.5 * 0.5 * 0.1 * 1 * 0.1 * 1 * 0.1 = 0.000125$

Useful for:

- What is the **most likely parse** for this sequence of terminals? *(useful for ambiguous grammars)*
- What is the **probability of generating** this string?

Neural definite clause grammars = DeepStochLog

0.5 :: E --> N

0.5 :: E --> E, P, N

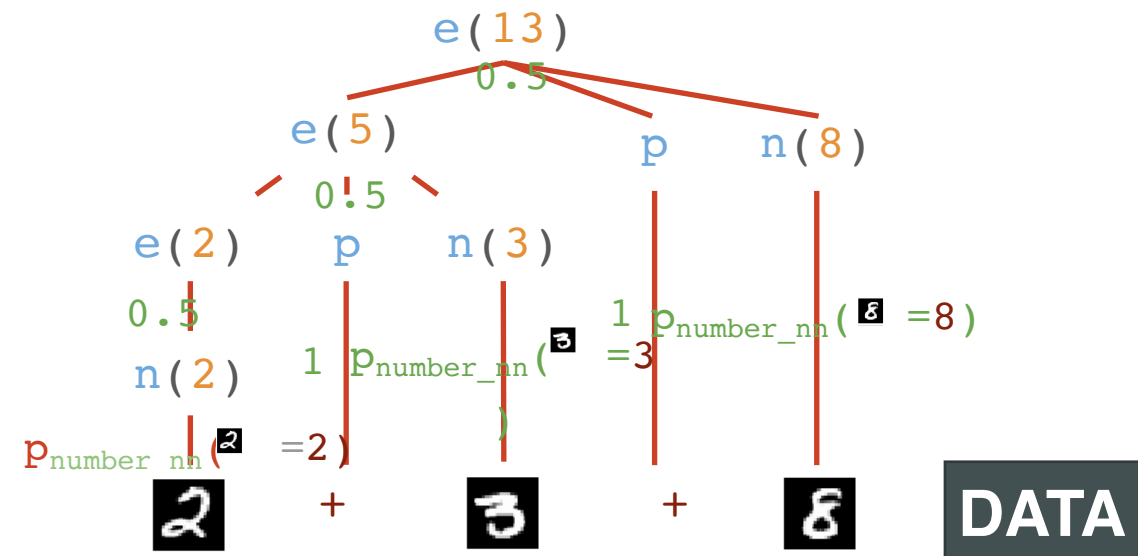
1.0 :: P --> ["+"]

0.1 :: N(0) --> [MNIST]

0.1 :: N(1) --> [MNIST]

...

0.1 :: N(9) --> [MNIST]



Probability of this parse =

$$0.5 * 0.5 * 0.5 * p_{\text{number_nn}}(2=2) * 1 * p_{\text{number_nn}}(3=3) * 1 * p_{\text{number_nn}}(8=8)$$

Useful for:

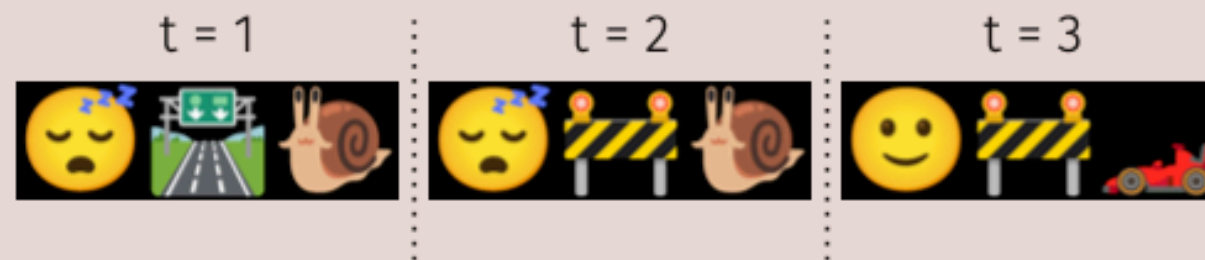
- **Subsymbolic** processing: e.g. tensors as terminals
- Learning rule probabilities using **neural networks**

Neurosymbolic Automata

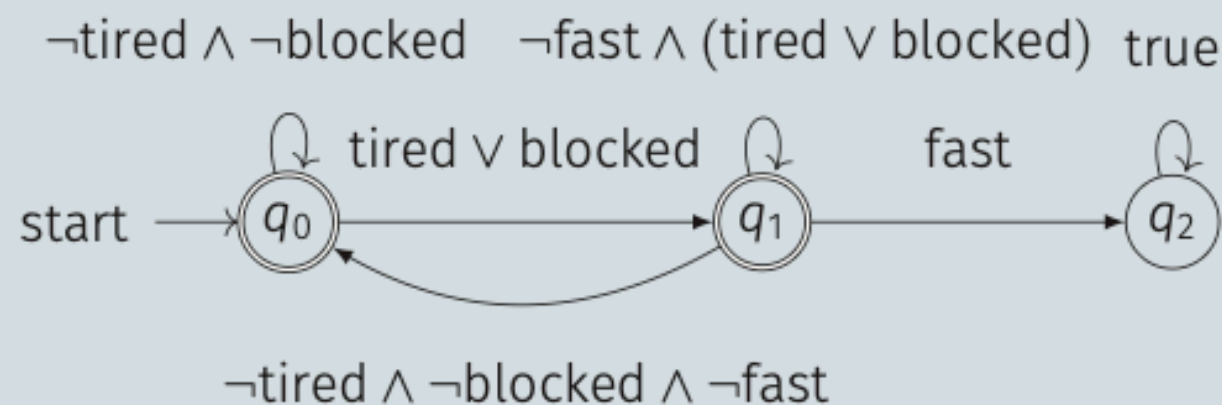
Example

- NeSyA is an integration of **probabilistic automata** and **subsymbolic sequences**

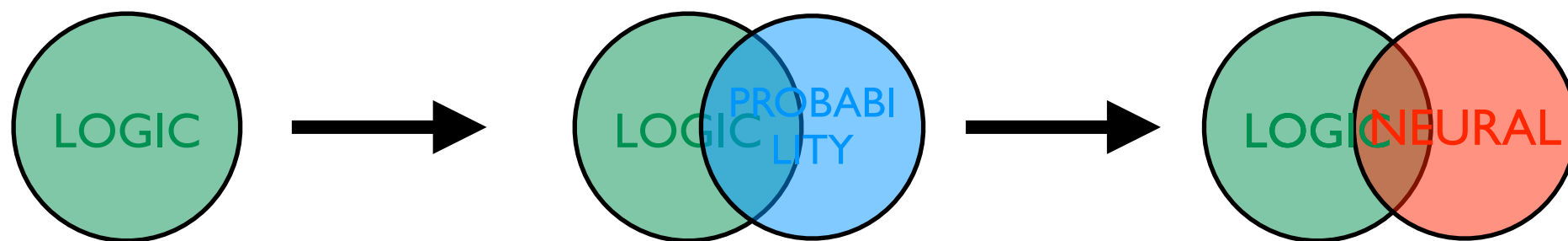
Sequence



Automaton



Dimension 3: Semantics



Semantics

- We are interested in answering the following family of questions:

*Given a **sentence** of a propositional (or propositionalized through grounding) language, what is its **value**?*

The nature of what **value** is differs in the different semantics.

Semantics

For simplicity,

- **labelling function** is the function ℓ_S that assigns, to the **sentence Q**, the value **v** according to **semantics S**.

$$\ell_S(Q) = v$$

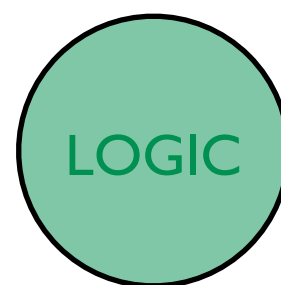
e.g.

$$\ell_B(\textit{human}(\textit{socrates})) = \textit{True}$$

$$\ell_F(\textit{tall}(\textit{john})) = 0.8$$

...

Dimension 3: Semantics



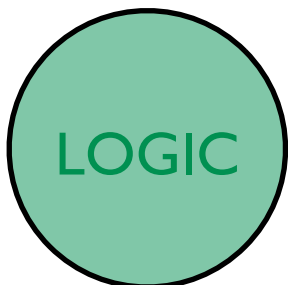
Semantics in Boolean Logic

- Defining a **semantics** for a propositional language L is about **assigning a truth value** to all the sentences of the logic
- Boolean truth values:

$\{True, False\}$

Three steps:

1. Truth values for propositions
2. Truth values for operators
3. Labelling formulas



Semantics in Boolean Logic

1. Providing the **labels** for propositions

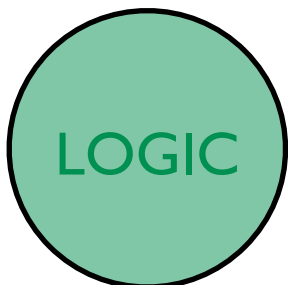
$L = \{burglary, earthquake, hears_alarm(john)\}$

$$\ell_B(burglary) = True$$

$$\ell_B(earthquake) = False$$

$$\ell_B(hears_alarm(john)) = True$$

*This is a **model** or a **possible world**, a “potential” assignment of truth values to all the propositional variables in the language.*



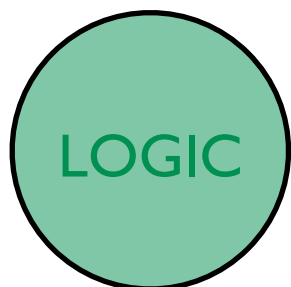
Semantics in Boolean Logic

2. Providing the semantics for operators

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

 $\ell_B^\wedge$

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

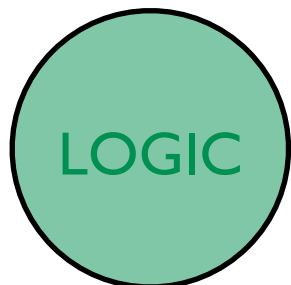
 $\ell_B^{\rightarrow}$ 

Semantics in Boolean Logic

3. The labels of **formulas** are defined **recursively** on the semantics of its components

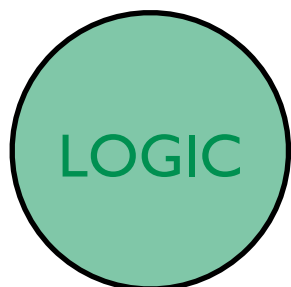
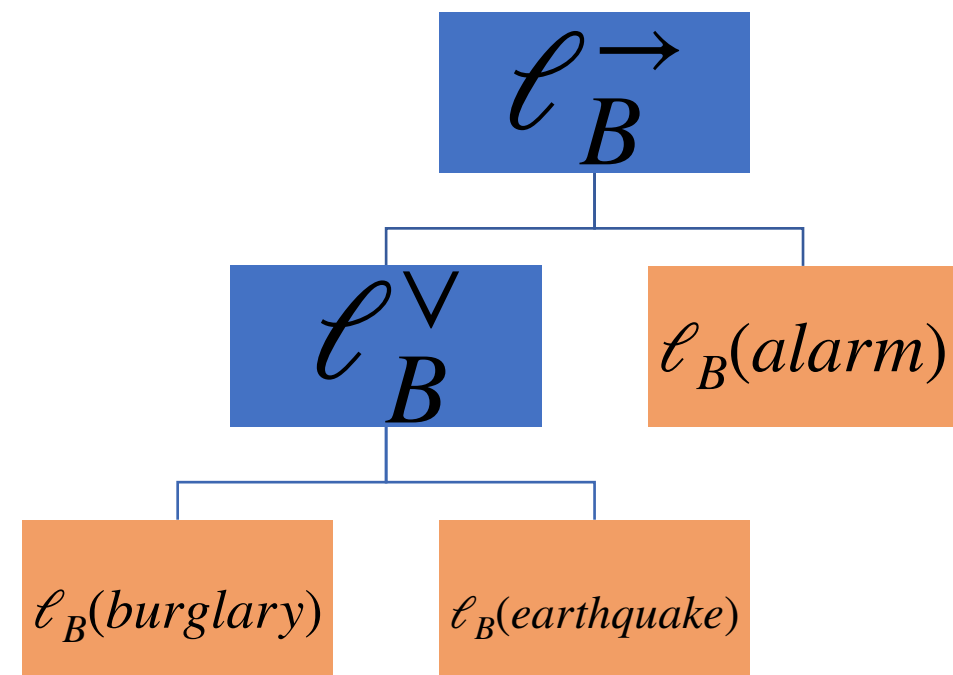
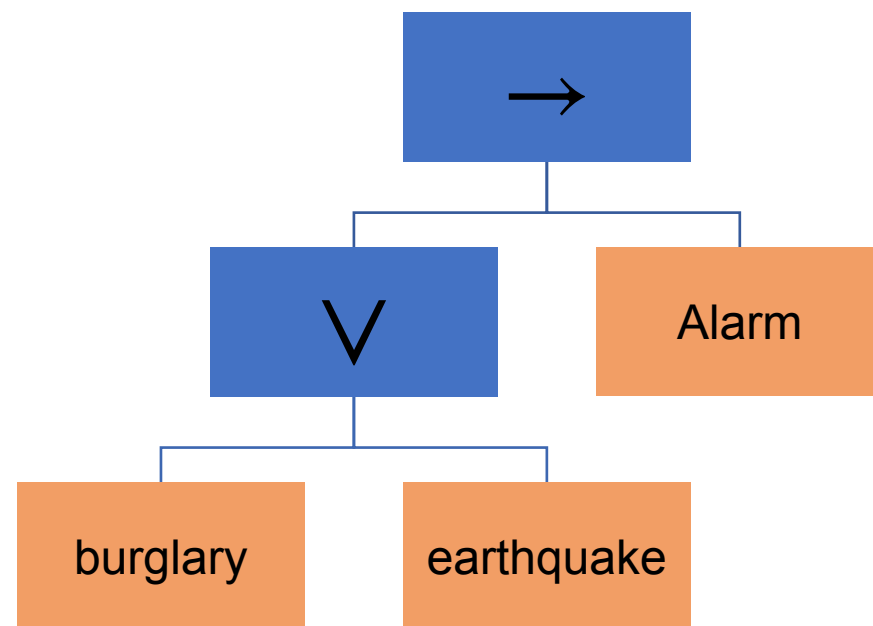
$$\ell_B(\text{earthquake} \wedge \text{burglary}) = \ell_B^\wedge(\ell_B(\text{earthquake}), \ell_B(\text{burglary}))$$

This recursive evaluation of formulas is said to be **extensional approach**.



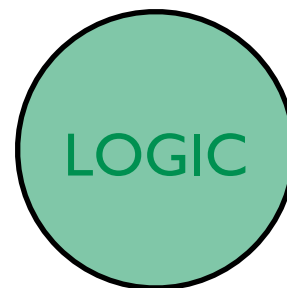
Semantics in Boolean Logic

- Consider: $(\text{burglary} \vee \text{earthquake}) \rightarrow \text{alarm}$

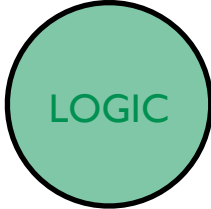


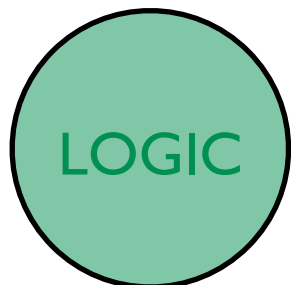
Dimension 3: Semantics

Fuzzy



Semantics in Fuzzy Logic

- Still a pure **logic** semantics: 
- There are many fuzzy logics
- Here we are interested in a subclass, in particular *t-norm fuzzy logic*



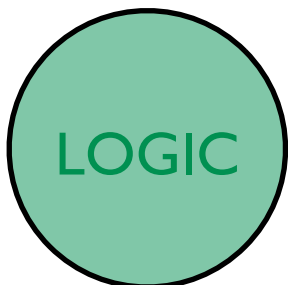
Semantics in Fuzzy Logic

- Defining a **semantics** for a propositional fuzzy language L is again about **assigning a membership degree** to all the sentences of the logic
- Fuzzy **truth/membership degrees**:

$$\ell_F: L \rightarrow [0,1]$$

Three steps:

1. Labels for propositions
2. Labels for operators
3. Labels for formulas



Semantics in Fuzzy Logic

1. Providing the **labels** for propositions

$L = \{burglary, earthquake, hears_alarm(john)\}$

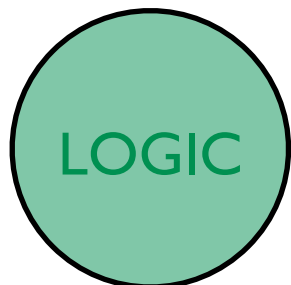
$$\ell_F(burglary) = 0.9$$

$$\ell_F(earthquake) = 0.1$$

$$\ell_F(hears_alarm(john)) = 0.8$$

Note: $\ell_F(earthquake) = 0.1$ -> very mild earthquake,

($\neq$ probability of earthquake = 0.1)



fuzzy is a measure of **intensity/vagueness** not of uncertainty

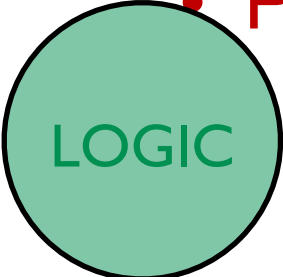
Semantics in Fuzzy Logic

2. Providing the labels for operators: t-norm theory

- A **t-norm** is a binary function that extends the **conjunction** to the continuous case

$$t : [0,1] \times [0,1] \rightarrow [0,1]$$

- There are 3 fundamental t-norms:
 - **Lukasiewicz t-norm:** $t_L(x, y) = \max(0, x + y - 1)$
 - **Goedel t-norm:** $t_G(x, y) = \min(x, y)$
 - **Product t-norm:** $t_P(x, y) = x \cdot y$



LOGIC

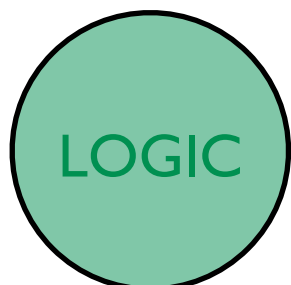
They are the continuous version of truth tables!!

Semantics in Fuzzy Logic

- All the other operators can be derived from the t-norm (and its residuum)

	Product	Łukasiewicz	Gödel
$x \wedge y$	$x \cdot y$	$\max(0, x + y - 1)$	$\min(x, y)$
$x \vee y$	$x + y - x \cdot y$	$\min(1, x + y)$	$\max(x, y)$
$\neg x$	$1 - x$	$1 - x$	$1 - x$
$x \Rightarrow y \ (x > y)$	y/x	$\min(1, 1 - x + y)$	y

They are the continuous version of truth tables!!



Semantics in Fuzzy Logic

3. The labels of **formulas** is defined **recursively** on the semantics of its components

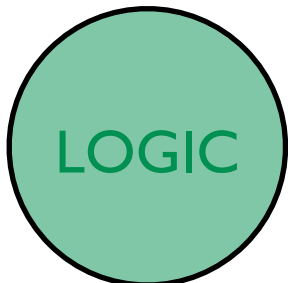
$$\ell_F(burglary \rightarrow alarm) = \ell_F^{\rightarrow}(\ell_F(burglary), \ell_F(alarm))$$

This recursive evaluation of formulas is said to be **extensional approach**.

e.g.

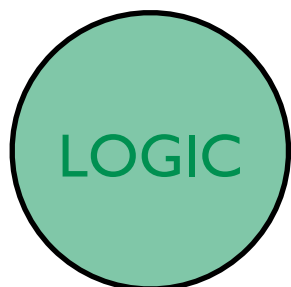
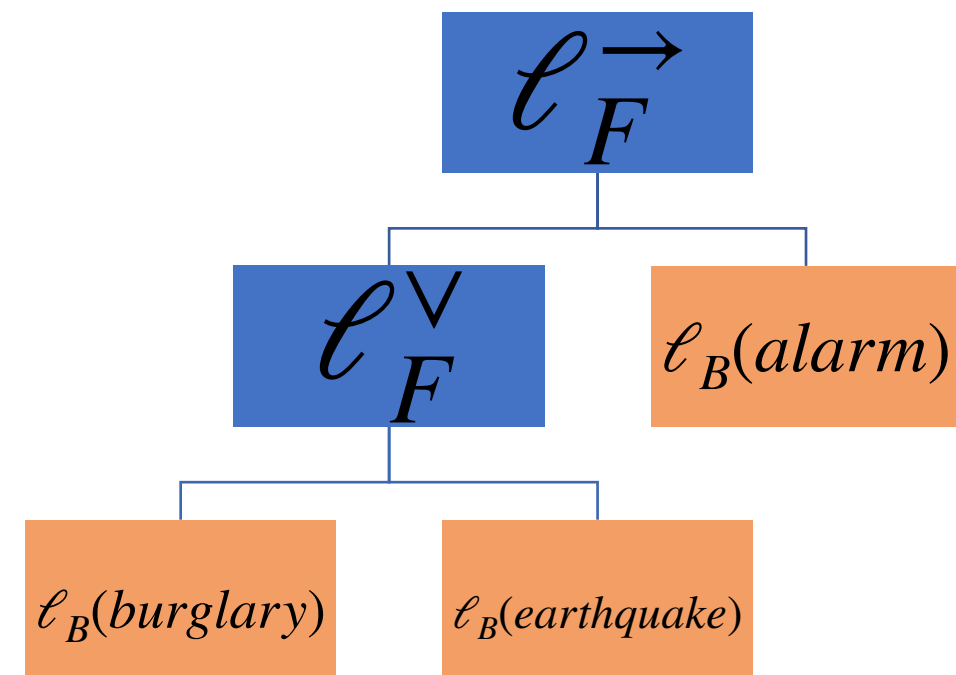
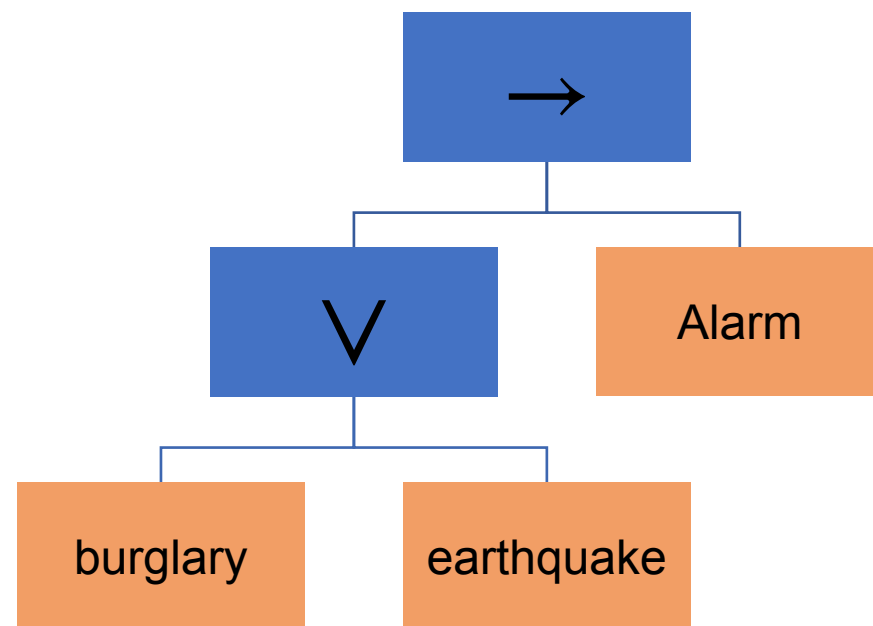
$$\ell_F(burglary) = 0.9, \ell_F(alarm) = 0.3,$$

$$\ell_F^{\rightarrow} = \min(1, 1 - x + y) = \min(1, 1 - 0.9 + 0.3) = 0.4$$



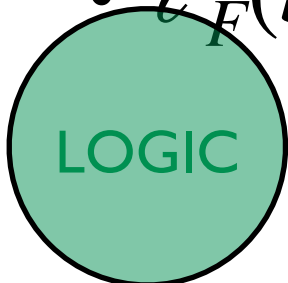
Semantics in Fuzzy Logic

- Consider: $(\text{burglary} \vee \text{earthquake}) \rightarrow \text{alarm}$



Fuzzy Logic Semantics

- Most common t-norms are:
 - **Continuous**
 - **Differentiable** -> This turns to be one of the reason of their adoption in NeSY
- Convex fragments of the logic can be defined (Giannini et al, 2019)
- But, $\ell_F(\text{human}(\text{Socrates})) = 0.5$?????
- $\ell_F(\text{bat}(\text{Socrates})) = 0.5$



Fuzzy vs Boolean

- Fuzzy and Boolean have different properties
- When fuzzy is used as a “relaxation” (**fuzzification**) of Boolean **undesired effects** can happen.
- Suppose: $A \vee B \vee C \vee D \vee E = 1$
- Satisfying assignments (Lukasiewicz)
 - $A = B = C = D = E = 1$ (all true)
 - $A = 1, B = C = D = E = 0$ (at least one true)
 - $A = B = C = D = E = 0.2$

Fuzzy Logic Semantics

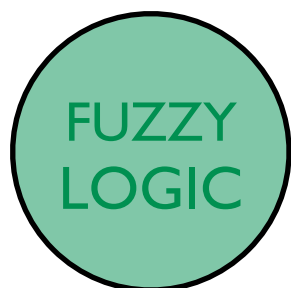
- All the t-norms fuzzy logics coincides with Boolean logic on the endpoints of the interval.

- E.g. $t_L(0,1) = \max(0, 0 + 1 - 1) = 0$

$$t_L(1,1) = \max(0, 1 + 1 - 1) = 1$$

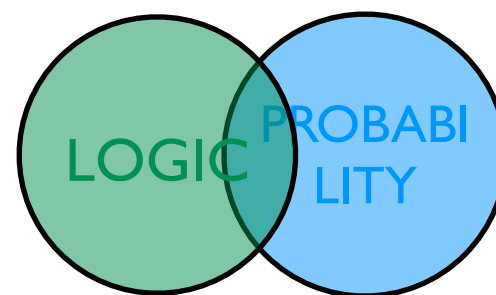
$$t_P(0,1) = 0 \cdot 1 = 0$$

$$t_P(1,1) = 1 \cdot 1 = 1$$



Convex fragment: Giannini et al, 2018
(threshold)-Godel vs Boolean correspondence: Daniele et al, 2025

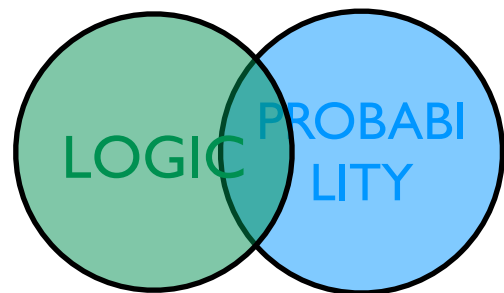
Dimension 3: Semantics



Probabilistic Logic Semantics

Given a propositional language L , the basic idea is to introduce a **probability function** p :

$$p : L \rightarrow [0,1]$$



Probabilistic Logic Semantics

Two steps:

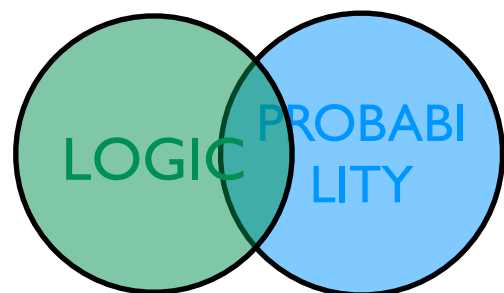
- Define a **probability distribution over interpretations / worlds** (i.e. boolean semantics)

$$p(\ell_B(x_1), \dots, \ell_B(x_n))$$

(E.g. $p(\ell_B(burglary) = True, \ell_B(earthquake) = False, \dots)$)

- Define the **probability of sentence Q of L**:

$$p(Q) = \sum_{\ell_B(x_1), \dots, \ell_B(x_n) \models Q} p(\ell_B(x_1), \dots, \ell_B(x_n))$$



Probabilistic Logic Semantics

Problog

0.1 :: burglary. (B)

0.05 :: earthquake. (E)

0.6 :: hears_alarm(john). (H)

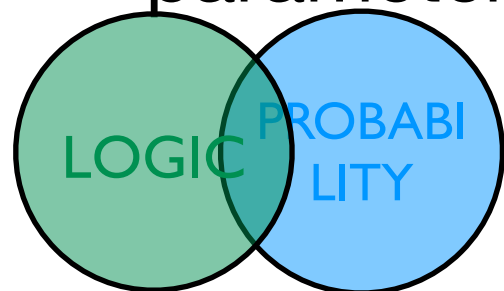
alarm :- earthquake.

alarm :- burglary.

calls(john) :- alarm, hears_alarm(john)

$$p(\ell_B(x_1), \dots, \ell_B(x_n)) = \prod_{i:\ell_B(x_i)=True} p(x_i) \prod_{i:\ell_B(x_i)=False} (1 - p(x_i))$$

parameters = the **labels for propositions** (i.e. probabilistic facts)



Probabilistic Logic Semantics

Problog

e.g. in Problog:

B	E	H	p(B,E,H)
F	F	F	0,342
F	F	T	0,513
F	T	F	0,018
F	T	T	0,027
T	F	F	0,038
T	F	T	0,057
T	T	F	0,002
T	T	T	0,003

0.1 :: burglary. (B)

0.05 :: earthquake. (E)

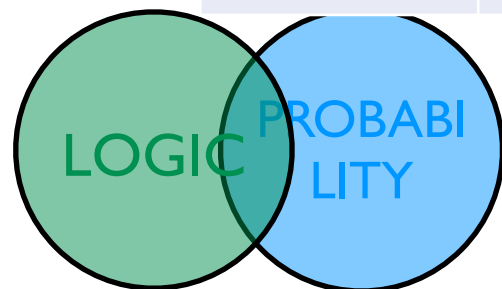
0.6 :: hears_alarm(john). (H)

alarm :- earthquake.

alarm :- burglary.

calls(john) :- alarm, hears_alarm(john)

$$0.1 \times 0.05 \times (1 - 0.6)$$



Probabilistic Logic Semantics

Markov Logic

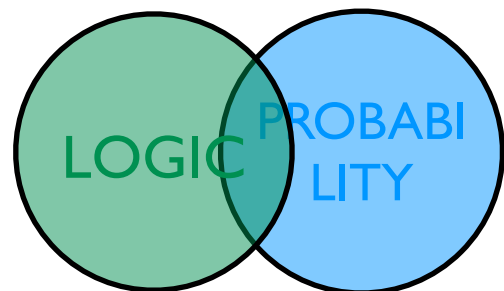
1.5 : calls(Mary) <- hears_alarm(Mary), alarm

2.0 : alarm <- earthquake

0.5 : alarm <- burglary

Weight formula 1 if α is True otherwise 0

$$p(\ell_B(x_1), \dots, \ell_B(x_n)) = \frac{1}{Z} \exp \left(\sum_{\alpha} w_{\alpha} \ell_B(\alpha) \right)$$



Probabilistic Logic Semantics

Markov Logic

1.5 : `calls(Mary) <- hears_alarm(Mary), alarm`

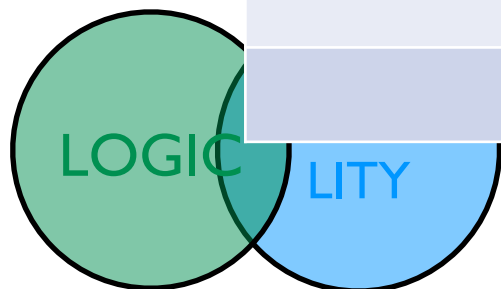
2.0 : `alarm <- earthquake`

0.5 : `alarm <- burglary`

B	E	A	H	C	p
T	F	T	T	T	0,05
T	F	T	T	F	0,01
...	...	...	...	...	...

$\propto \exp(\textcolor{brown}{1.5} + \textcolor{blue}{2.0} + \textcolor{teal}{0.5})$

$\propto \exp(\textcolor{brown}{0} + \textcolor{blue}{2.0} + \textcolor{teal}{0.5})$

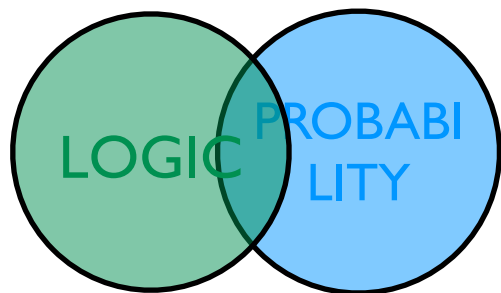


Probabilistic Logic Semantics

Given any **sentence** Q of the propositional language L , with variables $x_1, \dots, x_n$:

$$\ell_P(Q) = \sum_{\ell_B(x_1), \dots, \ell_B(x_n) \models Q} p(\ell_B(x_1), \dots, \ell_B(x_n))$$

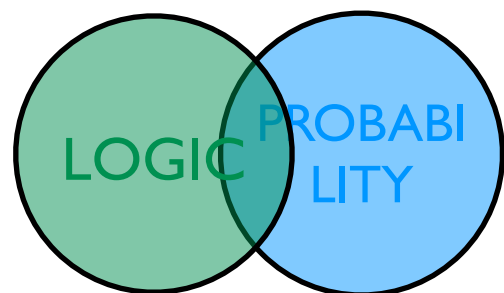
WMC - Weighted Model Counting
(for both ProbLog and Markov Logic)



Probabilistic Logic Semantics

Probabilistic Semantics is different from a pure logic semantics

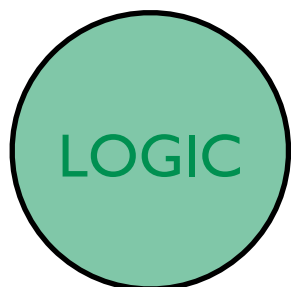
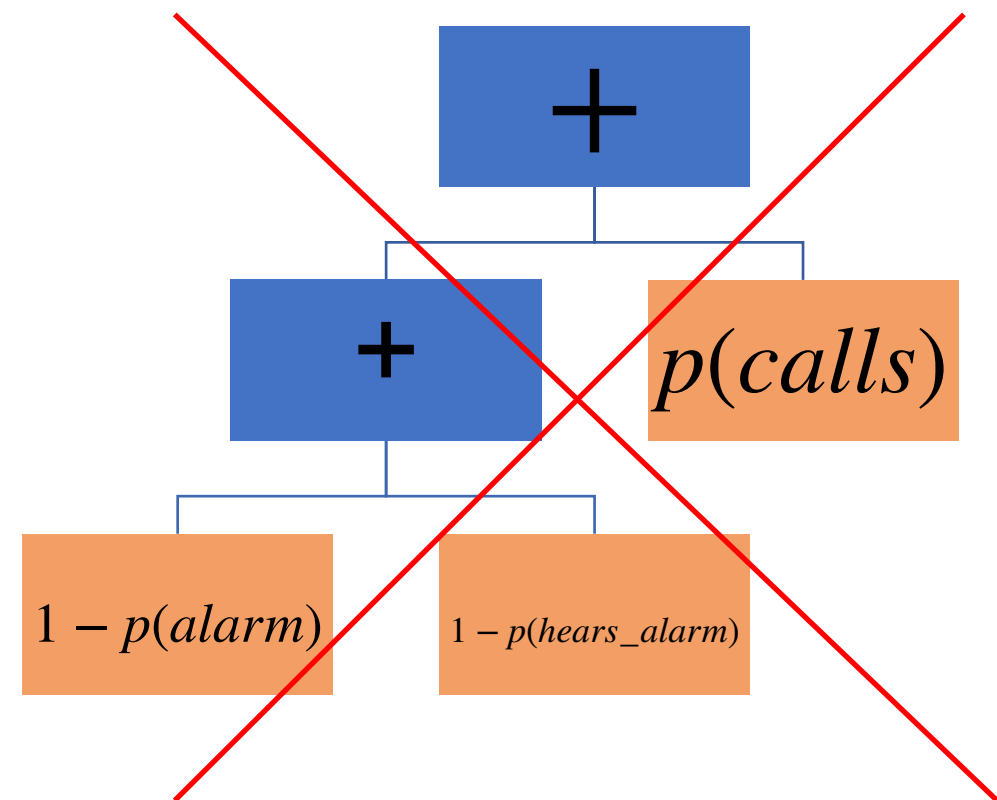
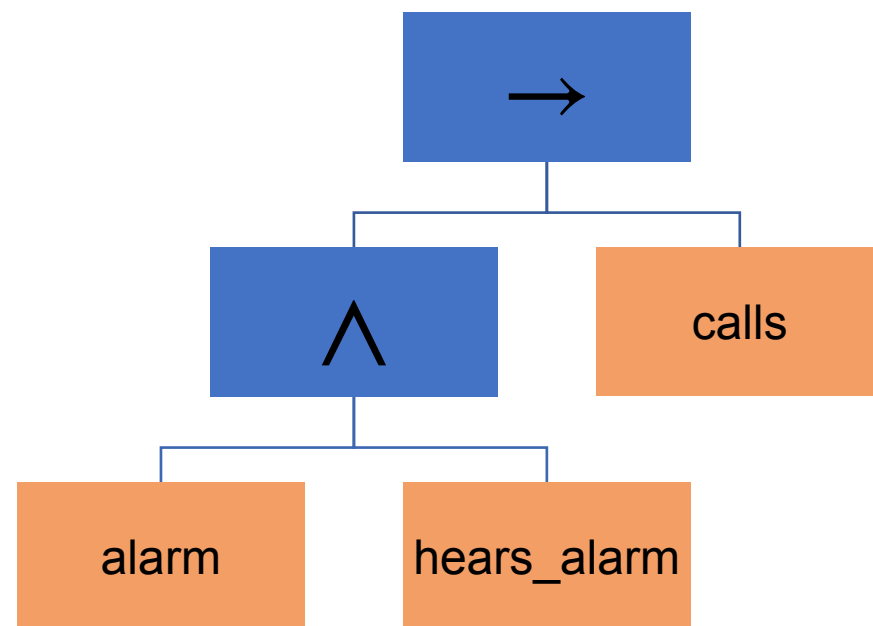
1. It is built on top of a logical semantics; $p(\ell_B(x_1), \dots, \ell_B(x_n))$.
2. Probability is **NOT truth-functional**, the probability of a formula
 - A. **cannot be defined recursively** by the probabilities of its arguments
 - B. requires **WMC**



Probabilistic Logic Semantics

$$(alarm \wedge hears_alarm) \rightarrow calls$$

- Consider:

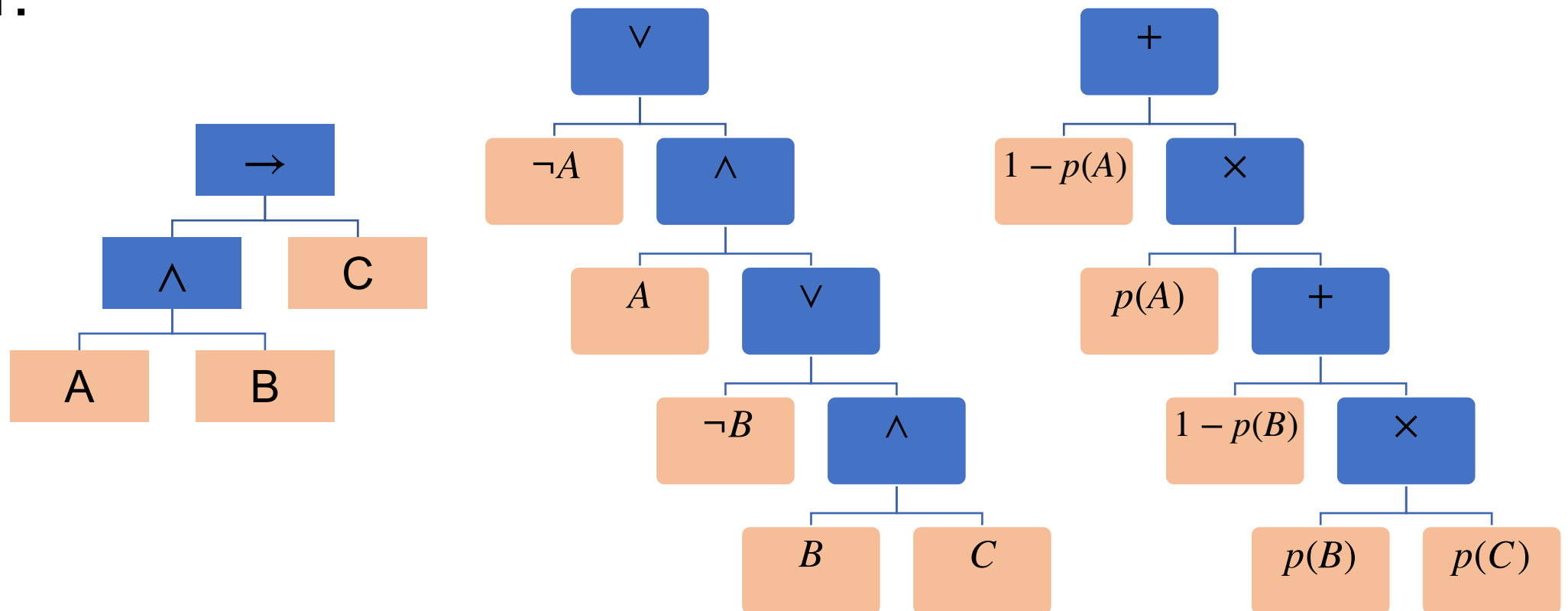


Probabilistic Logic Semantics

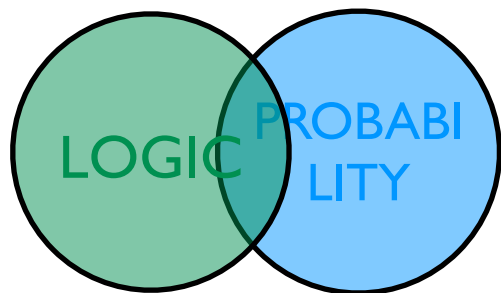
$$\ell_P(Q) = \sum_{\ell_B(x_1), \dots, \ell_B(x_n) \models Q} p(\ell_B(x_1), \dots, \ell_B(x_n))$$

- Consider:

$(A \wedge B) \rightarrow C$



Knowledge Compilation

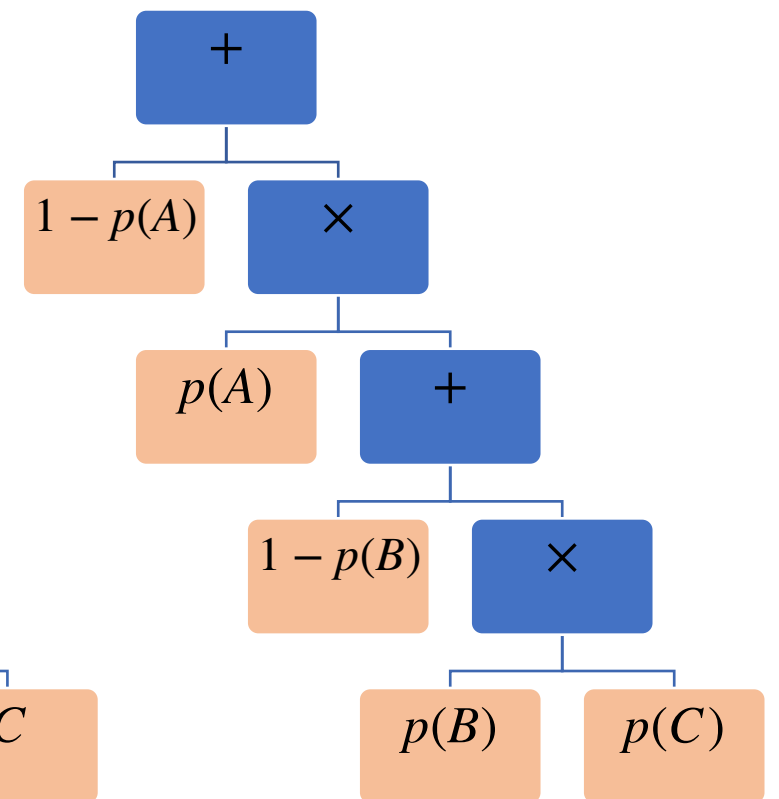
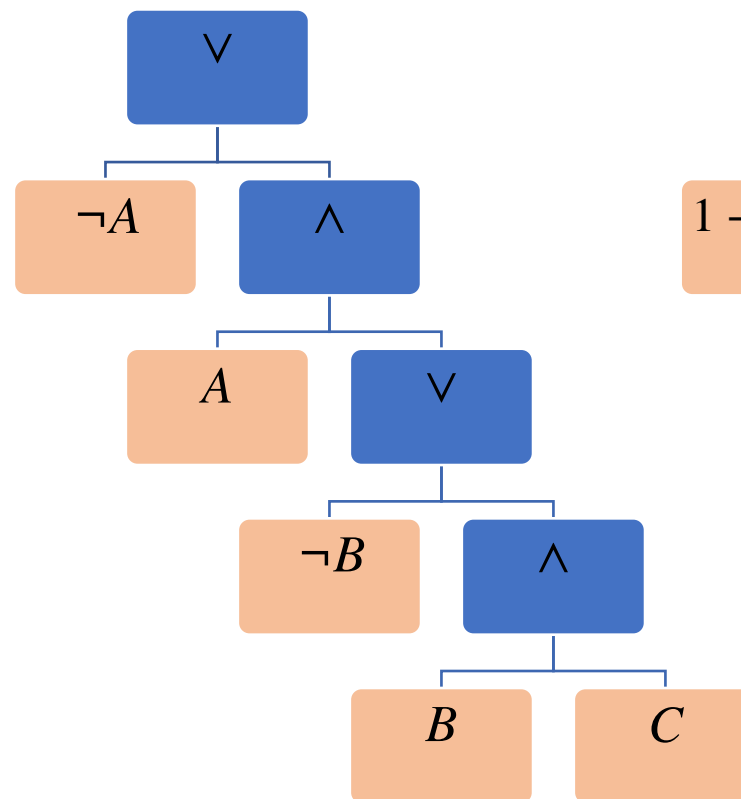
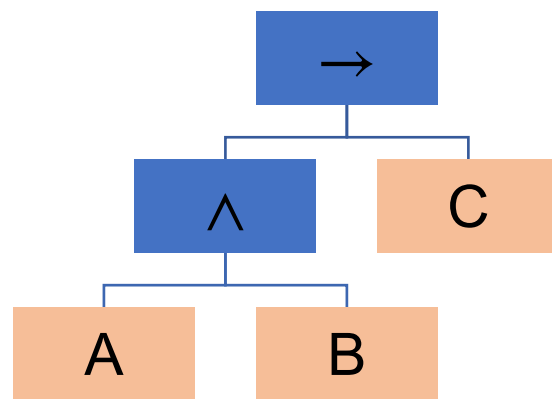


The probabilistic structure is now explicit in the compiled formula.

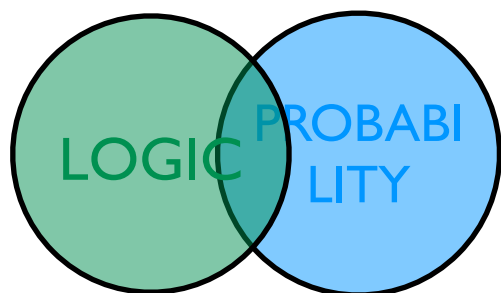
Probabilistic Logic Semantics

- Consider:

$$(A \wedge B) \rightarrow C$$



The circuit is differentiable!



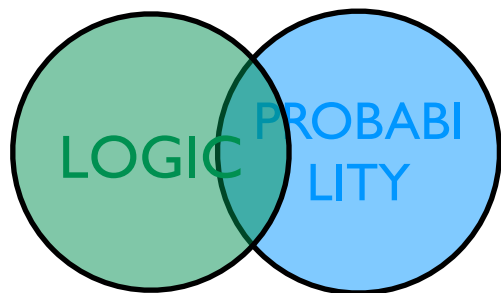
Probabilistic Logic Semantics

- WMC:

$$p(Q) = \sum_{\ell_B(x_1), \dots, \ell_B(x_n) \models Q} p(\ell_B(x_1), \dots, \ell_B(x_n))$$

- Another important inference task in MPE inference (connected to maxSAT)

$$\ell_B^\star(x_1), \dots, \ell_B^\star(x_n) = \max_{\ell_B(x_1), \dots, \ell_B(x_n) \models Q} p(\ell_B(x_1), \dots, \ell_B(x_n))$$



Boolean vs Fuzzy vs Probability

- Boolean and Fuzzy logic are two **alternative** logical semantics
- Probability is a semantics that is built on top of a logical one (i.e. “which is the **probability** of a given **truth assignments** / world?”)
- Can we have a probabilistic fuzzy logic as well?

Probabilistic Soft Logic (PSL)

Bach, Stephen H., et al. *JMLR* 2017

- Let's start by an example of a Markov Logic Network:

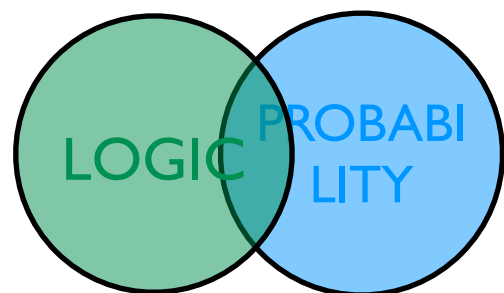
$$p(\ell_B(x_1), \dots, \ell_B(x_n)) = \frac{1}{Z} \exp \left(\sum_{\alpha} w_{\alpha} \ell_B(\alpha) \right)$$

- In PSL, we relax the **Boolean semantics** ℓ_B to a **fuzzy semantics** ℓ_F

$$p(\ell_F(x_1), \dots, \ell_F(x_n)) = \frac{1}{Z} \exp \left(\sum_{\alpha} \boxed{w_{\alpha}} \boxed{\ell_F(\alpha)} \right)$$

Weight formula

Each formula contributes with a value in $[0,1]$



Probabilistic Soft Logic (PSL)

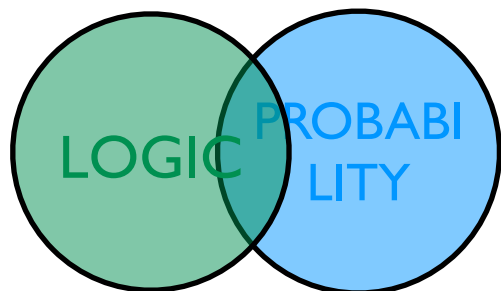
Let's consider:

$$\alpha(X) : \textit{stress}(X) \rightarrow \textit{smokes}(X)$$

$$\alpha(X) : \neg \textit{stress}(X) \vee \textit{smokes}(X)$$

Using Lukasiewicz t-norm:

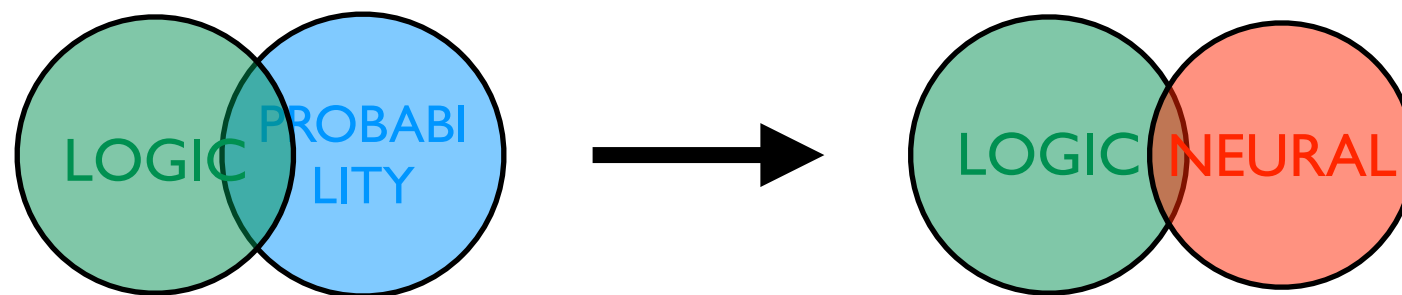
$$\ell_F(\alpha(X)) = \min(1, 1 - \ell_F(\textit{stress}(X)) + \ell_F(\textit{smokes}(X)))$$



Probabilistic vs Fuzzy

- Fuzzy is an alternative logical semantics and it can still coupled with the probabilistic ones
- Fuzzy logic is **sometimes** used as an approximation of MPE in probabilistic logic
- Fuzzy logic is **sometimes** used to solve **satisfiability** faster
 - **However**, it does not guarantee solutions coherent with the Boolean logic theory.
 - (Remember $A = B = C = D = E = 0.2$)

Dimension 3: Semantics

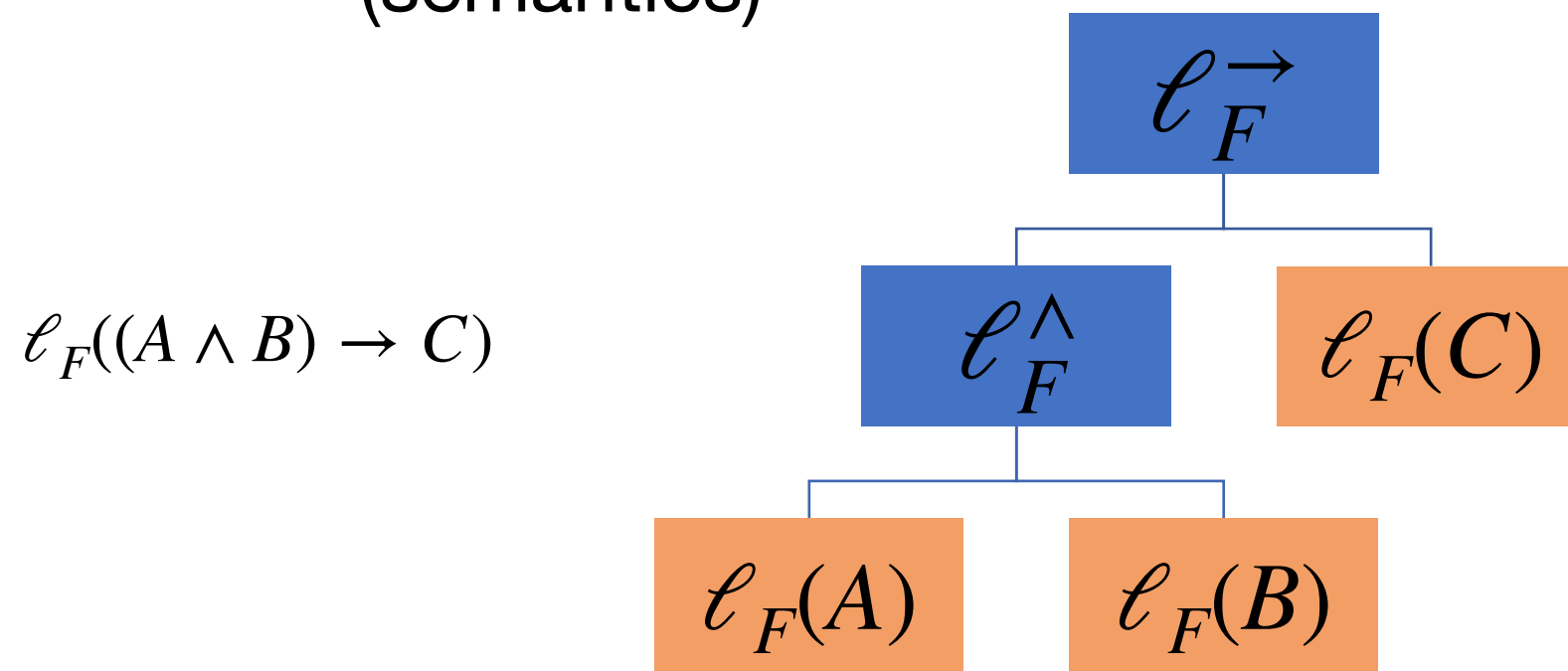


Neural Symbolic

How to carry over concepts from the semantics of StarAI to neural symbolic?

$$\ell(Q)$$

Labelling functions (semantics) = Parametric circuit



The query Q determine the **structure** (potentially after knowledge compilation)

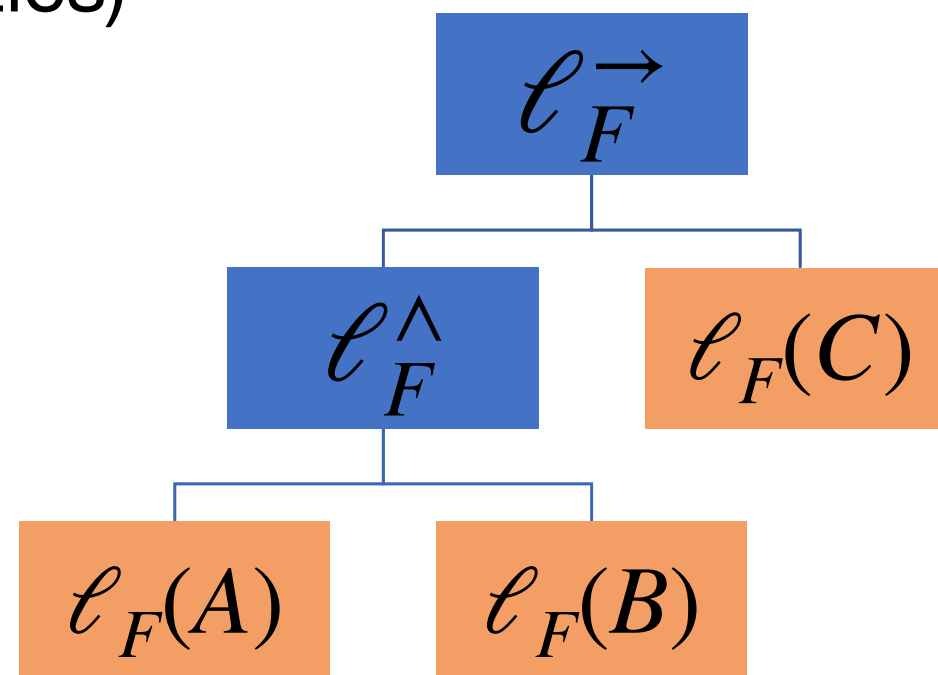
Neural Symbolic

How to carry over concepts from the semantics of StarAI to neural symbolic?

$$\ell(Q)$$

Labelling functions (semantics) = Parametric circuit

$$\ell_F((A \wedge B) \rightarrow C)$$



The leaves represent the
scalar parameters

Neural Symbolic

How to carry over concepts from the semantics of StarAI to neural symbolic?

- Atomic labels are just **scalar tables of parameters**

0.1 :: burglary. (B)
0.05 :: earthquake. (E)
0.6 :: hears_alarm(john). (H)
alarm :- earthquake.
alarm :- burglary.

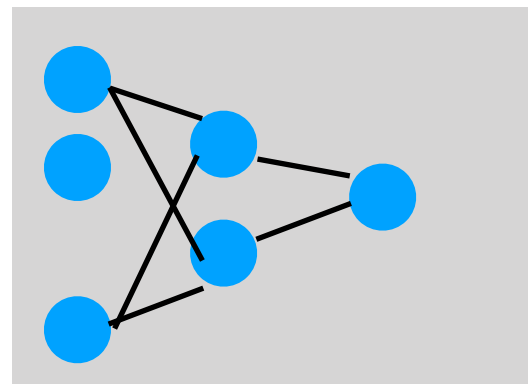
L	p
Burglary	0,1
Earthquake	0,05
...	

Neural Symbolic

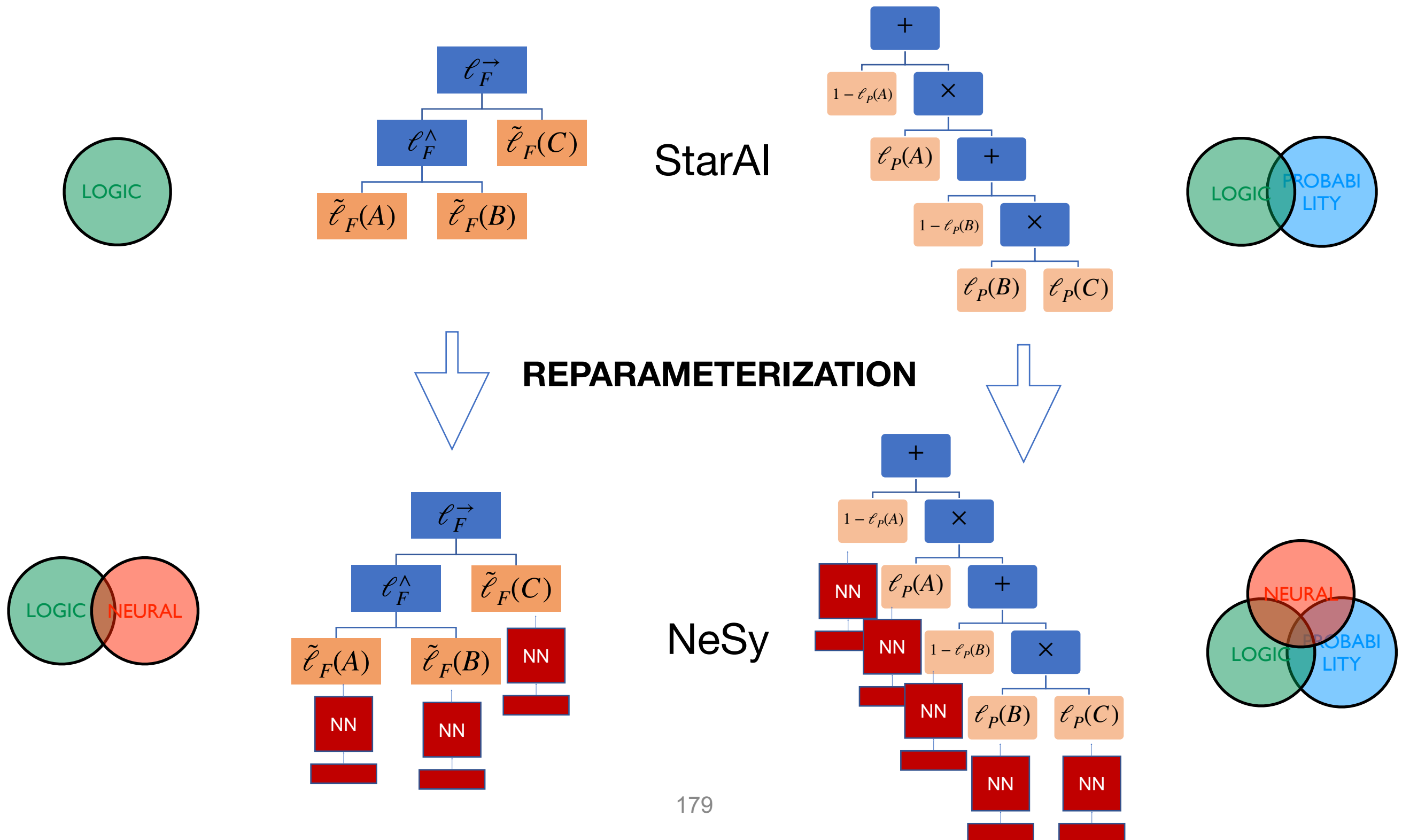
How to carry over concepts from the semantics of StarAI to neural symbolic?

- What if atomic labels are just **neural networks**?

? :: burglary()
? :: earthquake. ()
? :: hears_alarm(john).
alarm :- earthquake.
alarm :- burglary.

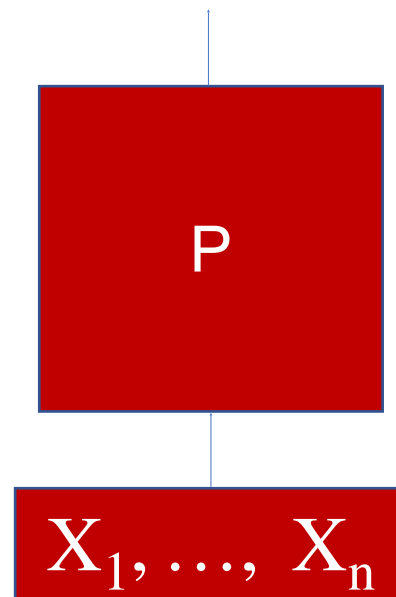
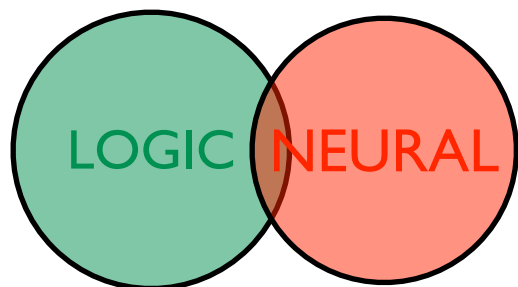


StarAI to Neural Symbolic



Fuzzy Reparameterization

- $\ell_F(\alpha(X)) = \min(1, 1 - \ell_F(\text{stress}(X)) + \ell_F(\text{smokes}(X)))$
- The fuzzy labels of the atoms, i.e. $\ell_F(\text{stress}(X))$, $\ell_F(\text{smokes}(X))$ are not scalar
- Fuzzy labels for atoms are parameterized by **neural networks**.
- $\ell_F\left(\text{P}(X_1, \dots, X_n)\right)$



Semantic Based Regularization
(Diligenti et al, AI 2017)

Logic Tensor Network (Donadello et
al, IJCAI 2017)

Probabilistic Reparameterization

■ Probabilistic parameters

- ProbLog:

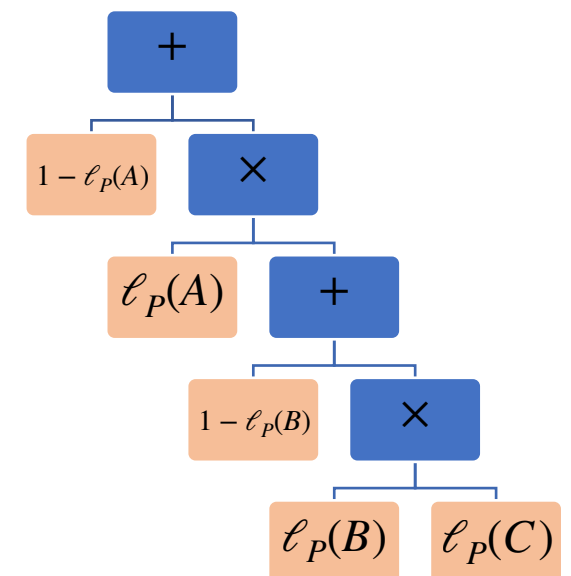
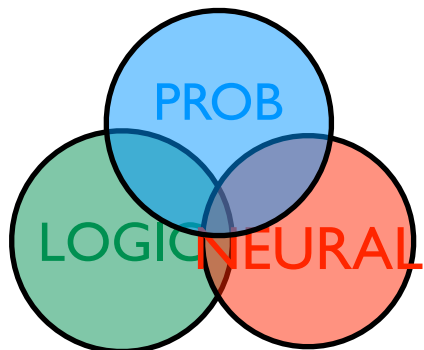
$$p(\ell_B(x_1), \dots, \ell_B(x_n)) = \prod_{i: \ell_B(x_i)=True} p(x_i) \prod_{i: \ell_B(x_i)=False} (1-p(x_i))$$

- Markov Logic:

$$p(\ell_B(x_1), \dots, \ell_B(x_n)) = \frac{1}{Z} \exp\left(\sum_{\alpha} w_{\alpha} \ell_B(\alpha)\right)$$

WMC

$$p(Q) = \sum_{\ell_B(x_1), \dots, \ell_B(x_n) \models Q} p(\ell_B(x_1), \dots, \ell_B(x_n))$$



Probabilistic Reparameterization

- **DeepProbLog** (Manhaeve et al, NeurIPS (2018)) ■ Neural parameters

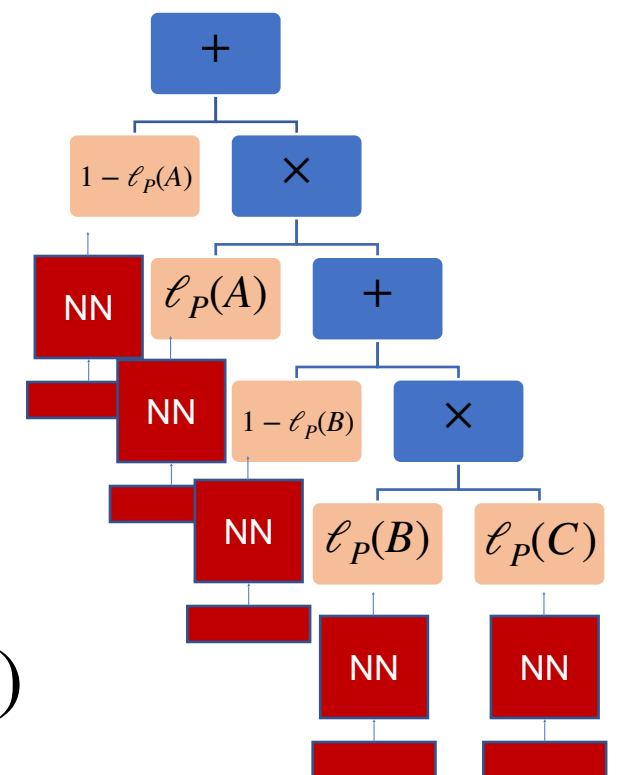
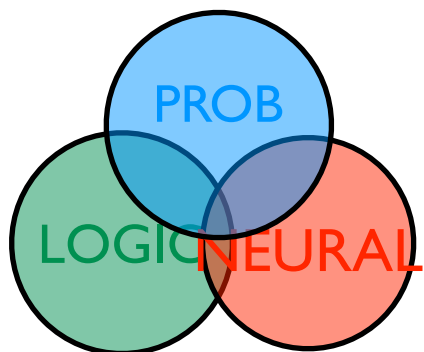
$$p(\ell_B(x_1), \dots, \ell_B(x_n)) = \prod_{i: \ell_B(x_i)=True} p(x_i) \prod_{i: \ell_B(x_i)=False} (1-p(x_i))$$

- **Relational Neural Machines** (Marra et al, ECAI 2020)

$$p(\ell_B(x_1), \dots, \ell_B(x_n)) = \frac{1}{Z} \exp\left(\sum_{\alpha} w_{\alpha} \ell_B(\alpha)\right)$$

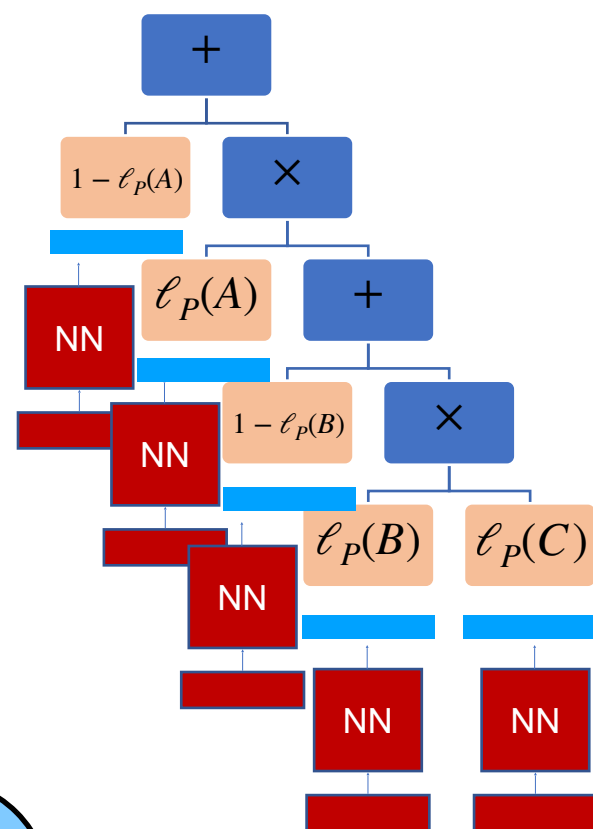
WMC

$$p(Q) = \sum_{\ell_B(x_1), \dots, \ell_B(x_n) \models Q} p(\ell_B(x_1), \dots, \ell_B(x_n))$$



Probabilistic Reparameterization

- **DeepProbLog** (Manhaeve et al, NeurIPS (2018))



Interface

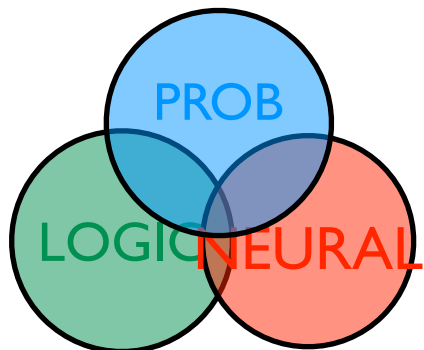
Probabilistic fact

0.01 :: burglary.



Neural Predicate

nn(mnist_net, [X], Y, [0 ... 9]) :: digit(X,Y).



Dimension 3: Semantics

Key Messages

- StarAI and NeSy share the same underlying semantics
- Semantics can be described in terms of parametric circuits
- Differentiable semantics/circuits allow an easy integration
- NeSy models can be seen as neural reparameterization of StarAI models

Inference from SAT to WMC

From SAT to #SAT and WMC

- SAT : does there exist a model for a logical theory ?
- #SAT : how many models are there ? (model counting)
- WMC : what is the *weighted* model count ?

From SAT to #SAT and WMC

- For the previous theory, there were 6 models.
- In the Bayesian network, each possible world had a probability of $1/8$ and each literal of 0.5 (weight of 0.5). We can now define the weighted model counting problem (WMC).
 - Given is a logical theory T (usually in CNF),
 - for each literal l , there is a (non-negative) weight $w(l)$.

The weighted model count of the theory $wmc(T)$ is then :

- $wmc(T) = \sum_{M \models T} w(M)$
(where M is model for T , M is the set of all true literals)
- $w(M) = \prod_{l \in M} w(l)$
- There is a close correspondence between Bayesian network inference and weighted model counting.

Probabilistic Logic Semantics

Problog

e.g. in Problog:

B	E	H	p(B,E,H)
F	F	F	0,342
F	F	T	0,513
F	T	F	0,018
F	T	T	0,027
T	F	F	0,038
T	F	T	0,057
T	T	F	0,002
T	T	T	0,003

0.1 :: burglary. (B)

0.05 :: earthquake. (E)

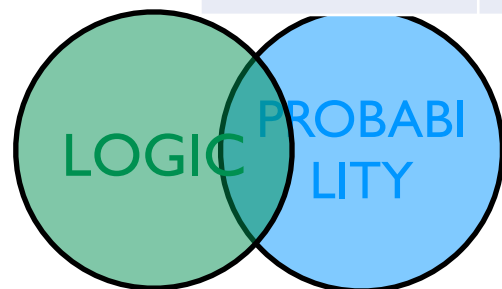
0.6 :: hears_alarm(john). (H)

alarm :- earthquake.

alarm :- burglary.

calls(john) :- alarm, hears_alarm(john)

$$0.1 \times 0.05 \times (1 - 0.6)$$



$$P(Q) = \sum_{F \cup R \models Q} \prod_{f \in F} p(f) \prod_{f \notin F} 1 - p(f)$$

Weighted Model Counting

propositional formula in conjunctive normal form (CNF)
 given by ProbLog program & query

$$WMC(\phi) = \sum_{I_V \models \phi} \prod_{l \in I_V} w(l)$$

interpretations (truth
 value assignments) of
 propositional variables
 possible worlds

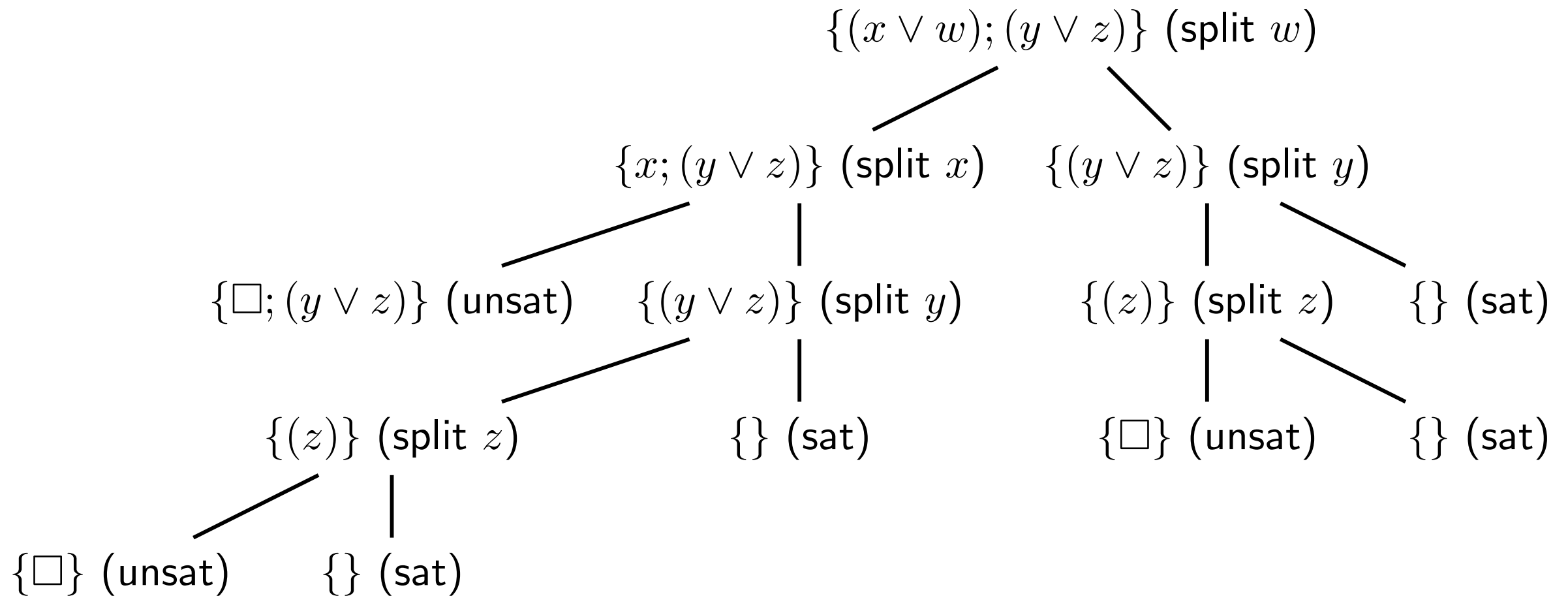
weight
 of literal
 for $p::f$,
 $w(f) = p$
 $w(\text{not } f) = 1 - p$

The DPLL Algorithm for SAT

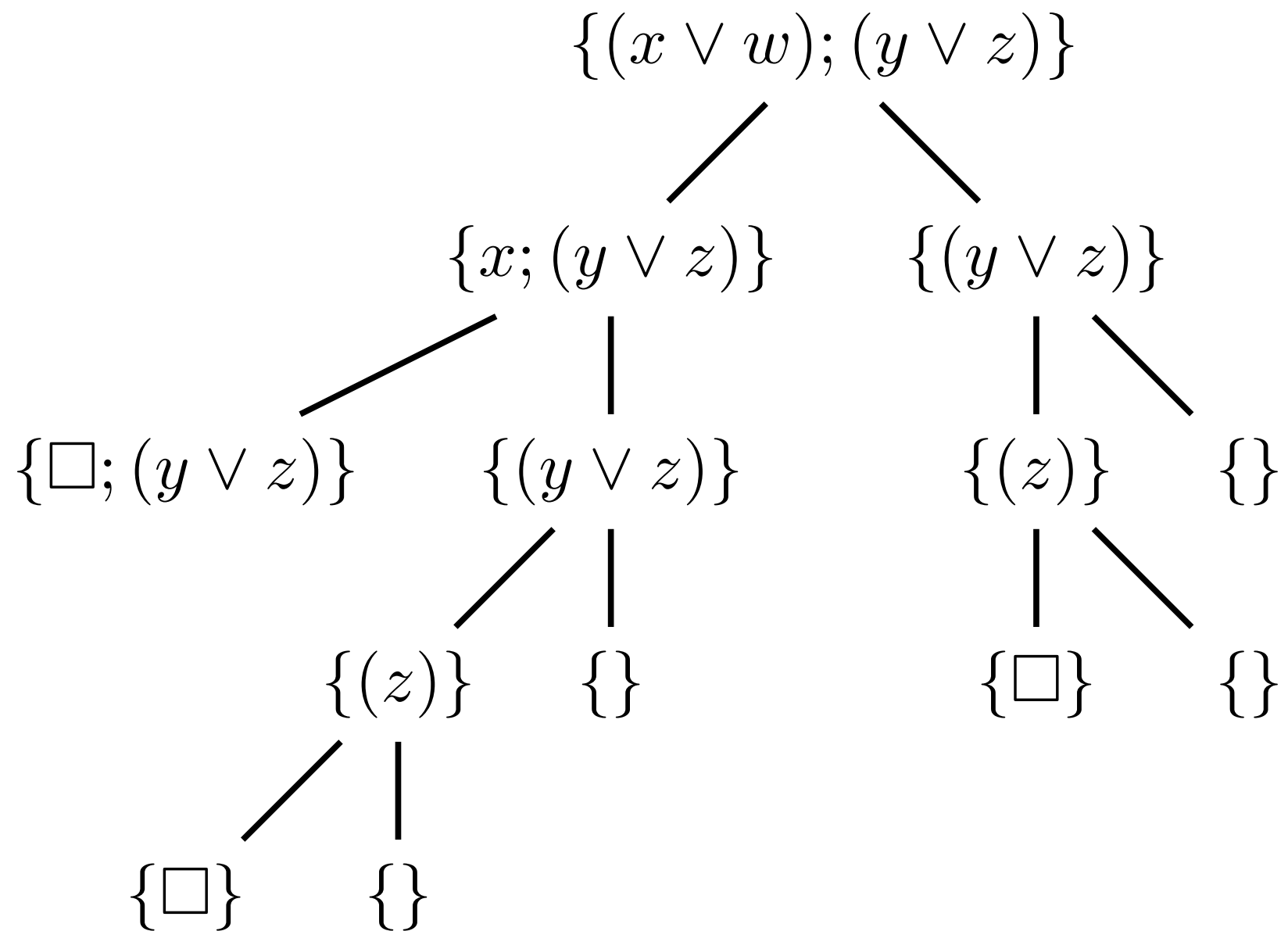
```
procedure DPLL(Vars : variables, S : set of clauses):  
  if S is empty  
    return 1  
  else if S contains an empty clause  
    return 0  
  else select  $v \in Vars$   
     $S_t := S$  where  $v = 1$  (making the variable true)  
     $S_f := S$  where  $v = 0$  (making the variable false)  
    return  $DPLL(Vars - \{v\}, S_t) + DPLL(Vars - \{v\}, S_f)$ 
```

- In a CNF theory $(A \vee \neg B) \wedge (C \vee D)$, the clauses are the disjunctions, that is, $(A \vee \neg B)$ and $(C \vee D)$
- A unit clause contains exactly one literal. E.g., A and $\neg A$ are both unit clauses. (It is possible to make DPLL more efficient by assigning unit clauses the appropriate value)
- An empty clause is a disjunction of 0 literals, at least one of which must be true. Therefore an empty clause is always unsatisfiable.

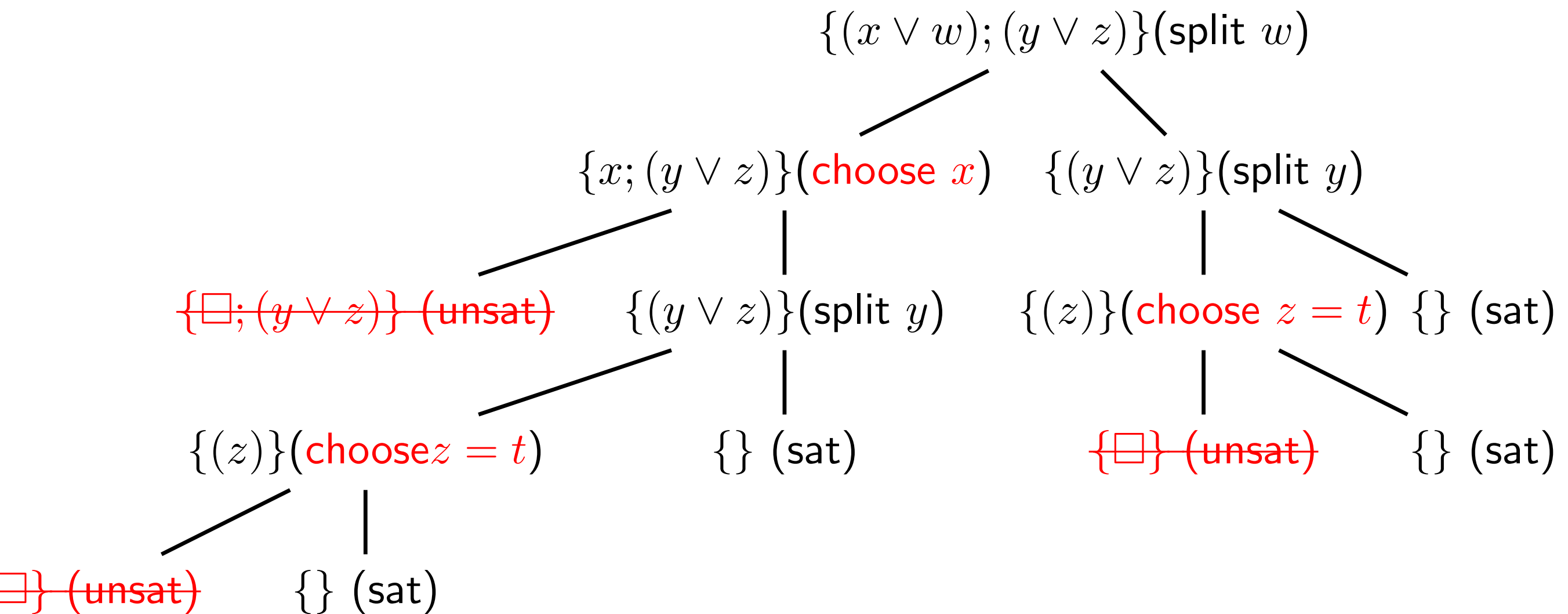
Example for SAT



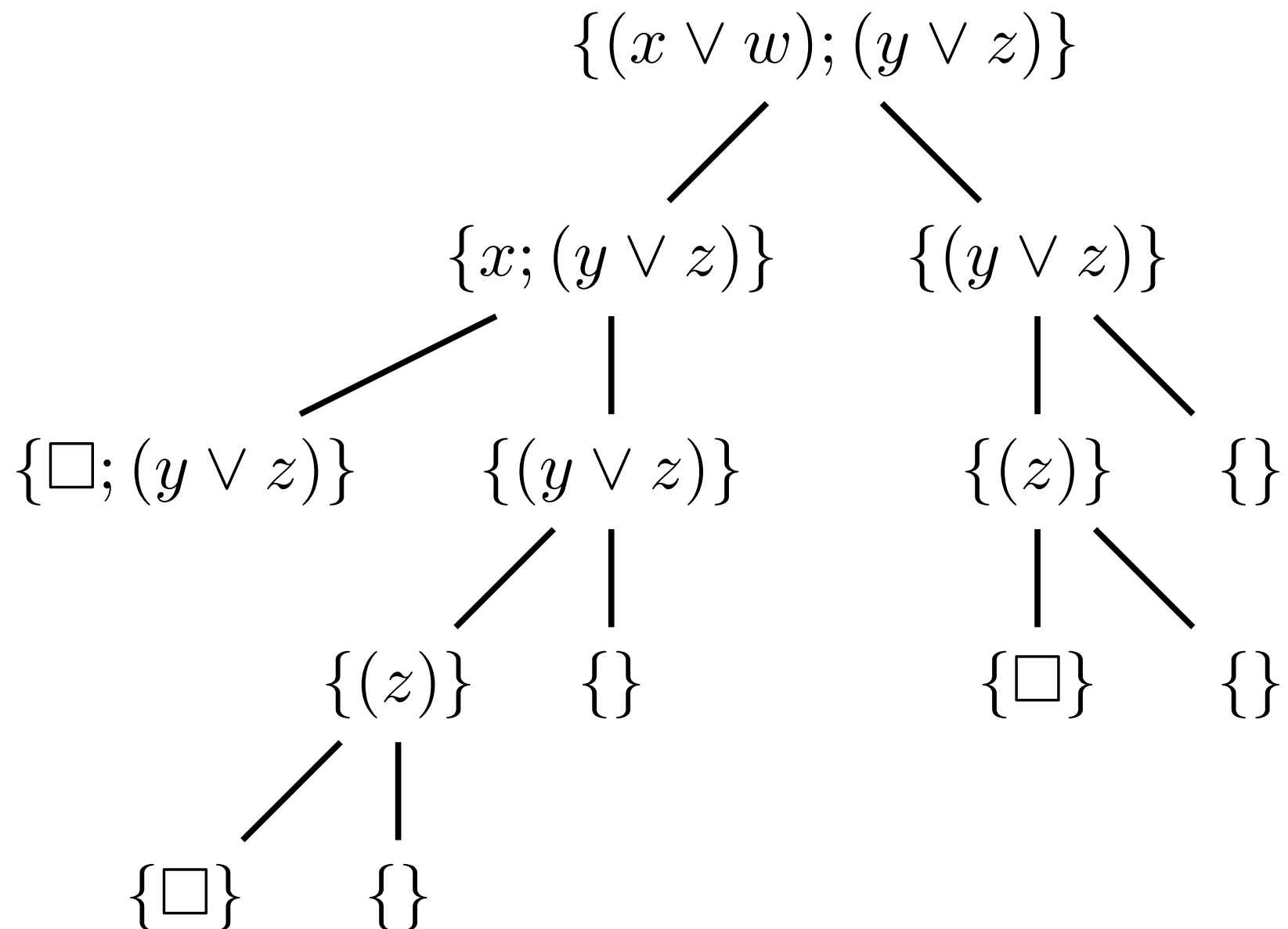
Shorthand Notation



Example - Unit Clause Propagation



Caching

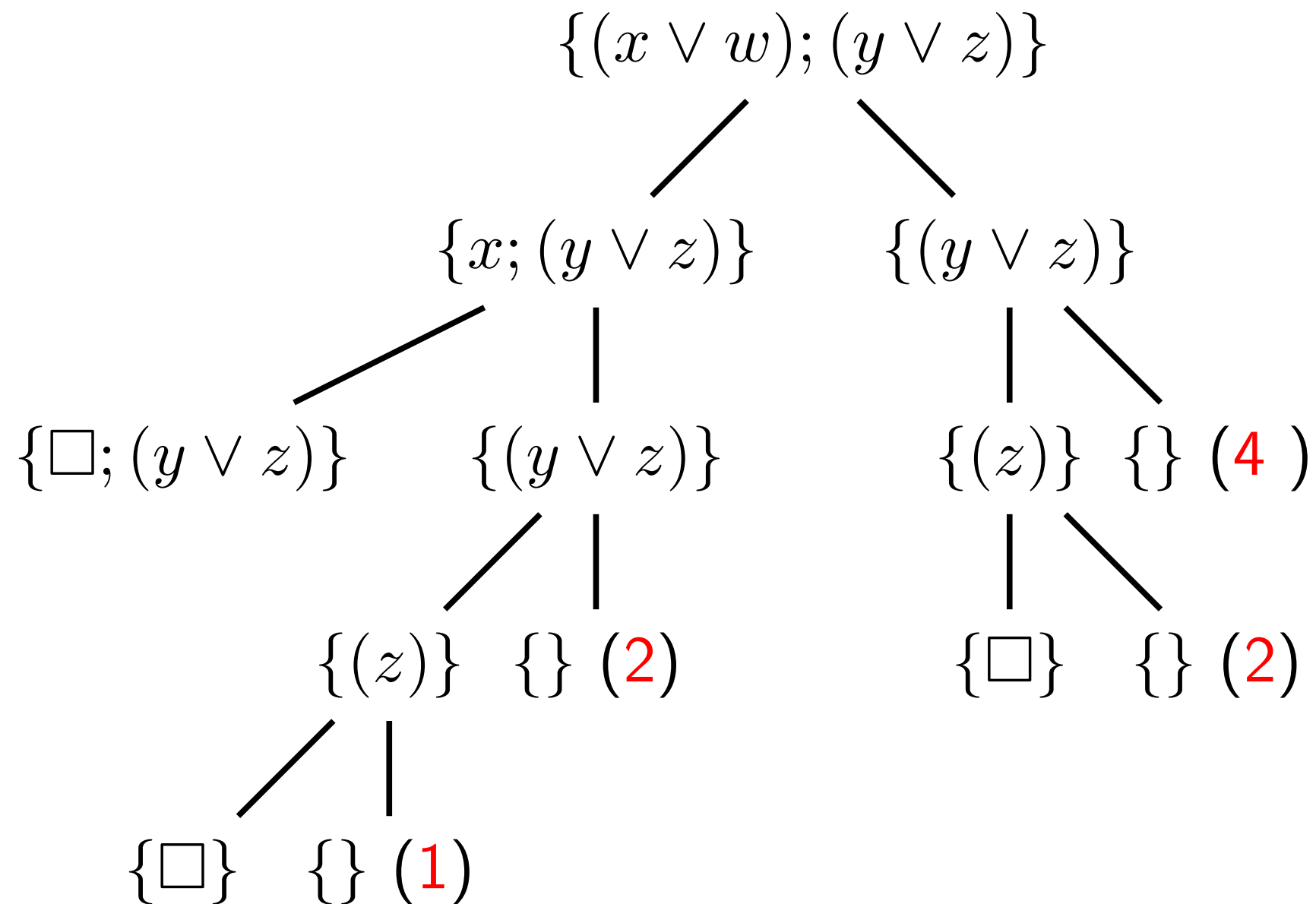


Solution : cache computed answers for sub trees and test whether sub formula was already encountered before.

DPLL Variant for #SAT

```
procedure #SAT( $Vars$  : variables,  $S$  : set of clauses):  
  if  $S$  is empty  
    return  $2^{|Vars|}$   
  else if  $S$  contains an empty clause  
    return 0  
  else select  $v \in Vars$   
     $S_t := S$  where  $v = 1$  (making the variable true)  
     $S_f := S$  where  $v = 0$  (making the variable false)  
    return #SAT( $Vars - \{v\}, S_t$ ) + #SAT( $Vars - \{v\}, S_f$ )
```


Example #SAT



DPLL Variant for WMC

```
procedure  $WMC(Vars : \text{variables}, S : \text{set of clauses})$ :  
  if  $S$  is empty  
    return  $\prod_{v \in Vars} w(v) + w(\neg v)$   
  else if  $S$  contains an empty clause  
    return 0  
  else select  $v \in Vars$   
     $S_t := S$  where  $v = 1$  (making the variable true)  
     $S_f := S$  where  $v = 0$  (making the variable false)  
    return  $w(v) WMC(Vars - \{v\}, S_t) + w(\neg v) WMC(Vars - \{v\}, S_f)$ 
```

What have we done ?

We have used sum products — semi-rings

A semiring is a structure $(A, \oplus, \otimes, e^\oplus, e^\otimes)$, where addition $\oplus$ and multiplication $\otimes$ are associative binary operations over the set A , $\oplus$ is commutative, $\otimes$ distributes over $\oplus$, $e^\oplus \in A$ is the neutral element of $\oplus$, $e^\otimes \in A$ that of $\otimes$, and for all $a \in A$, $e^\oplus \otimes a = a \otimes e^\oplus = e^\oplus$. In a commutative semiring, $\otimes$ is commutative as well.

Algebraic Model Counting

- commutative semiring $(A, \oplus, \otimes, e^\oplus, e^\otimes)$
- algebraic literals
 $L(F) = \{f_1, \dots, f_n\} \cup \{\neg f_1, \dots, \neg f_n\}$
- labeling function $\alpha: L(F) \rightarrow A$
- propositional logical theory T

$$AMC(T) = \bigoplus_{T \models w} \bigotimes_{l \in w} \alpha(l)$$

Useful Semirings

task	$\mathcal{A}$	$e^\oplus$	$e^\otimes$	$\oplus$	$\otimes$	$\alpha(v)$	$\alpha(\neg v)$	ref
SAT	$\{true, false\}$	<i>false</i>	<i>true</i>	$\vee$	$\wedge$	<i>true</i>	<i>true</i>	B, BT, G, GK, K, L, M
#SAT	$\mathbb{N}$	0	1	+	$\cdot$	1	1	B, G, GK, K, L
WMC	$\mathbb{R}_{\geq 0}$	0	1	+	$\cdot$	$\in \mathbb{R}_{\geq 0}$	$\in \mathbb{R}_{\geq 0}$	
PROB	$\mathbb{R}_{\geq 0}$	0	1	+	$\cdot$	$\in [0, 1]$	$1 - \alpha(v)$	B, BT, E, G, K
SENS	$\mathbb{R}[\mathcal{V}]$	0	1	+	$\cdot$	v or $\in [0, 1]$	$1 - \alpha(v)$	K
GRAD	$\mathbb{R}_{\geq 0} \times \mathbb{R}$	(0, 0)	(1, 0)	Eq. (4)	Eq. (5)	Eq. (2)	Eq. (3)	E, K
MPE	$\mathbb{R}_{\geq 0}$	0	1	max	$\cdot$	$\in [0, 1]$	$1 - \alpha(v)$	B, BT, G, K, L, M
S-PATH	$\mathbb{N}^\infty$	∞	0	min	+	$\in \mathbb{N}$	0	BT, GK, K
W-PATH	$\mathbb{N}^\infty$	0	∞	max	min	$\in \mathbb{N}$	∞	BT
FUZZY	$[0, 1]$	0	1	max	min	$\in [0, 1]$	1	GK, M
k WEIGHT	$\{0, \dots, k\}$	k	0	min	$+^k$	$\in \{0, \dots, k\}$	$\in \{0, \dots, k\}$	M
OBDD $_{<}$	OBDD $_{<}(\mathcal{V})$	OBDD $_{<}(0)$	OBDD $_{<}(1)$	$\vee$	$\wedge$	OBDD $_{<}(v)$	$\neg$ OBDD $_{<}(v)$	K
WHY	$\mathcal{P}(\mathcal{V})$	$\emptyset$	$\emptyset$	$\cup$	$\cup$	$\{v\}$	n/a	GK
$\mathcal{RA}^+$	$\mathbb{N}[\mathcal{V}]$	0	1	+	$\cdot$	v	n/a	GK

Table 1: Examples of commutative semirings and labeling functions. The **WHY** and $\mathcal{RA}^+$ provenance semirings apply to positive literals only. Reference key: B (Bacchus et al., 2009), BT (Baras and Theodorakopoulos, 2010), E (Eisner, 2002), G (Goodman, 1999), GK (Green et al., 2007), K (Kimmig et al., 2011), L (Larrosa et al., 2010), M (Meseguer et al., 2006); more examples can be found in these references.

From Kimmig, Vanden Broeck and De Raedt, 2016

more examples

semiring

label function

task	$\mathcal{A}$	$e^\oplus$	$e^\otimes$	$a \oplus b$	$a \otimes b$	$\alpha(f)$	$\alpha(\neg f)$
PROB	$\mathbb{R}_{\geq 0}$	0	1	$a + b$	$a \cdot b$	$\alpha(f_i) \in [0, 1]$	$1 - \alpha(f_i)$
MPE	$\mathbb{R}_{\geq 0}$	0	1	$\max(a, b)$	$a \cdot b$	$\alpha(f_i) \in [0, 1]$	$1 - \alpha(f_i)$
MPE State	$\mathbb{R}_{\geq 0} \times 2^{\{J \mid J \subseteq I \in \mathcal{I}(\mathcal{F})\}}$	$(0, \emptyset)$	$(1, \{\emptyset\})$	Eq. (8)	Eq. (7)	$\alpha(f_i) = (p_i, \{\{f_i\}\})$ with $p_i \in [0, 1]$	$(1 - p_i, \{\{\neg f_i\}\})$
SAT	$\{true, false\}$	<i>false</i>	<i>true</i>	$a \vee b$	$a \wedge b$	$\alpha(f_i) = true$	<i>true</i>
#SAT	$\mathbb{N}$	0	1	$a + b$	$a \cdot b$	$\alpha(f_i) = 1$	1
BDD	$BDD(\mathcal{V})$	$bdd(0)$	$bdd(1)$	$a \vee_{bdd} b$	$a \wedge_{bdd} b$	$\alpha(f_i) = bdd(b_i)$	$\neg_{bdd} bdd(b_i)$
sensitivity	$\mathbb{R}[X]$	0	1	$a + b$	$a \cdot b$	$\alpha(f_i) = x$ or $\alpha(f_i) \in [0, 1]$	$1 - \alpha(f_i)$
gradient	$\mathbb{R}_{\geq 0} \times \mathbb{R}$	$(0, 0)$	$(1, 0)$	Eq. (9)	Eq. (10)	Eq. (11)	Eq. (12)

$$(p, S) \otimes (q, T) = (p \cdot q, \{I \cup J \mid I \in S, J \in T\}) \quad (7)$$

$$(p, S) \oplus (q, R) = \begin{cases} (p, S) & \text{if } p > q \\ (q, R) & \text{if } p < q \\ (p, S \cup R) & \text{if } p = q \end{cases} \quad (8)$$

$$(a_1, a_2) \oplus (b_1, b_2) = (a_1 + b_1, a_2 + b_2) \quad (9)$$

$$(a_1, a_2) \otimes (b_1, b_2) = (a_1 \cdot b_1, a_1 \cdot b_2 + a_2 \cdot b_1) \quad (10)$$

$$\alpha(f_i) = \begin{cases} (p_i, 1) & \text{if } i = k \\ (p_i, 0) & \text{if } i \neq k \end{cases} \quad (11)$$

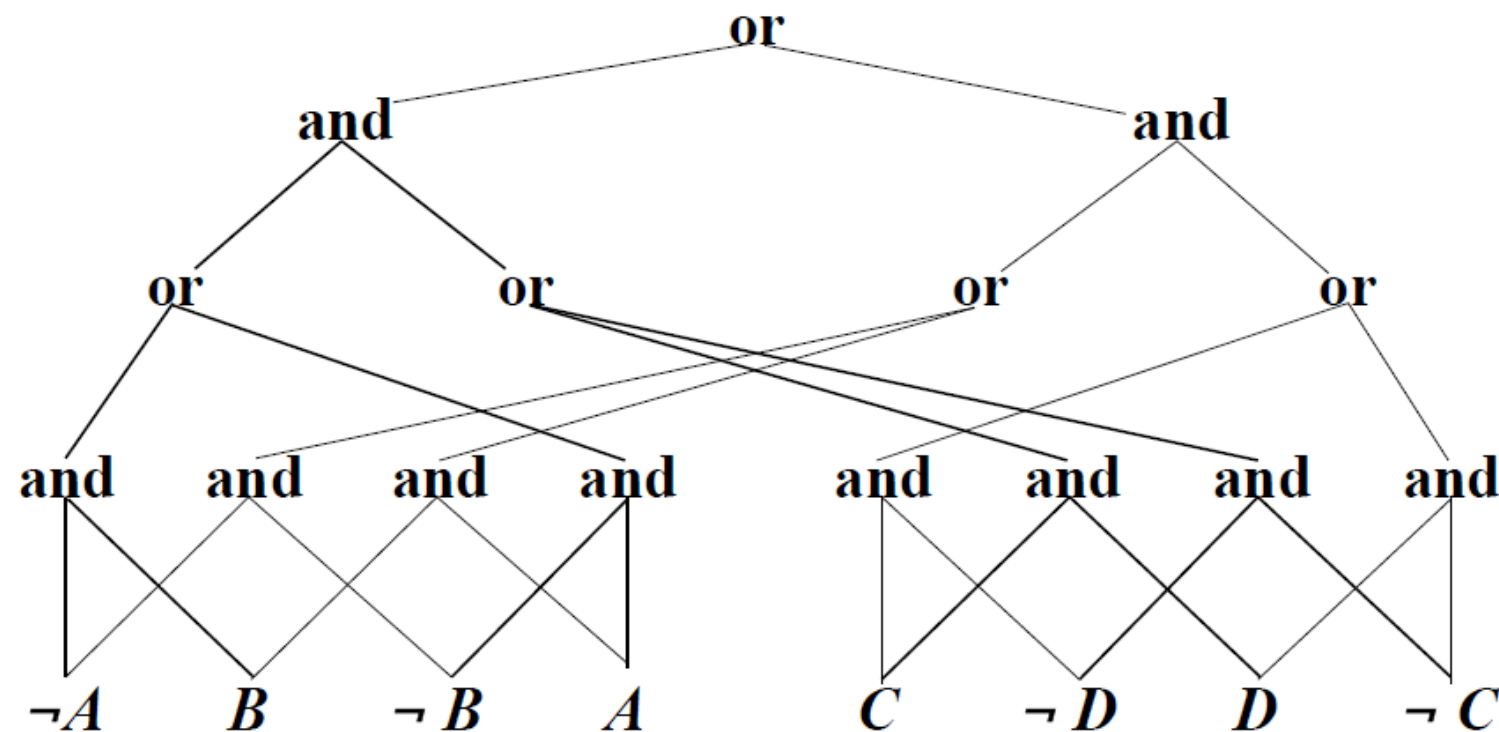
$$\alpha(\neg f_i) = \begin{cases} (1 - p_i, -1) & \text{if } i = k \\ (1 - p_i, 0) & \text{if } i \neq k \end{cases} \quad (12)$$

Knowledge Compilation from SAT to WMC

Types of Circuits

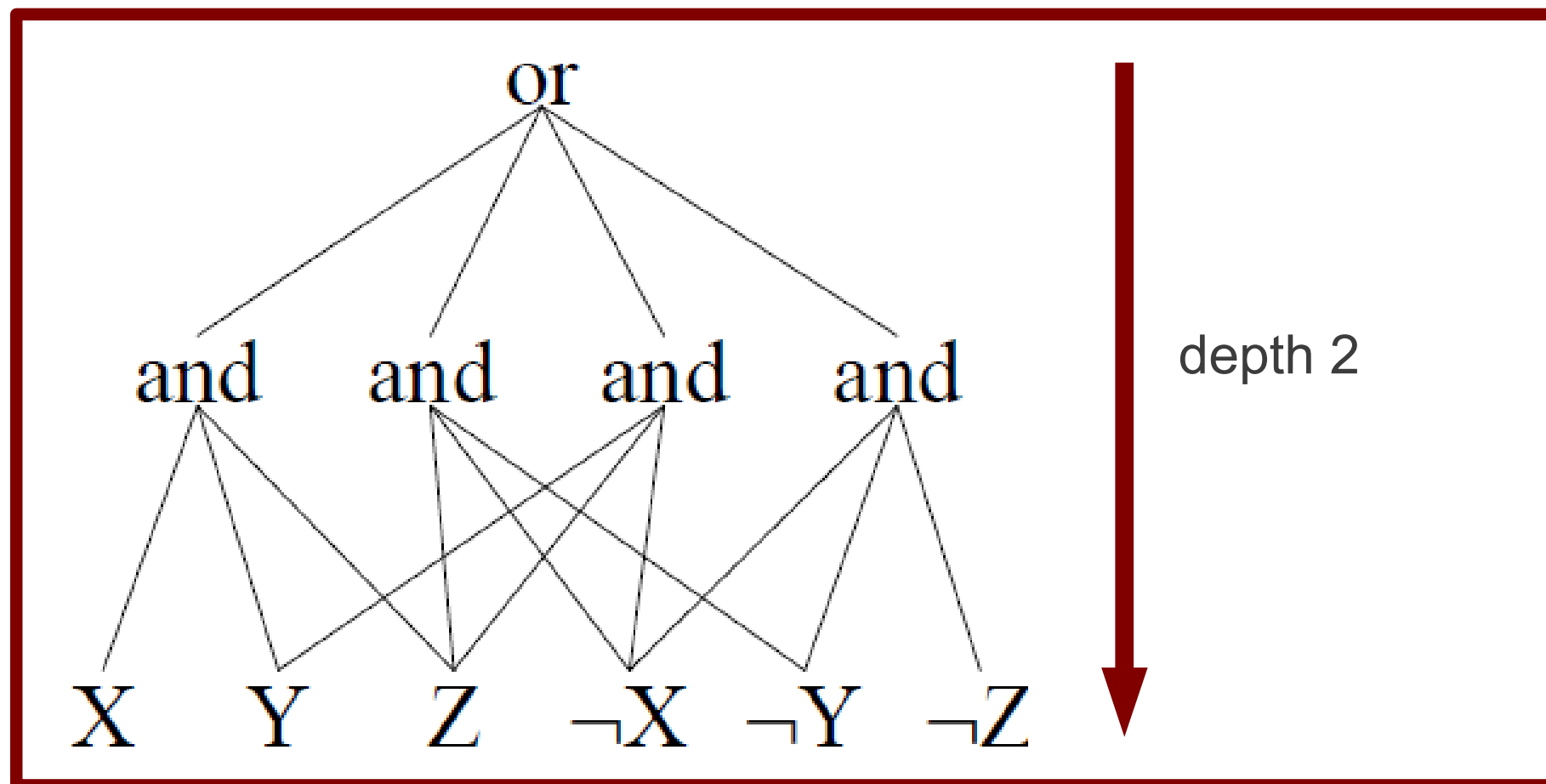
Negation normal form (NNF):

- only AND and OR allowed, furthermore leafs are literals (variables or their negation)
- we shall interpret this as logical circuits ...
- fill in values for the variables and compute whether the assignment satisfies the circuit or not.



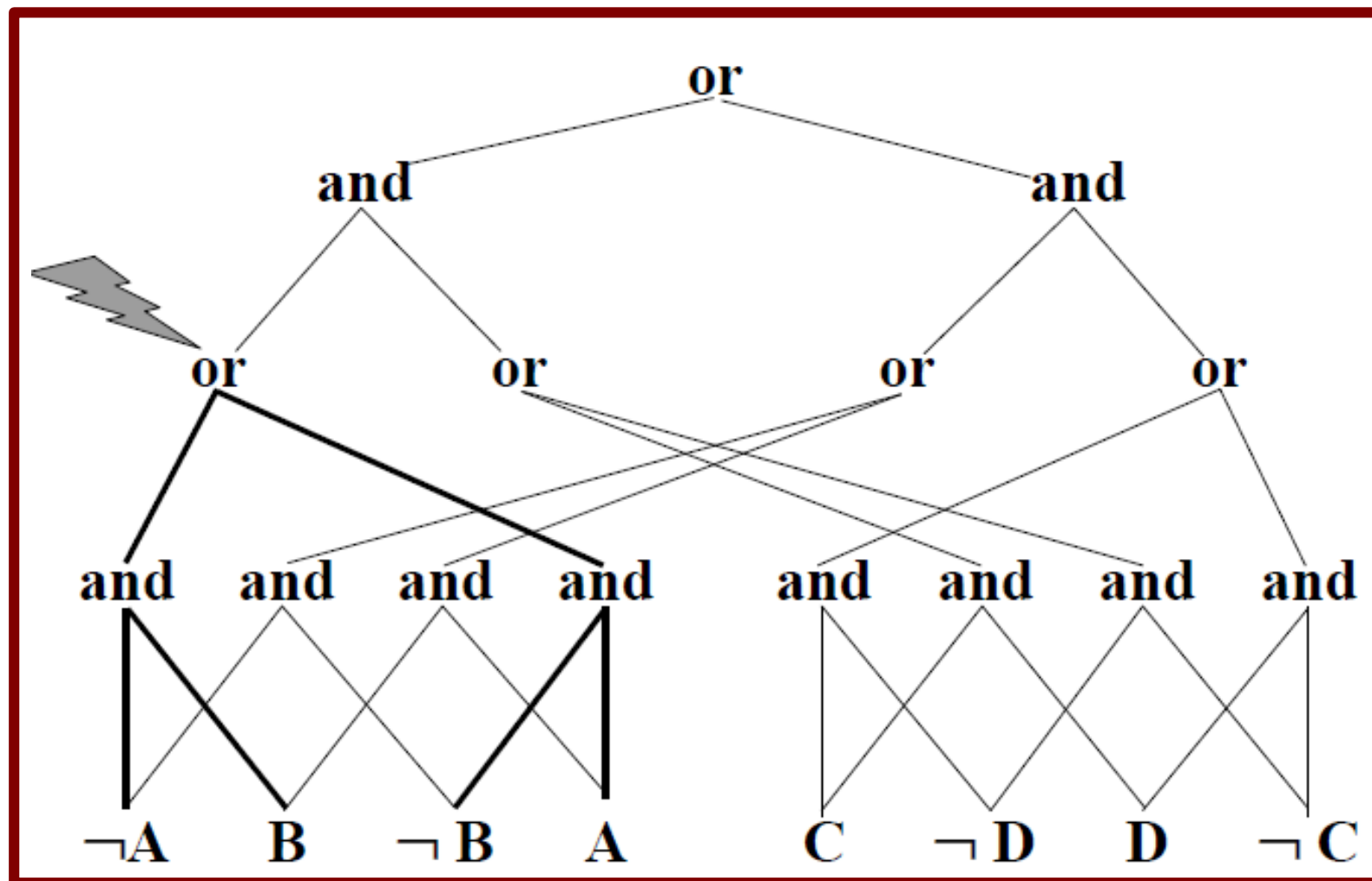
NNFs: special forms

- CNF (clausal normal form) = depth 2 and AND at the top, OR at next level
- DNF (disjunctive normal form) = depth 2 and OR at the top, AND at next level
- DNF example



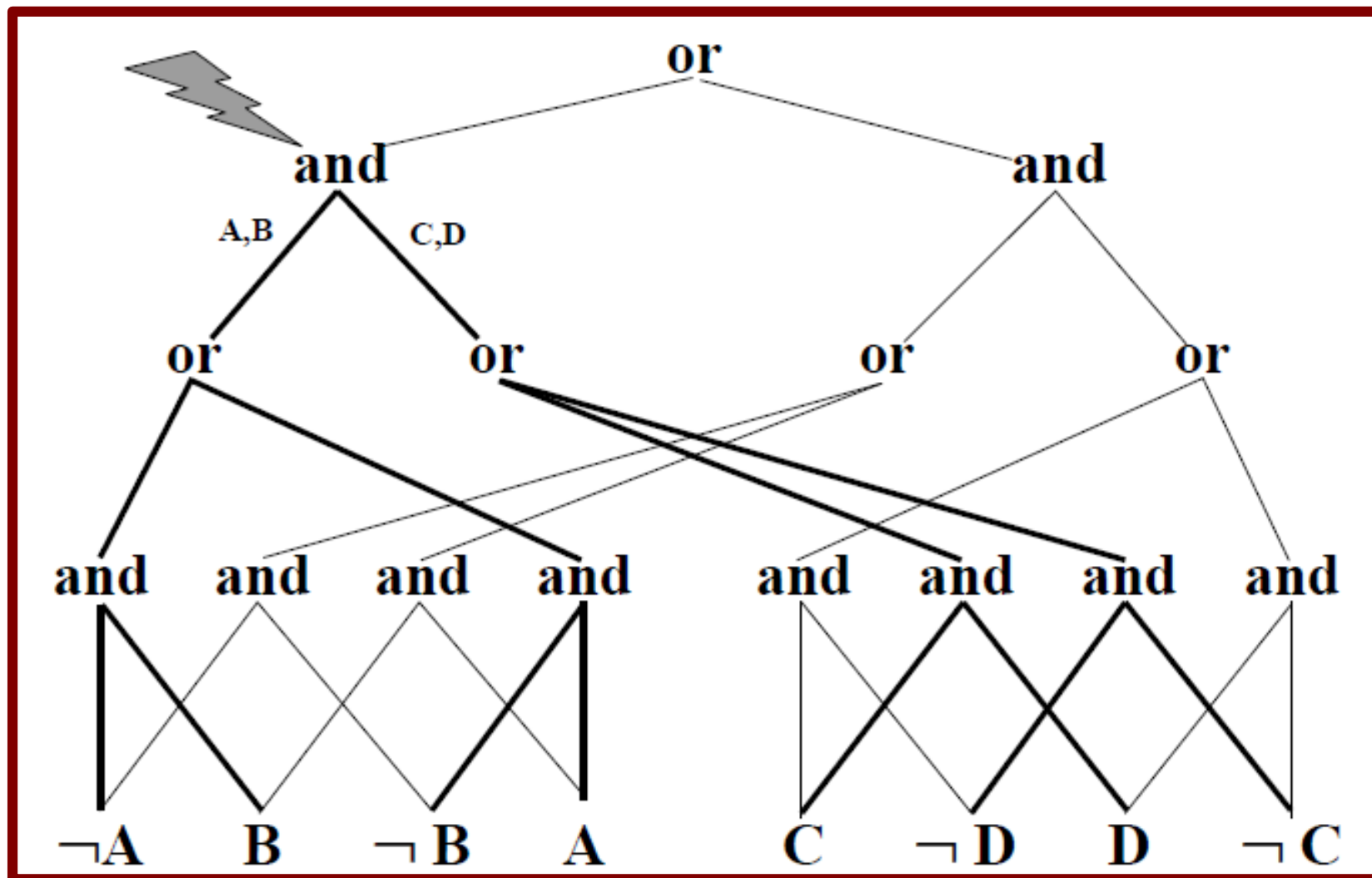
NNFs: special forms

- d-NNF is deterministic NNF
- determinism = disjuncts are logically disjoint;
(two disjuncts cannot be true at the same time)
- example – marked node is deterministic



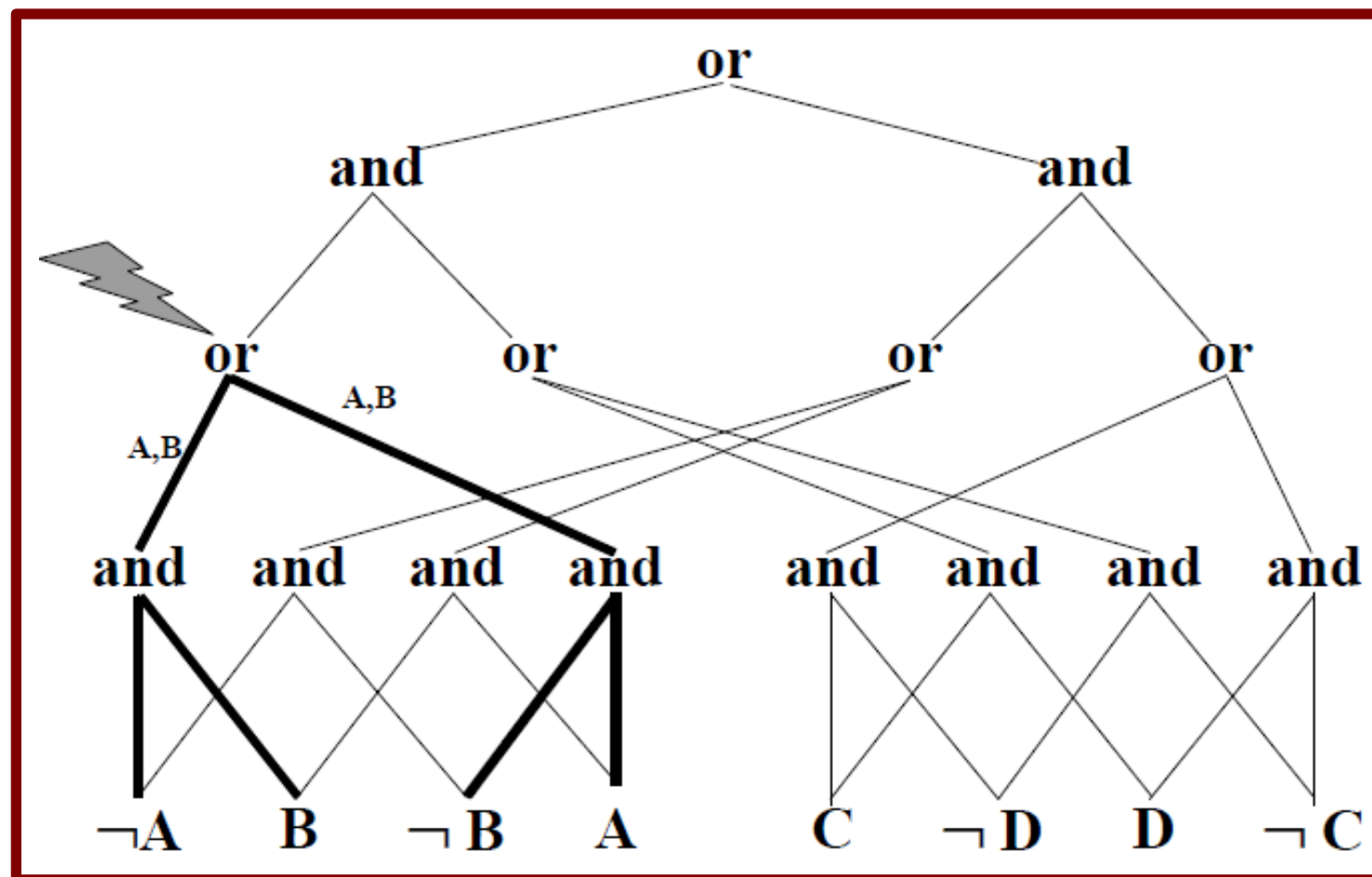
NNFs: special forms

- DNNF is decomposable NNF
- decomposable = conjuncts do not share variables
- example – marked node is decomposable



NNFs: special forms

- s-NNF is smooth NNF,
- smooth = disjuncts mention the same set of variables of the Bayesian net (we do not consider the θ here)
- **REMARK** it is easy to smooth a circuit, just introduce the missing variables
- **REMARK 2** the NNF on this slide is also smooth,

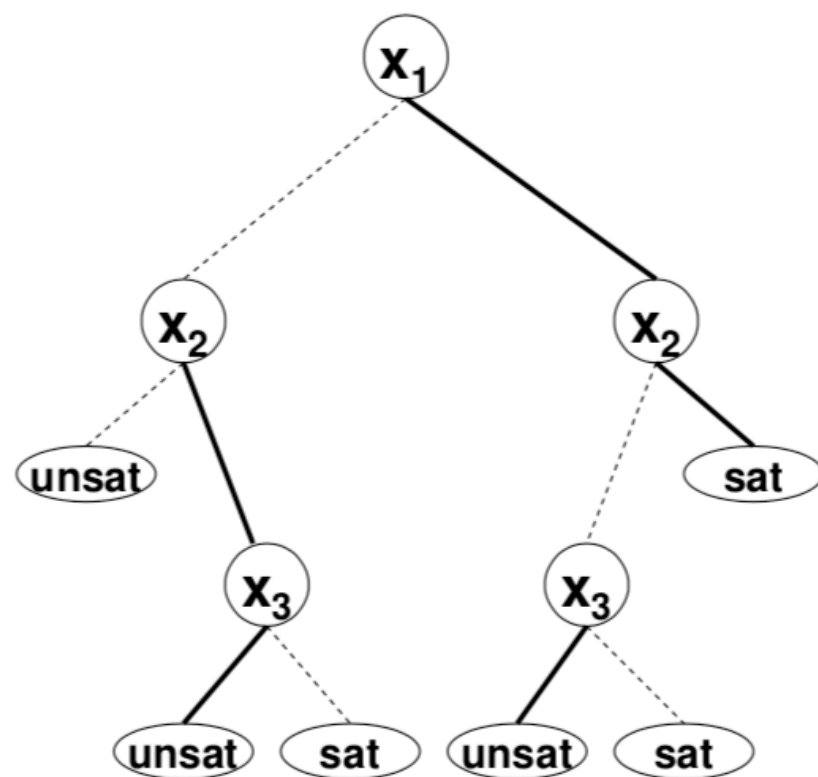


NNFs : special forms

- Why is this important ?
 - remember that we replace and by $\times$ and or by $+$
- decomposability allows to rewrite $P(A \wedge B) = P(A) \times P(B)$
- determinism allows to rewrite $P(A \vee B) = P(A) + P(B)$
- without smoothness you might not take into account all variables
- these eqs. do not hold for arbitrary formula A and B !

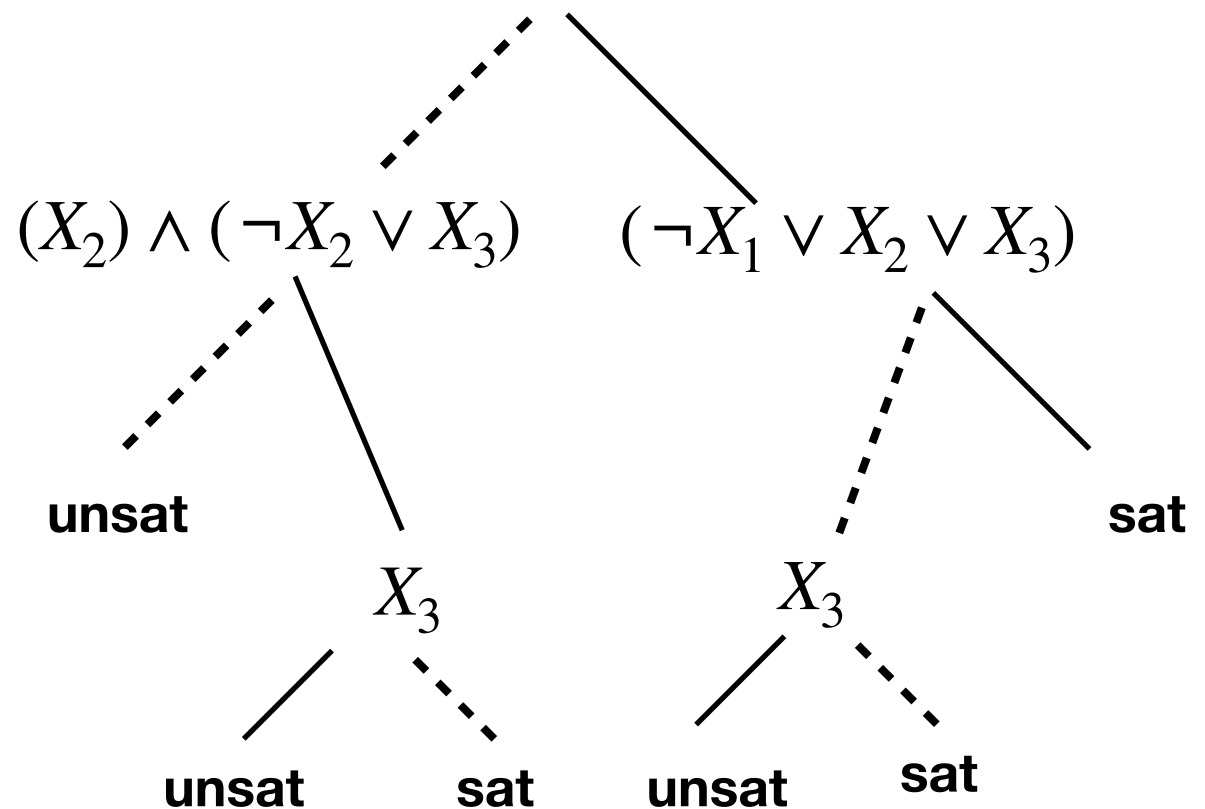
NNFs and Decision Nodes

DPLL

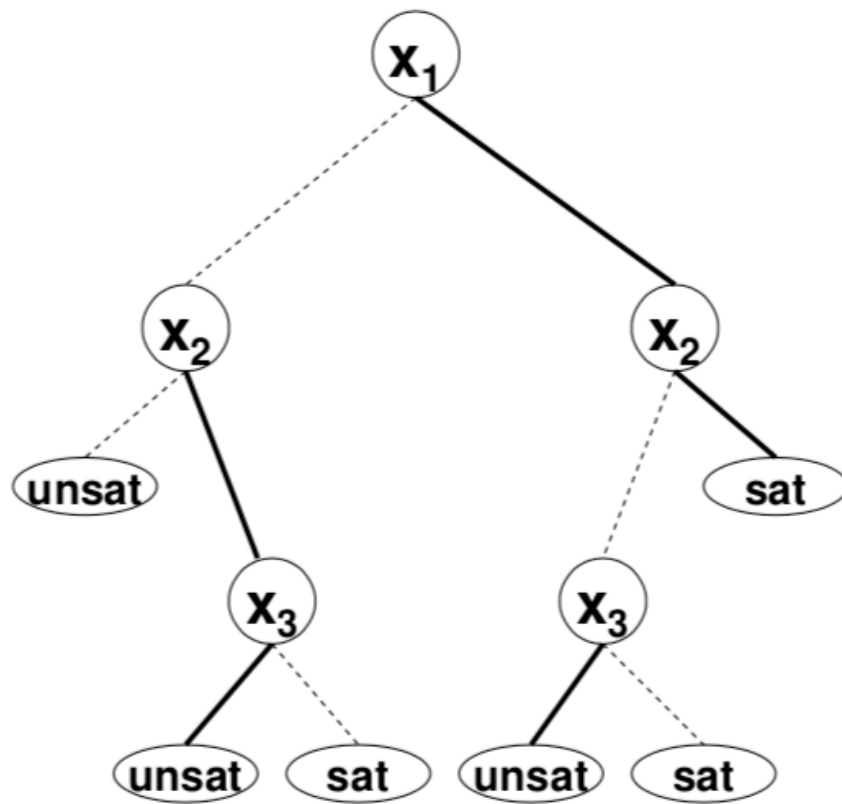


(a) Termination tree

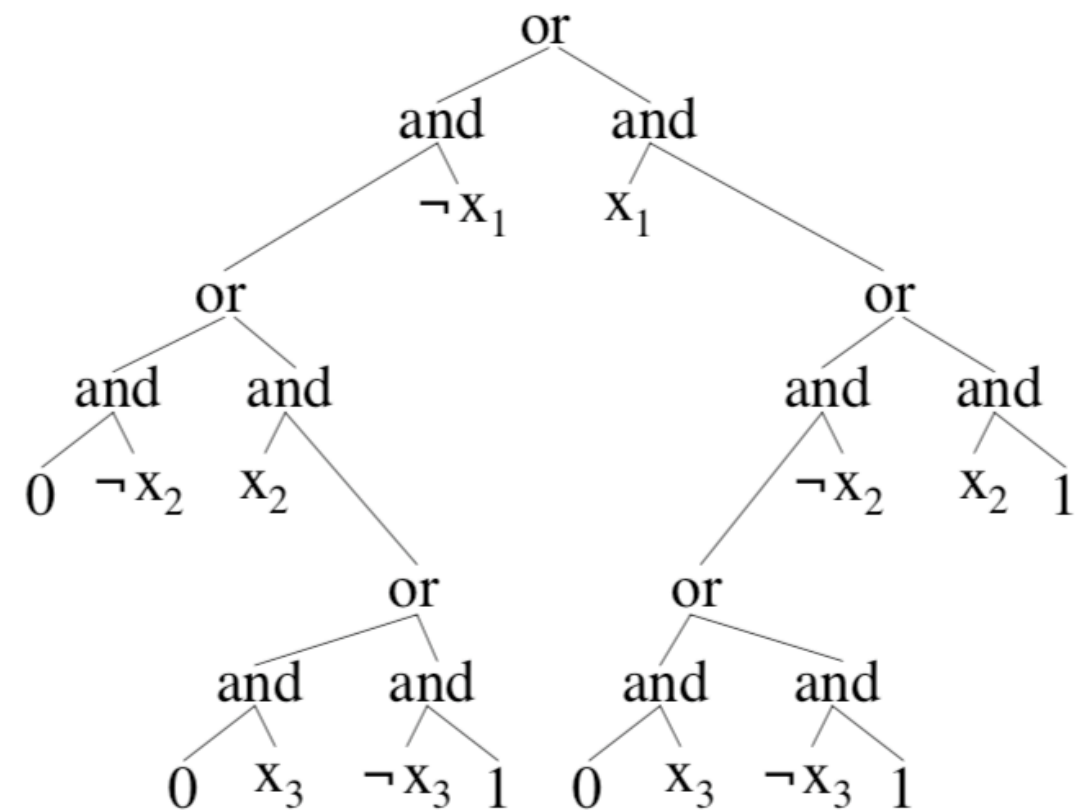
$$(X_1 \vee X_2) \wedge (X_1 \vee \neg X_2 \vee X_3) \wedge (\neg X_1 \vee X_2 \vee X_3)$$



NNFs and Decision Nodes



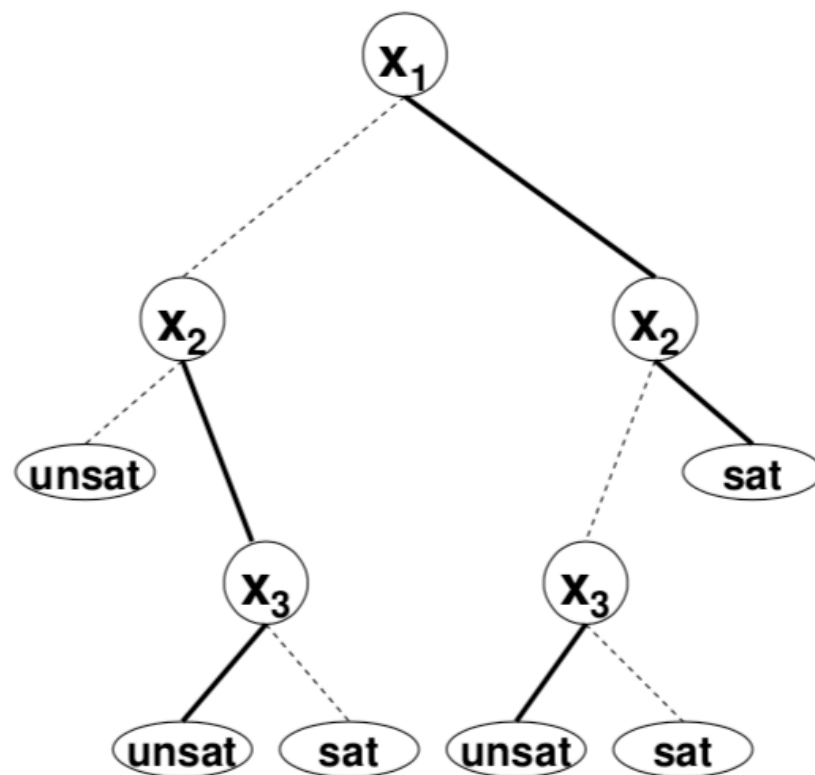
(a) Termination tree



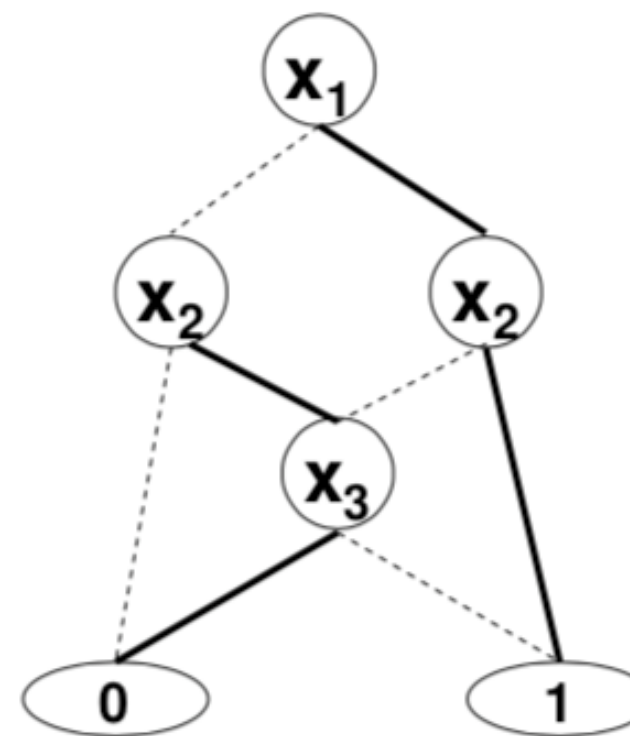
(b) Equivalent NNF circuit

from [Huang & Darwiche, JAIR 2007]

NNFs and Decision Nodes



(a) Termination tree



(c) OBDD

**DPLL +
Caching**

**Ordered =
same order vars
along all paths
(not really used here)**

Identifying and Exploiting Isomorphisms

from [Huang & Darwiche, JAIR 2007]

NNFs : special forms

- Why is this important ?
 - remember that we replace and by $\times$ and or by $+$
- decomposability allows to rewrite $P(A \wedge B) = P(A) \times P(B)$
- determinism allows to rewrite $P(A \vee B) = P(A) + P(B)$
- without smoothness you might not take into account all variables
- these eqs. do not hold for arbitrary formula A and B !

NNFs and Decision Nodes

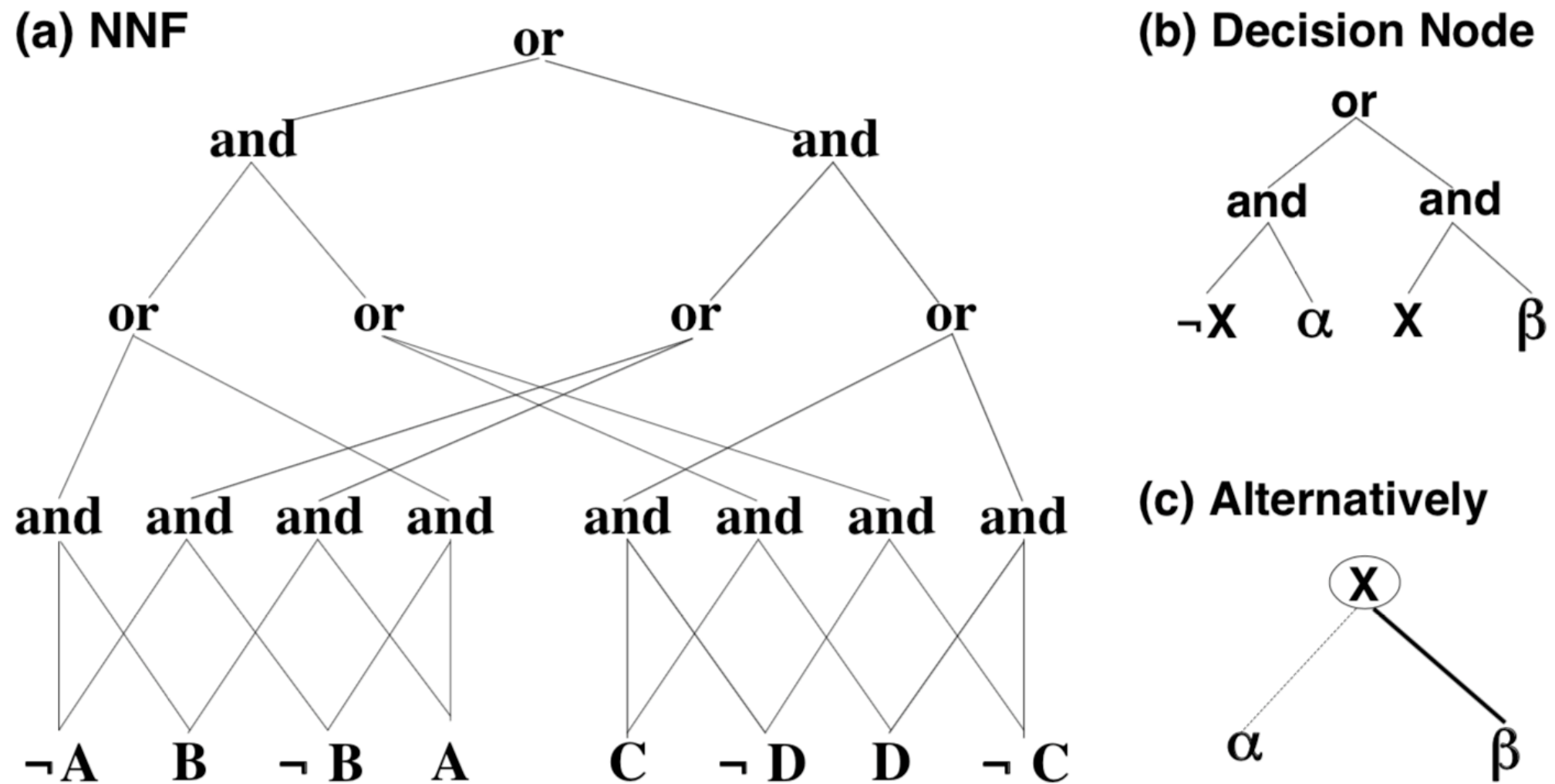
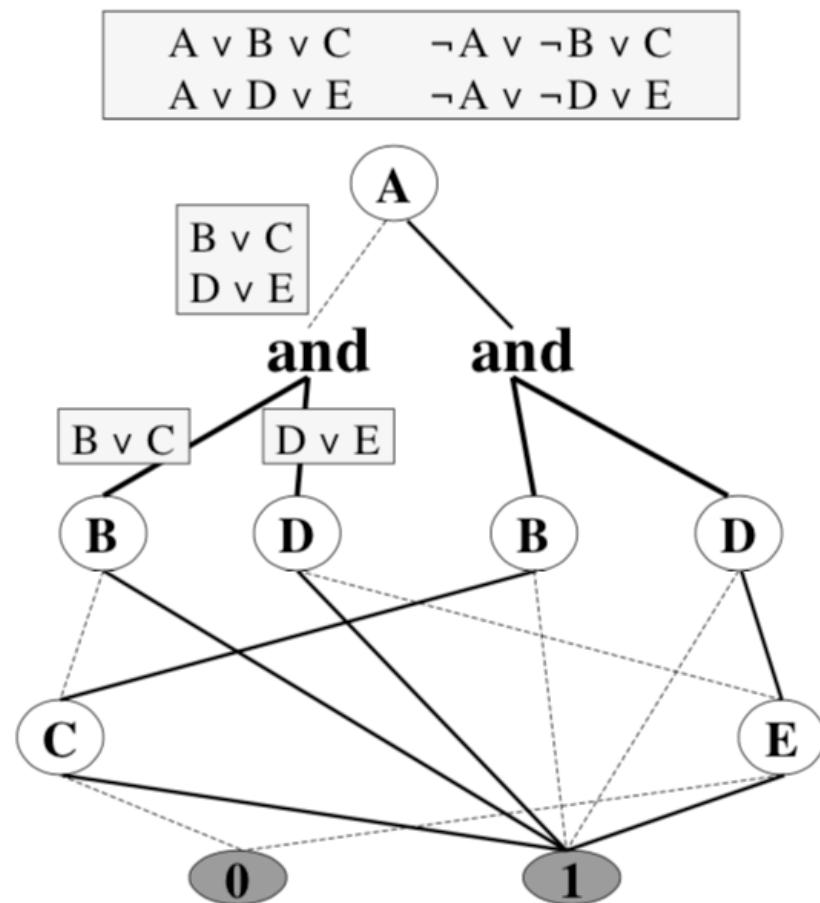


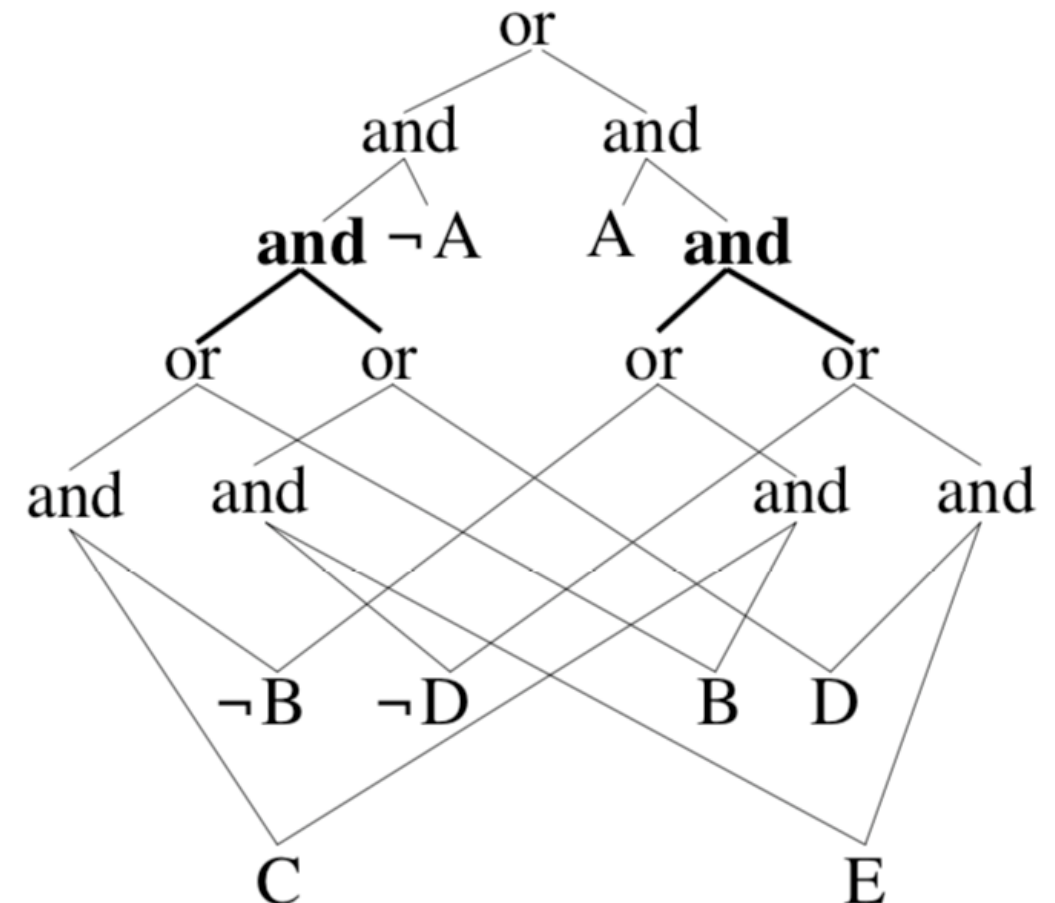
Figure 1: An NNF circuit and a decision node.

from [Huang & Darwiche, JAIR 2007]

Compiling into d-DNNF



(a) Mixture of decision and conjunction nodes



(b) Equivalent NNF circuit

Figure 5: Trace of exhaustive DPLL with decomposition.

from [Huang & Darwiche, JAIR 2007]

Compiling into d-DNNF

see [Huang Darwiche] for details

```
procedure DPLL(S : set of clauses):  
  if S is empty  
    return true sink  
  if S contains an empty clause  
    return false sink  
  components = disjoint partitions of S  
  if |components| > 1 then  
    forall C ∈ components do  
      conjunctions = conjunctions ∪ DPLL(C)  
  else select v ∈ Vars  
    St := S where v = 1 (making the variable true)  
    Sf := S where v = 0 (making the variable false)  
    return node(v, DPLL(St), DPLL(Sf))  
  
disjoint = components do not share variables
```

The caching is not described explicitly in the algorithm !

after [Huang & Darwiche, JAIR 2007]

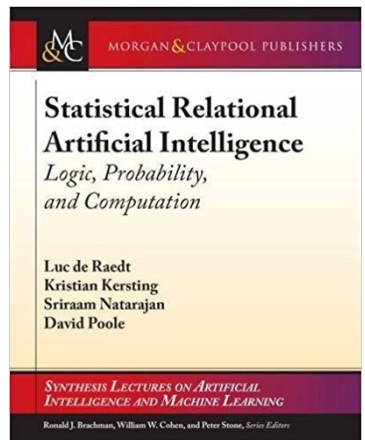
Conclusions

Key Message



**StarAI and NeSy share similar problems
and thus similar solutions apply**

See also [De Raedt et al., IJCAI 20; Marra et al. AIJ 24]



Some Dimensions

1. Proof vs Model based
2. Directed vs Undirected
3. Semantics
4. Logic vs Probability vs Neural

Many questions to ask

- What properties should integrated representations satisfy ?
 - Should one representation take over ? (As in most approaches to NeSy — push the logic inside and forget about it afterwards)
 - Should one build a pipeline or an interface between the integrated representations ?
 - Should one have the originals as a special case ?
 - (yes we believe you should be able to do all what you can do with the original representations)

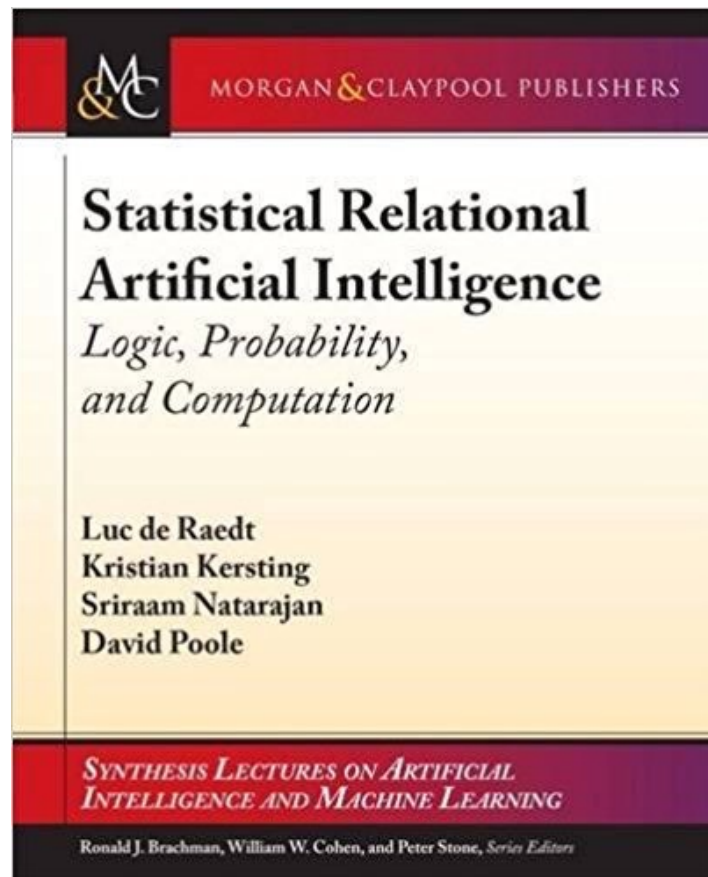
Many questions to ask

- Which learning and reasoning techniques apply ?
 - Can you still reason logically / probabilistically ?
 - Can you still apply standard learning methods (like gradient descent) ?
 - Is everything explainable / trustworthy ?
- How to evaluate integrated representations ?
 - $1 + 1 = 3$?
 - Can they do what the originals can do, and can they do more ?
 - Can they do something different ?

Challenges

- For NeSy,
 - scaling up
 - which models to use
 - real life applications
 - peculiarities of neural nets
 - logical inference can be expensive
- **This is an excellent area for starting researchers / PhDs**

Key references



From statistical relational to neurosymbolic artificial intelligence: A survey

Giuseppe Marra^{a,*}, Sebastijan Dumančić^c, Robin Manhaeve^a, Luc De Raedt^{a,b}

^a KU Leuven, Department of Computer Science and Leuven.AI, Belgium

^b Örebro University, Center for Applied Autonomous Sensor Systems, Sweden

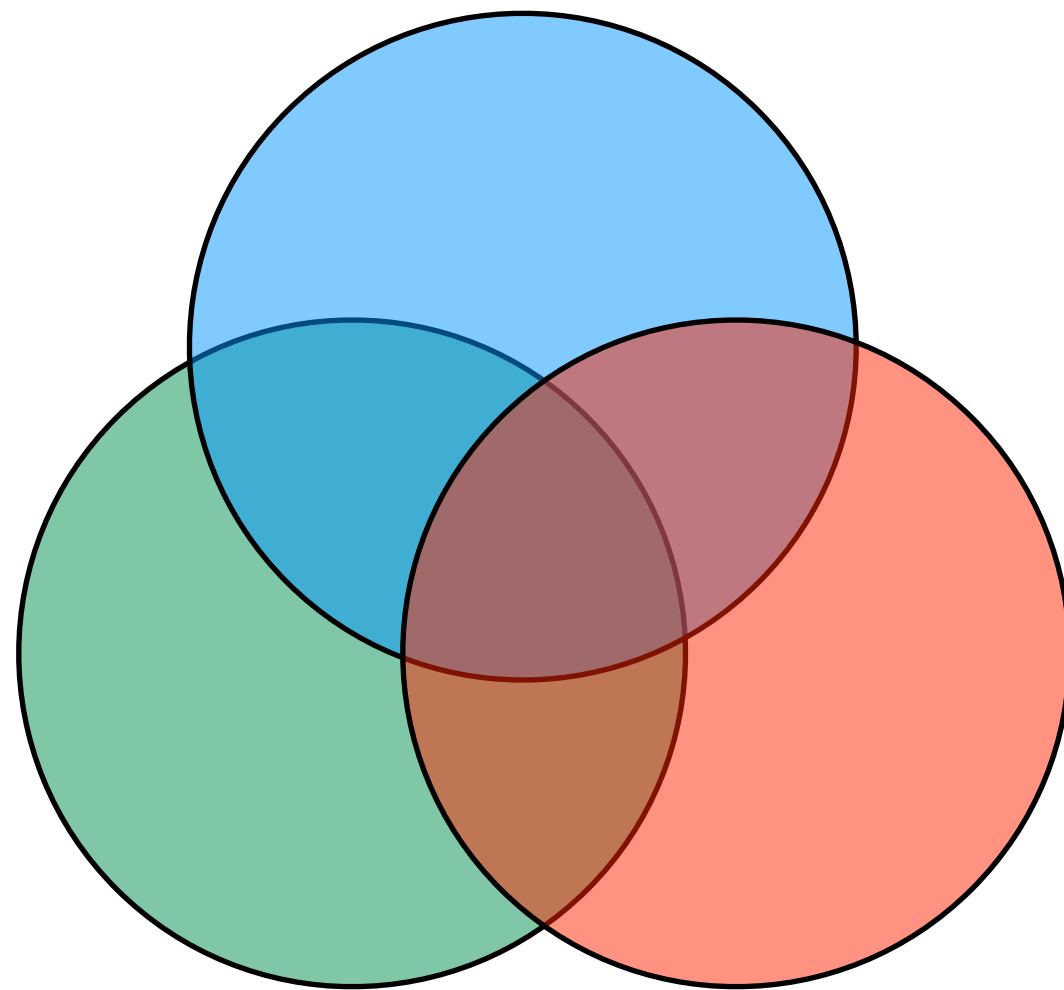
^c Delft University of Technology, Department of Software Technology, Netherlands

ARTICLE INFO

Keywords:
Neurosymbolic AI
Statistical relational AI
Learning and reasoning
Probabilistic logics

ABSTRACT

This survey explores the integration of learning and reasoning in two different fields of artificial intelligence: neurosymbolic and statistical relational artificial intelligence. Neurosymbolic artificial intelligence (NeSy) studies the integration of symbolic reasoning and neural networks, while statistical relational artificial intelligence (StarAI) focuses on integrating logic with probabilistic graphical models. This survey identifies seven shared dimensions between these two subfields of AI. These dimensions can be used to characterize different NeSy and StarAI systems. They are concerned with (1) the approach to logical inference, whether model or proof-based; (2) the syntax of the used logical theories; (3) the logical semantics of the systems and their extensions to facilitate learning; (4) the scope of learning, encompassing either parameter or structure learning; (5) the presence of symbolic and subsymbolic representations; (6) the degree to which systems capture the original logic, probabilistic, and neural paradigms; and (7) the classes of learning tasks the systems are applied to. By positioning various NeSy and StarAI systems along these dimensions and pointing out similarities and differences between them, this survey contributes fundamental concepts for understanding the integration of learning and reasoning.



THANKS

References

- Tarek R. Besold, Artur S. d'Avila Garcez, Sebastian Bader, Howard Bowman, Pedro M. Domingos, Pascal Hitzler, Kai-Uwe Kühnberger, Luís C. Lamb, Daniel Lowd, Priscila Machado Vieira Lima, Leo de Penning, Gadi Pinkas, Hoifung Poon, and Gerson Zaverucha. Neural-symbolic learning and reasoning: A survey and interpretation. CoRR, abs/1711.03902, 2017.
- Matko Bošnjak, Tim Rocktäschel, Jason Naradowsky, and Sebastian Riedel. Programming with a differentiable forth interpreter. In ICML, 2017.
- William W. Cohen, Fan Yang, and Kathryn Mazaitis. Tensorlog: Deep learning meets probabilistic dbs. CoRR, abs/1707.05390, 2017.
- Andrew Cropper. Playgol: Learning programs through play. In IJCAI 2019, 2019.
- Andrew Cropper and Stephen H. Muggleton. Metagol system. <https://github.com/metagol/metagol>, 2016.
- Adnan Darwiche. Sdd: A new canonical representation of propositional knowledge bases. In IJCAI, 2011.
- Artur S. d'Avila Garcez, Marco Gori, Luís C. Lamb, Luciano Serafini, Michael Spranger, and Son N. Tran. Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning. FLAP, 6, 2019.
- Luc De Raedt, Sebastian Dumančić., Robin Manhaeve and Giuseppe Marra. From statistical relational to neuro-symbolic artificial intelligence. In IJCAI 2020.
- Luc De Raedt. Logical and relational learning. Springer, 2008.
- Luc De Raedt, Kristian Kersting, Sriraam Natarajan, and David Poole. Statistical Relational Artificial Intelligence: Logic, Probability, and Computation. Morgan & Claypool Publishers, 2016.

References

- Luc De Raedt and Angelika Kimmig. Probabilistic (logic) programming concepts. *Machine Learning*, 100, 2015.
- Luc De Raedt, Robin Manhaeve, Sebastijan Dumanžić, Thomas Demeester, and Angelika Kimmig. Neuro-symbolic= neural+ logical+probabilistic. In *NeSy @ IJCAI*, 2019.
- Thomas Demeester, Tim Rocktäschel, and Sebastian Riedel. Lifted rule injection for relation embeddings. In *EMNLP*, 2016.
- Michelangelo Diligenti, Marco Gori, and Claudio Saccà. Semantic-based regularization for learning and inference. *Artif. Intell.*, 244, 2017.
- Ivan Donadello, Luciano Serafini, and Artur S. d’Avila Garcez. Logic tensor networks for semantic image interpretation. In *IJCAI*, 2017.
- Honghua Dong, Jiayuan Mao, Tian Lin, Chong Wang, Lihong Li, and Denny Zhou. Neural logic machines. In *ICLR*, 2019.
- Sebastijan Dumanžić, Tias Guns, Wannes Meert, and Hendrik Blockeel. Learning relational representations with auto-encoding logic programs. In *IJCAI*, 2019.
- Kevin Ellis, Lucas Morales, Mathias Sablé-Meyer, Armando Solar-Lezama, and Josh Tenenbaum. Learning libraries of subroutines for neurally-guided bayesian program induction. In *NeurIPS*, 2018.
- Kevin Ellis, Maxwell I. Nye, Yewen Pu, Felix Sosa, Josh Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a REPL. *CoRR*, abs/1906.04604, 2019.
- Richard Evans and Edward Grefenstette. Learning explanatory rules from noisy data. *J. Artif. Intell. Res.*, 61, 2018.

References

- Daan Fierens, Guy Van den Broeck, Joris Renkens, Dimitar Shterionov, Bernd Gutmann, Ingo Thon, Gerda Janssens, and Luc De Raedt. Inference and learning in probabilistic logic programs using weighted boolean formulas. *Theory and Practice of Logic Programming*, 15, 2015.
- Peter Flach. *Simply Logical: Intelligent Reasoning by Example*. John Wiley & Sons, Inc., 1994.
- Nir Friedman, Lise Getoor, Daphne Koller, and Avi Pfeffer. Learning probabilistic relational models. In *IJCAI*, 1999.
- Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. Answer set solving in practice. *Synthesis lectures on artificial intelligence and machine learning*, 6, 2012.
- L. Getoor and B. Taskar, editors. *An Introduction to Statistical Relational Learning*. MIT Press, 2007.
- Francesco Giannini, Michelangelo Diligenti, Marco Gori, and Marco Maggini. On a convex logic fragment for learning and reasoning. *IEEE TFS*, 27, 2018. CV Radhakrishnan et al.: Preprint submitted to Elsevier
- Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. *arXiv preprint arXiv:1704.01212*, 2017.
- Goldman, O., Latcinnik, V., Naveh, U., Globerson, A., & Berant, J.. Weakly-supervised semantic parsing with abstract examples. *ACL 2018*
- Bernd Gutmann, Angelika Kimmig, Kristian Kersting, and Luc De Raedt. Parameter learning in probabilistic databases: A least squares approach. In *ECML&PKDD*, 2008.
- Manfred Jaeger. Model-theoretic expressivity analysis. In Luc De Raedt, Paolo Frasconi, Kristian Kersting, and Stephen Muggleton, editors, *Probabilistic Inductive Logic Programming - Theory and Applications*, volume 4911 of *LNCS*. Springer, 2008.

References

- Ashwin Kalyan, Abhishek Mohta, Oleksandr Polozov, Dhruv Batra, Prateek Jain, and Sumit Gulwani. Neural-guided deductive search for real-time program synthesis from examples. In ICLR, 2018.
- Kristian Kersting and Luc De Raedt. Bayesian logic programming: Theory and tool. In L. Getoor and B. Taskar, editors, An introduction to Statistical Relational Learning. MIT Press, 2007.
- Stanley Kok and Pedro Domingos. Learning the structure of markov logic networks. In ICML, 2005.
- Daphne Koller and Nir Friedman. Probabilistic Graphical Models - Principles and Techniques. MIT Press, 2009.
- Marco Lippi and Paolo Frasconi. Prediction of protein beta-residue contacts by markov logic networks with grounding-specific weights. Bioinform., 25, 2009.
- John W Lloyd. Foundations of logic programming. Springer Science & Business Media, 2012.
- Daniel Lowd and Pedro Domingos. Efficient weight learning for markov logic networks. In ECML&PKDD, 2007.
- Robin Manhaeve, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. Deepprolog: Neural probabilistic logic programming. In NeurIPS, 2018.
- Jiayuan Mao, Chuang Gan, Pushmeet Kohli, Joshua B. Tenenbaum, and Jiajun Wu. The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision. In ICLR, 2019.
- Giuseppe Marra, Michelangelo Diligenti, Francesco Giannini, Marco Gori, and Marco Maggini. Relational neural machines. In ECAI, 2020.
- Giuseppe Marra and Ondrej Kuželka. Neural markov logic networks. CoRR, abs/1905.13462, 2019.

References

- Pasquale Minervini, Matko Bošnjak, Tim Rocktäschel, Sebastian Riedel, and Edward Grefenstette. Differentiable reasoning on large knowledgebases and natural language. In AAI, 2020.
- Pasquale Minervini, Thomas Demeester, Tim Rocktäschel, and Sebastian Riedel. Adversarial sets for regularising neural link predictors. In UAI, 2017.
- Stephen Muggleton. Stochastic logic programs. Advances in inductive logic programming, 32, 1996.
- Maxwell I. Nye, Armando Solar-Lezama, Josh Tenenbaum, and Brenden M. Lake. Learning compositional rules via neural program synthesis. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual, 2020.
- David Poole. The independent choice logic and beyond. In Probabilistic Inductive Logic Programming - Theory and Applications, volume 4911 of LNCS. Springer, 2008.
- Matthew Richardson and Pedro M. Domingos. Markov logic networks. Machine Learning, 62, 2006.
- Tim Rocktäschel and Sebastian Riedel. End-to-end differentiable proving. In NIPS, 2017.
- Tim Rocktäschel, Sameer Singh, and Sebastian Riedel. Injecting logical background knowledge into embeddings for relation extraction. In NAACL HLT, 2015.
- Stuart Russell. Unifying logic and probability. Communications of the ACM, 58, 2015.

References

- Xujie Si, Mukund Raghothaman, Kihong Heo, and Mayur Naik. Synthesizing datalog programs using numerical relaxation. In IJCAI, 2019.
- Lazar Valkov, Dipak Chaudhari, Akash Srivastava, Charles A. Sutton, and Swarat Chaudhuri. Houdini: Lifelong learning as program synthesis. In NeurIPS, 2018.
- Guy Van den Broeck, Dan Suciu, et al. Query processing on probabilistic data: A survey. Foundations and Trends® in Databases, 7, 2017.
- Emile van Krieken, Erman Acar, and Frank van Harmelen. Analyzing differentiable fuzzy logic operators. CoRR, abs/2002.06100, 2020.
- Wenya Wang and Sinno Jialin Pan. Integrating deep learning with logic fusion for information extraction. CoRR, abs/1912.03041, 2019.
- Wang, P., Wu, Q., Shen, C., Hengel, A. V. D., & Dick, A. . Explicit knowledge-based reasoning for visual question answering. IJCAI 2017
- Leon Weber, Pasquale Minervini, Jannes Münchmeyer, Ulf Leser, and Tim Rocktäschel. Nlprolog: Reasoning with weak unification for question answering in natural language. In ACL, 2019.
- Jingyi Xu, Zilu Zhang, Tal Friedman, Yitao Liang, and Guy Van den Broeck. A semantic loss function for deep learning with symbolicknowledge. In ICMML, 2018.
- Fan Yang, Zhilin Yang, and William W Cohen. Differentiable learning of logical rules for knowledge base reasoning. In NIPS, 2017.
- Zhun Yang, Adam Ishay, and Joohyung Lee. Neurasp: Embracing neural networks into answer set programming. In Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI, pages 1755–1762,

References

- Kexin Yi, Jiajun Wu, Chuang Gan, Antonio Torralba, Pushmeet Kohli, and Josh Tenenbaum. Neural-symbolic vqa: Disentangling reasoning from vision and language understanding. In NeurIPS, 2018.
- Lotfi A Zadeh. Fuzzy logic and approximate reasoning. Synthese, 30(3-4):407–428, 1975.
- Pedro Zuidberg Dos Martires, Vincent Derkinderen, Robin Manhaeve, Wannes Meert, Angelika Kimmig, and Luc De Raedt. Transforming probabilistic programs into algebraic circuits for inference and learning. In Program Transformations for ML Workshop at NeurIPS, 2019.
- Gustav Šourek, Vojtech Aschenbrenner, Filip Zelezný, Steven Schockaert, and Ondrej Kuželka. Lifted relational neural networks: Efficient learning of latent relational structures. J. Artif. Intell. Res., 62, 2018